

Universitatea Națională de Știință și Tehnologie POLITEHNICA București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

*Studiul și implementarea soluțiilor cloud scalabile
pentru aplicații web*

Proiect de diplomă

prezentat ca cerință parțială pentru obținerea titlului de *Inginer*
în domeniul *Calculatoare și tehnologia informației*
programul de studii de licență
Ingineria informației

Conducător științific
Ș.L.Dr.Ing. George Valentin STOICA

Absolvent
Maria-Cătălina IONESCU

2025

TEMA PROIECTULUI DE DIPLOMĂ
a studentului **IONESCU D. Maria-Cătălina, 444A**

1. Titlul temei: Studiul și implementarea soluțiilor cloud scalabile pentru aplicații web

2. Descrierea temei:

Proiectul își propune studiul, implementarea și testarea soluțiilor cloud scalabile pentru aplicații web. Se va realiza testarea de performanță și se va analiza impactul volumului de date și a numărului de utilizatori simultani asupra performanței arhitecturilor implementate. Aplicația Web va avea o structură multi-tenant, multiutilizator, multiresursă, și va implementa mecanisme tranzacționale de acces la resurse (aplicație Web la alegere: rezervări on-line, eCommerce, programări on-line). Deploymentul se va realiza pe o infrastructură de tip cloud (la alegerea studentului).

3. Contribuția originală:

Contribuția studentului va consta în implementarea unei aplicații Web multinivel și adoptarea unor soluții scalabile pentru nivelele front-end, back-end sau nivelul de date de tip load balancer, containerizare sau baze de date multinod.

Se va utiliza o infrastructură On-Prem sau Cloud.

4. Resurse și bibliografie:

5. Data înregistrării temei: 2024-12-17 08:31:58

Conducător(i) lucrare,

Ș.L.Dr.Ing. George Valentin STOICA

Student,

IONESCU D. Maria-Cătălina

Director departament,

Ș.L. dr. ing Bogdan FLOREA

Decan,

Prof. dr. ing. Mihnea UDREA

Cod Validare: **72030a43f0**

Declarație de onestitate academică

Prin prezenta declar că lucrarea cu titlul *Studiul și implementarea soluțiilor cloud scalabile pentru aplicații web*, prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității "Politehnica" din București ca cerință parțială pentru obținerea titlului de *Inginer* în domeniul Inginerie Electronică și Telecomunicații/ Calculatoare și Tehnologia Informației, programul de studii *Ingineria informației* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate. Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare. Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, Septembrie 2025.

Absolvent: Ionescu Maria Cătălina



.....

Student: Videanu Florin-Cătălina

Profesor Coordonator: Vasile Florentin

Cuprins

Lista de figuri	i
Lista de tabele	ii
Lista de acronime	iii
1 Introducere	1
1.1 Prezentarea lucrării	1
1.2 Scopul lucrării	1
1.3 Structura lucrării	2
2 Principii de scalabilitate si tehnologii de implementare	3
2.1 Concepte fundamentale de scalabilitate	3
2.2 Introducere în aplicații web	4
2.3 Arhitecturi comune	4
2.4 Protocolul HTTP	5
2.5 Conexiunea HTTP	9
2.6 Mesaje HTTP	10
2.7 Framework-ul Flask Python	14
2.8 Sistemul de baze de date MySQL	16
2.9 Internet Information Services	18
2.10 Nginx	18
3 Arhitectura si structura aplicatiei	21
3.1 Introducere	21
3.2 Arhitectura generală a sistemului distribuit	21
3.3 Arhitectura bazei de date	23
3.4 Fluxul datelor și funcționarea bazei de date	26
3.5 Asigurarea consistenței datelor prin integritate referențială	27
3.6 Structura internă a aplicației Flask	27
4 Funcționalitățile aplicației	30
4.1 Introducere în mecanismele funcționale ale aplicației	30
4.2 Înregistrarea și autentificarea utilizatorilor	30
4.2.1 Formularul de înregistrare	30
4.2.2 Formularul de autentificare	33
4.2.3 Mecanismele de securitate	34
4.3 Mecanismul de gestiune a sesiunii	36
4.4 Interacțiunea cu baza de date	37
4.5 Pagina principală	38
4.6 Pagina de profil	40
4.7 Modificarea datelor de profil	41
4.8 Panoul de administrare	42

4.9	Fluxul de căutare și recomandări personalizate	44
4.10	Adăugarea unei proprietăți noi	47
4.11	Gestiunea proprietăților personale	49
4.12	Modificarea unei proprietăți existente	50
4.13	Ștergerea unei proprietăți	52
4.14	Procesul de rezervare	52
4.15	Confirmarea și finalizarea rezervării	54
4.16	Pagina de confirmare a rezervării	55
4.17	Vizualizarea și gestionarea rezervărilor	56
4.18	Anularea rezervării	57
4.19	Gestionarea rezervărilor proprietăților personale	57
5	Arhitectura de suport și înaltă disponibilitate	59
5.1	Introducere în arhitectura de suport	59
5.2	Rolul Nginx în contextul arhitecturii implementate	59
5.3	Impactul pozitiv al arhitecturii distribuite asupra aplicației	60
5.4	Importanța replicării MySQL (Master-Slave) în optimizarea performanței	61
5.5	Rolul virtualizării în testarea arhitecturii	62
6	Testarea performanței și analiza scalabilității	64
6.1	Introducere în testarea de performanță	64
6.1.1	Alegerea instrumentului de testare: Locust	64
6.2	Procesul de execuție și monitorizare	66
6.3	Arhitectura mediului de testare	66
6.3.1	Pregătirea și rularea testelor Locust	67
6.4	Scenarii de Testare Implementate	68
6.4.1	Cazul 1: arhitectura monolitică	69
6.4.2	Cazul 2: Arhitectura distribuită	75
6.5	Compararea performanței între arhitectura monolitică și cea distribuită	81
6.6	Concluzii	83
7	Concluzii	85
7.1	Contribuții personale	85
7.2	Dificultăți în timpul lucrării	85
7.3	Cum poate fi dezvoltată lucrarea mai departe	86
	Anexă	87
	Bibliografie	92

Listă de figuri

2.1	Structura user-agent-ului	6
2.2	Componentele URL-ului	6
2.3	Tipul de protocol URL	7
2.4	Numele resursei și portul	7
2.5	Calea către resursă	8
2.6	Parametrii	8
2.7	Ancoră URL	8
2.8	Diagrama modului de funcționare a server-ului web	9
2.9	Diagrama conexiunii HTTP	10
2.10	Structura mesajelor HTTP	11
2.11	Exemplu de cerere HTTP	11
2.12	Structura cererii HTTP	11
2.13	Structura unei linii de cerere HTTP	12
2.14	Structura răspunsului HTTP	13
2.15	Componentele liniei de stare	13
2.16	Exemplu de response și representation header	14
3.1	Diagrama arhitecturală a componentelor sistemului distribuit	22
3.2	Structura bazei de date „rezervări”	23
4.1	Formularul de înregistrare a utilizatorilor	31
4.2	Formularul de autentificare a utilizatorilor	33
4.3	Pagina home - utilizator neautentificat	38
4.4	Pagina home - utilizator autentificat	39
4.5	Interfața paginii de profil	40
4.6	Pagina Editează Profilul	41
4.7	Panou Administrator	43
4.8	Formularul de autentificare a utilizatorilor	45
4.9	Rezultatele căutării	45
4.10	Calculatorul de disponibilitate și recomandări	46
4.11	Formular pentru înregistrarea unei proprietăți noi	47
4.12	Editarea avansată a caracteristicilor camerelor	48
4.13	Prezentarea generală a proprietăților utilizatorului	49
4.14	Interfața de editare a detaliilor generale ale proprietății	50
4.15	Gestionarea dinamică a tipurilor de camere pentru o proprietate	51
4.16	Lista proprietăților rezultate în urma căutării	53
4.17	Pagina cu detalii și opțiuni de rezervare pentru o proprietate	53
4.18	Formular de rezervare	54
4.19	Pagina de confirmare a rezervării	55
4.20	Interfața paginii „Rezervările mele”	56
4.21	Vizualizarea rezervărilor pentru o proprietate	58
6.1	Utilizarea resurselor după 2-3 minute de test	70
6.2	Resurse utilizate după 15 minute	71

6.3	Număr de cereri pe secundă (RPS)	71
6.4	Timpii de răspuns (ms)	72
6.5	Numărul de utilizatori	72
6.6	Resurse utilizate după 2-3 minute	76
6.7	Număr de cereri pe secundă (RPS)	77
6.8	Timpi de răspuns (ms)	77
6.9	Numărul de utilizatori	77

Listă de tabele

6.1	Timp de răspuns agregat pentru toate cererile	73
6.2	Timp de răspuns pentru endpoint-ul POST /login.	73
6.3	Timp de răspuns pentru endpoint-ul POST /edit-profile.	74
6.4	Timp de răspuns pentru GET /my-reservations.	74
6.5	Performanța medie a cererilor pentru ambele seturi de teste	78
6.6	Timp de răspuns pentru POST /login	79
6.7	Timp de răspuns pentru cazul 2	79
6.8	Timp de răspuns pentru GET /my-reservations	80
6.9	Configurarea mediului în cele două cazuri de testare	81
6.10	Timp de răspuns pentru endpoint-ul POST /login.	82
6.11	Timp de răspuns agregat pentru toate cererile	83

Lista de acronime

CI/CD Continuous Integration / Continuous Delivery - Integrare Continuă / Livrare Continuă

CRUD Create, Read, Update, Delete - Creare, Citire, Actualizare, Ștergere

CSS Cascading Style Sheets

DCL Data Control Language - Limbaj de Control al Datelor

DDL Data Definition Language - Limbaj de Definire a Datelor

DML Data Manipulation Language - Limbaj de Manipulare a Datelor

HTML HyperText Markup Language

HTTP Hypertext Transfer Protocol

HTTPS Hypertext Transfer Protocol Secure

IaaS Infrastructure as a Service - Infrastructură ca Serviciu

IIS Internet Information Services

IoT Internet of Things - Internetul Lucrurilor

IP Internet Protocol

JS JavaScript

KPI Key Performance Indicator - Indicator Cheie de Performanță

LAN Local Area Network - Rețea Locală

RBAC Role-Based Access Control - Controlul Accesului Bazat pe Roluri

RPS Requests Per Second - Cereri pe Secundă

SGBD Sistem de Gestiune a Bazelor de Date

SHA Secure Hash Algorithm - Algoritm Securizat de Hashing

SQL Structured Query Language

TCL Transaction Control Language - Limbaj de Control al Tranzacțiilor

TCP Transmission Control Protocol

URI Uniform Resource Identifier

URL Uniform Resource Locator

VM Virtual Machine - Mașină Virtuală

WSGI Web Server Gateway Interface

1 Introducere

1.1 Prezentarea lucrării

Cerințele privind performanța aplicațiilor web sunt în continuă creștere datorită gamei extinse de dispozitive inteligente și IoT folosite de către utilizatori pentru a-și îmbunătăți viața de zi cu zi. Numărul mare de dispozitive care accesează în mod concomitent un site web duce la un volum copleșitor de cereri către servere pentru a accesa informația cerută, ceea ce poate cauza o experiență întârziată sau chiar imposibilă a utilizatorului. Datorită acestor circumstanțe, traficul de date crește, iar aplicațiile web trebuie să facă față la un flux considerabil de informații.

O soluție esențială pentru această provocare este adoptarea unor arhitecturi scalabile, care permit sistemului să-și adapteze resursele software și hardware pentru a menține performanța sub sarcină. Această lucrare își propune să studieze, să implementeze și să testeze astfel de soluții în contextul unei aplicații web de rezervări online. Proiectul analizează practic beneficiile unei arhitecturi distribuite, care utilizează un load balancer (Nginx) și o bază de date multi-nod (MySQL Master-Slave), comparând performanța acesteia cu o arhitectură monolitică tradițională.

1.2 Scopul lucrării

Lucrarea „**Studiul și implementarea soluțiilor cloud scalabile pentru aplicații web**” își propune studierea, implementarea și testarea soluțiilor software scalabile pentru o aplicație web, utilizând framework-ul Flask Python, sistemul de gestiune a bazelor de date MySQL și serverul Nginx ca load balancer și reverse proxy.

În mod ideal, scalabilitatea presupune gestionarea optimă a resurselor, astfel încât, la un număr mare de cereri, aplicația să le gestioneze eficient, fără a compromite viteza de răspuns. Scalabilitatea software, implementată prin soluții adecvate, asigură funcționarea eficientă a aplicației la un volum de utilizatori variabil. Aceasta, folosită concomitent cu scalabilitatea hardware, poate aduce rezultate impresionante în gestionarea traficului, oferind utilizatorilor o experiență cât mai fluidă [1].

Scopul principal al acestei lucrări este de a analiza capacitatea aplicației de a răspunde la un număr mare de cereri simultane, urmărind îmbunătățirea performanței și a disponibilității prin implementarea unei soluții scalabile ce include un load balancer și o bază de date distribuită de tip Master-Slave.

Aplicația, o platformă web destinată rezervărilor, a fost dezvoltată și testată într-un mediu de lucru compus dintr-o stație de lucru principală (Host) și două mașini virtuale (VM 1, VM 2), interconectate într-o rețea IP/LAN privată. La nivel arhitectural, s-a implementat o bază de date distribuită de tip Master-Slave pentru a optimiza operațiile de citire-scriere.

Deși tema proiectului vizează soluțiile cloud, pentru implementare și testare s-a optat pentru o infrastructură On-Premise virtualizată. Această decizie a fost motivată de nevoia unui control total asupra mediului, de flexibilitatea în configurare și de posibilitatea de a simula

scenarii de eșec într-un mod controlat și reproductibil, fără costurile asociate unui provider de cloud public. Această abordare simulează un mediu de tip IaaS, permițând studiul și aplicarea principiilor fundamentale de scalabilitate și înaltă disponibilitate, care stau la baza soluțiilor cloud moderne.

Obiectivele specifice ale acestei lucrări sunt:

- Dezvoltarea unei aplicații web complete folosind framework-ul Flask Python, care să includă rute HTTP pentru funcționalități de bază precum: autentificare, managementul utilizatorilor, administrarea proprietăților și rezervări.
- Implementarea arhitecturii scalabile la nivel de aplicație prin intermediul unui sistem de load balancing cu Nginx, care va distribui cererile între cele două instanțe de server (Host și VM 2), crescând astfel puterea de procesare.
- Optimizarea performanței la nivelul bazei de date, prin configurarea unei arhitecturi Master-Slave. Aceasta îmbunătățește timpii de răspuns și asigură redundanța datelor prin direcționarea operațiunilor de **scriere** (INSERT, UPDATE, DELETE) către serverul **Master** și a operațiunilor de **citire** (SELECT) către serverul **Slave**.

Semnificația practică a acestei lucrări constă în evidențierea rezultatelor obținute în urma aplicării soluțiilor scalabile. Rezultatele oferă o perspectivă clară asupra comportamentului unei aplicații web sub un volum semnificativ de cereri și demonstrează cum pot fi îmbunătățite performanța și costurile de operare.

1.3 Structura lucrării

Lucrarea prezintă proiectul unei aplicații web scalabile, realizate cu tehnologiile Flask, MySQL și Nginx, evidențiind soluții care permit gestionarea eficientă a unui număr mare de utilizatori și volume mari de date. Capitolul întâi introduce contextul și motivația proiectului, precum și obiectivele urmărite, printre care se numără dezvoltarea unei platforme de rezervări și optimizarea performanței. Capitolul doi abordează fundamentele teoretice, prezentând arhitecturile software, protocolul HTTP și tehnologiile utilizate. Capitolul trei descrie arhitectura sistemului și modul în care serverele, mașinile virtuale și baza de date interacționează. În capitolul patru sunt detaliate funcționalitățile aplicației, de la autentificare și gestionarea profilului până la fluxurile de rezervare și administrarea proprietăților. Capitolul cinci evidențiază rolul Nginx și al replicării MySQL în asigurarea scalabilității și rezilienței sistemului. Capitolul șase prezintă testele de performanță și analiza comparativă între arhitecturile monolitică și distribuită, demonstrând avantajele celei din urmă. Lucrarea se încheie cu concluzii care sintetizează rezultatele obținute, dificultățile întâmpinate și direcțiile viitoare de dezvoltare.

2 Principii de scalabilitate si tehnologii de implementare

Acest capitol introduce fundamentele aplicațiilor web, explicând conceptele esențiale precum scalabilitatea și realizând o comparație între principalele arhitecturi software, monolitică și N-Tier. De asemenea, se oferă o descriere detaliată a protocolului HTTP, acoperind componentele acestuia, modul de stabilire a conexiunii și structura mesajelor de cerere și răspuns.

Ulterior, capitolul se concentrează pe tehnologiile utilizate în implementarea proiectului. Logica de business este dezvoltată cu micro-framework-ul Flask Python care comunică cu o bază de date MySQL, scalabilă printr-un sistem de replicare Master-Slave. Întregul sistem este gestionat de serverele web Nginx și IIS, care asigură funcțiile de reverse proxy și load balancing, contribuind astfel la o distribuție eficientă a traficului și la menținerea performanței în arhitectura distribuită.

2.1 Concepte fundamentale de scalabilitate

Scalabilitatea reprezintă capacitatea unui sistem de a-și gestiona resursele în mod adaptiv și eficient pentru a crește performanța în funcție de cerințe. Aceasta se clasifică în funcție de tipul de resurse:

- **Scalabilitatea software** se referă la abilitatea unei aplicații de a-și menține performanța la o variație bruscă a volumului de utilizatori, date sau operațiuni [2].
- **Scalabilitatea hardware** reprezintă capacitatea unui sistem de a-și mări puterea de procesare prin adăugarea de noi resurse hardware [3].

O componentă cheie în obținerea scalabilității la nivel de date este utilizarea unei baze de date distribuite. Aceasta este o bază de date ale cărei componente sunt distribuite pe mai multe calculatoare interconectate, permițând o gestionare mai eficientă a unui volum mare de interogări și asigurând o disponibilitate ridicată a datelor.

În cadrul acestui proiect, scalabilitatea este realizată printr-o arhitectură distribuită ce integrează atât scalabilitatea software, prin utilizarea micro-framework-ului Flask pentru gestionarea logicii de business, cât și scalabilitatea hardware, prin replicarea Master-Slave a bazei de date MySQL și folosirea serverelor Nginx și IIS pentru distribuirea uniformă a încărcării și coordonarea traficului. Această combinație permite sistemului să răspundă eficient la creșterea numărului de utilizatori și a volumului de date, menținând performanța și disponibilitatea ridicate.

2.2 Introducere în aplicații web

O **aplicație web** este o aplicație de tip software accesată prin intermediul unui browser web care poate rula pe un server local sau cloud. Aceasta poate fi accesată de pe orice tip de dispozitiv conectat la internet cum ar fi calculatoarele, telefoanele, consolele ș.a.m.d. și este implementată prin intermediul unor limbaje de programare web ca HTML, CSS și JavaScript. Ceea ce diferențiază o aplicație web de cele mobile sau desktop este faptul că poate rula pe unul sau mai multe servere interconectate și poate fi accesată prin intermediul unui simplu browser web.

Principalele caracteristici și funcționalități ale aplicațiilor web se reflectă în mai multe aspecte esențiale, care asigură o experiență optimă utilizatorilor și o funcționare eficientă a sistemului. Unul dintre aceste aspecte este scalabilitatea, care garantează capacitatea aplicației de a gestiona un număr mare de utilizatori și de a optimiza performanța prin implementarea unor soluții adaptabile la cerințele acestora. De asemenea, accesibilitatea asigură faptul că orice utilizator, indiferent de locație sau de caracteristicile tehnice ale dispozitivului, poate accesa aplicația printr-un dispozitiv conectat la internet. Integrarea cu diverse servicii și aplicații permite personalizarea experienței utilizatorilor și extinderea funcționalităților, sporind astfel interacțiunea acestora cu aplicația. În plus, actualizarea eficientă facilitează lansarea rapidă a noilor versiuni, după ce acestea au trecut testele necesare, fără a necesita intervenția directă a utilizatorului, actualizările realizându-se automat pe server.

2.3 Arhitecturi comune

Dezvoltarea aplicațiilor web moderne a condus la adoptarea unor diverse arhitecturi software, fiecare cu avantaje și dezavantaje specifice în contextul cerințelor de performanță, scalabilitate și mentenanță. Printre cele mai răspândite se numără arhitectura monolitică și arhitectura N-Tier (sau multi-strat).

Arhitectura monolitică

Acest model arhitectural presupune construirea unei aplicații ca o singură unitate logică și indivizibilă, în care toate componentele funcționale: interfața cu utilizatorul, logica de business, stratul de acces la date și alte module sunt integrate într-un singur bloc de cod și rulate ca un proces unic. Deși simplifică dezvoltarea inițială și implementarea pentru aplicații de dimensiuni mici și medii, arhitectura monolitică prezintă provocări semnificative în contextul scalabilității și mentenanței pe termen lung [4].

Arhitectura N-Tier

Această arhitectură logică organizează funcționalitățile unei aplicații în straturi distincte, fiecare cu un rol specific și responsabilități separate. Modelul cel mai răspândit este cel cu trei straturi (3-tier), care include:

- Stratul de prezentare: Responsabil pentru interfața cu utilizatorul (frontend), gestionează afișarea informațiilor și colectarea datelor de intrare. Este adesea implementat folosind

tehnologii web precum HTML, CSS și JavaScript.

- Stratul de logică de business/aplicație: Găzduiește logica principală a aplicației, procesează cererile de la stratul de prezentare, implementează regulile de business și coordonează interacțiunile cu stratul de date. În contextul acestei lucrări, framework-ul Flask Python operează la acest nivel.
- Stratul de date: Asigură persistența datelor, stocarea și gestionarea informațiilor critice ale aplicației. Este reprezentat, în general, de un sistem de gestiune a bazelor de date relaționale, cum ar fi MySQL [5].

Avantajele arhitecturii N-Tier constau în modularitatea ridicată, separarea clară a responsabilităților și posibilitatea de scalare independentă a fiecărui strat. În cadrul proiectului, a fost concepută și implementată o astfel de arhitectură, iar stratul de logică de business a fost ulterior extins într-o arhitectură distribuită multi-nod prin integrarea mecanismelor de load balancing și replicare a bazelor de date.

2.4 Protocolul HTTP

Pentru a construi și a înțelege pe deplin arhitectura scalabilă propusă, este esențială o analiză detaliată a tehnologiilor fundamentale care stau la baza acesteia. Deoarece orice aplicație web modernă se bazează pe „dialogul” constant dintre client și server, protocolul HTTP reprezintă pilonul central al acestei comunicări. În continuare, se va analiza modul de funcționare al protocolului HTTP, a cărui înțelegere este necesară pentru a aborda rolul și integrarea celorlalte componente tehnologice ale sistemului.

Protocolul HTTP este un protocol la nivelul stratului de aplicație, fără reținerea stării (serverul nu reține date despre sesiunea activă), de tip client-server care stă la baza oricărui schimb de date între un client și un server pentru preluarea unor resurse disponibile pe internet, cum ar fi documentele HTML, fișiere audio, video ș.a.m.d. [6]

Un sistem bazat pe HTTP este alcătuit din mai multe componente esențiale. Clientul, de obicei un browser sau o aplicație, inițiază cereri către server. Serverul primește aceste cereri, le procesează și returnează răspunsuri, fie sub formă de fișiere statice, fie de conținut dinamic. Protocolul HTTP/HTTPS stabilește regulile de comunicare, metodele de cerere și formatele de răspuns. Resursele reprezintă fișiere sau date accesibile prin URL-uri, cum ar fi pagini web, imagini sau informații din baze de date. În plus, componentele intermediare, precum proxy-urile, load balancer-ele sau cache-urile, contribuie la optimizarea performanței și la securizarea schimbului de date.

User-agent-ul

User-agent-ul este un header de tip string folosit într-o cerere HTTP pentru a trimite informații despre aplicație, sistemul de operare, furnizorul și versiunea folosită către server [7].

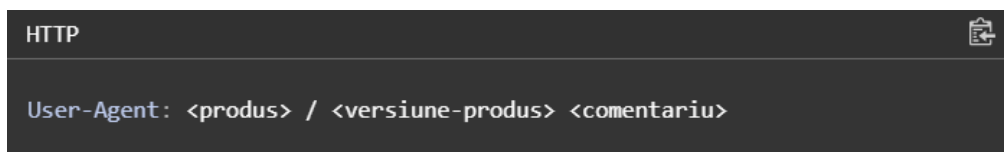


Figura 2.1: Structura user-agent-ului

Ca să afișeze o pagină web, în primă fază, browser-ul trimite o cerere inițială pentru a prelua documentul HTML solicitat de către client. În cea de-a doua fază, acesta analizează fișierul primit și trimite cereri suplimentare pentru a obține informații despre stilizarea paginii (CSS), eventuale scripturi ce trebuiesc executate și sub-resursele aflate în pagină (imagini sau videoclipuri). În fază finală a procesului, browser-ul combină toate aceste resurse pentru a afișa varianta finală a documentului.

Serverul web

Serverul web se ocupă cu livrarea resurselor cerute de către client. Din punct de vedere al structurii hardware, acesta poate fi reprezentat de către o stație de lucru individuală sau de o colecție de servere interconectate în aceeași rețea care au rol în a gestiona schimbul de date stocate (de exemplu: documente HTML, imagini, fișiere JavaScript) cu alte dispozitive [8].

Pe de altă parte, din punct de vedere al structurii software, un server web are rolul de a interpreta adresele URL și de a procesa cererile și răspunsurile transmise prin protocolul HTTP.

Componente URL

URL reprezintă adresa unei resurse unice de pe internet și unul dintre mecanismele folosite de browsere pentru a prelua resurse cum ar fi pagini HTML, documente, imagini ș.a.m.d. Orice URL introdus în bara de adrese trimite o solicitare directă către un server web pentru a accesa resursa specifică, iar în funcție de răspunsul serverului, resursa solicitată fie este afișată, fie un mesaj de eroare este returnat. Conform figurii 2.2, un URL este alcătuit din mai multe componente cum ar fi: tipul de protocolul URL, numele resursei, portul, calea către resursă, parametrii și fragmentul de identificare internă [9].

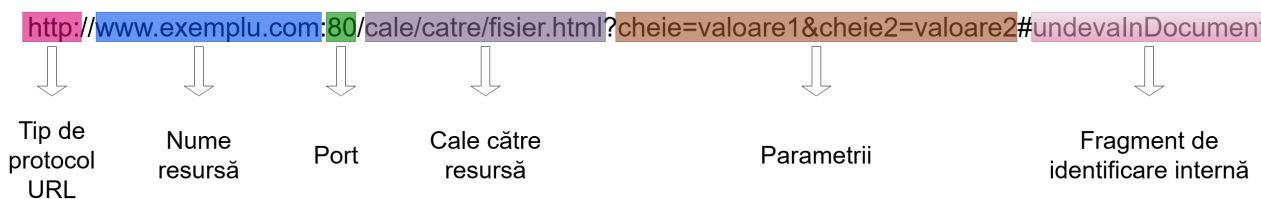


Figura 2.2: Componentele URL-ului

Tipul de protocol URL

Prima componentă regăsită într-un URL reprezintă tipul de protocol pe care browser-ul trebuie să îl folosească pentru a trimite o cerere de acces la resursă. Protocoalele cele mai des întâlnite sunt HTTPS sau HTTP (varianta nesecurizată), dar există cazuri în care este folosit și protocolul `mailto:` care deschide un client de tip mail.



Figura 2.3: Tipul de protocol URL

Numele resursei și portul

Cea de-a doua componentă a URL-ului este formată din numele resursei și port. Numele resursei sau numele domeniului, separat de tipul de protocol prin șirul de caractere `://`, reprezintă adresa paginii web redactată sub forma unui șir de caractere. Orice pagină web are atribuit un IP public unic din intervalul `1.0.0.0 - 9.255.255.255`, numere care sunt greu de reținut sau scris pentru orice persoană. Din acest motiv, cu ajutorul numele domeniului, procesul de accesare a unei pagini web devine mai rapid pentru utilizatori prin „traducerea în limbaj natural” a adreselor IP într-un format ușor de înțeles și accesibil pentru oricine. Portul este o „poartă” tehnică folosită pentru a accesa resursele pe serverul web. În cele mai multe cazuri, declararea portului este omisă deoarece majoritatea serverelor folosesc în mod implicit portul 80 pentru conexiunea HTTP și 443 pentru conexiunea HTTPS, dar în situațiile în care este necesară utilizarea unui alt port, acesta trebuie declarat în mod obligatoriu.

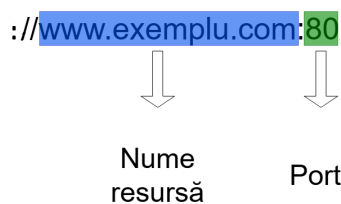


Figura 2.4: Numele resursei și portul

Calea către resursă

Următoarea componentă a unui URL este calea către resursă. Calea indică locația exactă a unei pagini, imagini sau altei resurse aflate pe server.

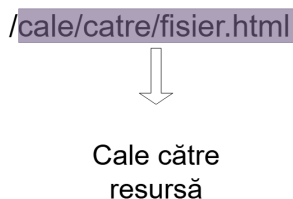


Figura 2.5: Calea către resursă

Parametrii

Cea de-a patra componentă constă într-un atribut unic denumit parametrii. Parametrii sunt o listă de chei și valori folosite, delimitate de simbolul `&` de către serverul web utilizat pentru transmiterea de informații suplimentare către server. Fiecare server web are o listă de chei sau de valori specifice, nu există o regulă specifică în crearea acestora.

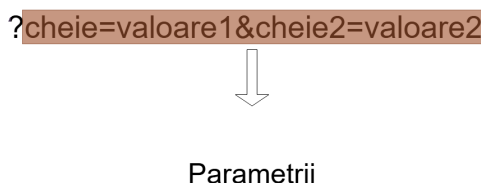


Figura 2.6: Parametrii

Fragment de identificare internă

Ultima componentă ce intră în alcătuirea unui URL se numește fragment de identificare internă. Un fragment de identificare internă, cunoscut și ca ancoră și delimitat de simbolul `#`, are rolul de a afișa o anumită secțiune a resursei. Ancorele funcționează ca un fel de „semn de carte”, acestea indicând aplicației web până unde va trebui să parcurgă documentul ca să ajungă la secțiunea marcată și într-un final să o afișeze.

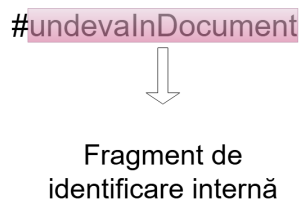


Figura 2.7: Ancoră URL

2.5 Conexiunea HTTP

Accesarea directă a unui server HTTP se realizează prin introducerea URL-ului în bara de adrese, acesta livrând conținutul solicitat utilizatorilor. La nivel fundamental, modul de funcționare presupune ca de fiecare dată când un browser are nevoie de un fișier stocat pe un server web, acesta va iniția o cerere HTTP pentru a primi fișierul respectiv. Odată ce serverul HTTP a recepționat cererea, verifică dacă URL-ul solicitat există. În situația în care resursa este disponibilă, serverul trimite utilizatorilor conținutul corespunzător. În cazul contrar, serverul mai întâi acesta verifică dacă este necesară generarea unui fișier dinamic pentru a răspunde solicitării. În cele din urmă, dacă niciuna dintre opțiuni nu este disponibilă, serverul va returna codul de eroare 404 Not Found.

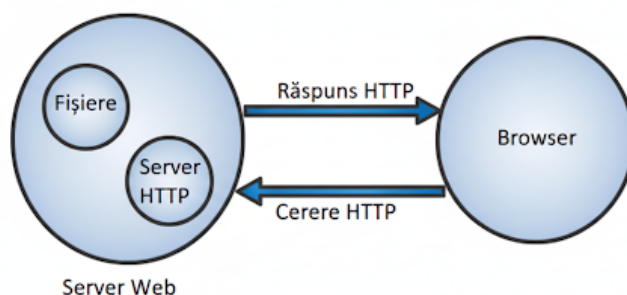


Figura 2.8: Diagrama modului de funcționare a server-ului web

Cum funcționează o conexiune HTTP

O conexiune HTTP este controlată la nivelul stratului de transport, ceea ce implică faptul că se va folosi un protocol de transport pentru gestionarea conexiunii între client și server. Pentru a transmite mesaje cu o rată de pierdere foarte mică s-a optat în folosirea protocolului TCP. Datorită fiabilității sale, se utilizează protocolul TCP care garantează trimiterea mesajelor cu un număr minim erori, cu prețul unei viteze de transfer mai mici.

Primul pas spre a începe o conexiune HTTP îl reprezintă schimbul de cerere/răspuns dintre client și server. Clientul are posibilitatea de a deschide o nouă conexiune, de a folosi o conexiune existentă sau de a deschide mai multe conexiuni TCP către servere. Acest schimb începe odată ce s-a stabilit o conexiune TCP, un proces care trimite unul sau mai multe cereri din partea clientului pentru a primi un răspuns de la server.

Al doilea pas presupune trimiterea mesajului HTTP către server și răspunsul serverului la cererea clientului cu informația cerută. Dacă serverul răspunde cu codul 200 OK, înseamnă că s-a stabilit cu succes conexiunea, iar serverul poate transmite clientului informația cerută.

În cele din urmă, odată ce s-a făcut schimbul de resurse, există posibilitatea de a închide sau reutiliza conexiunea pentru viitoare sesiuni.

2.6 Mesaje HTTP

Mesajele HTTP sunt șiruri de caractere scrise în limbaj natural, simplist, începând cu versiunea HTTP/1.1 până la versiunea HTTP/2. Acestea sunt clasificate în două categorii: cerere (request) și răspuns (response), reprezentând un mecanism de schimb de date între server și un client. O cerere este transmisă de client către server pentru a obține un răspuns, iar serverul furnizează informația solicitată clientului. Crearea și modificarea mesajelor HTTP se datorează API-urilor disponibile în browser, fișierelor de configurație ale proxy-urilor sau serverelor sau altor interfețe software dedicate.

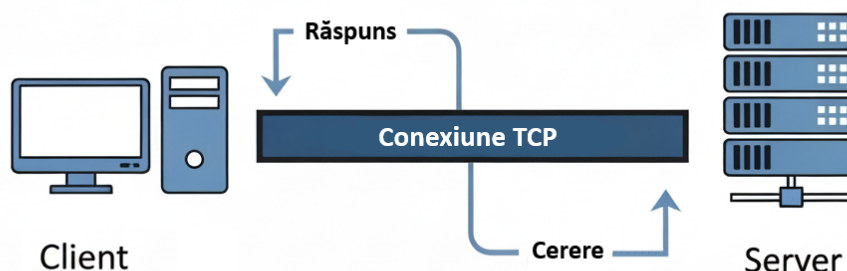


Figura 2.9: Diagrama conexiunii HTTP

Structura mesajelor HTTP

Conform figurii 2.10 se poate observa faptul că mesajele de tip cerere și răspuns prezintă o structură asemănătoare. Partea superioară (head) a mesajului HTTP este alcătuită din linia de start și anteturi, iar corpul, deși conține date transmise între client și server, este o componentă opțională. Partea superioară a mesajului conține informații esențiale pentru interpretarea acestuia. Linia de start, compusă dintr-un singur rând, precizează versiunea HTTP și metoda cererii sau codul de răspuns. Antetul (header) cuprinde metadata folosite pentru a descrie mesajul transmis și facilitează schimbul de informații suplimentare între client și server, fie în cadrul unei cereri, fie al unui răspuns.

În funcție de rolul lor, antetele se pot împărți în mai multe categorii: antetul de cerere (request header) conține informații despre resursa ce urmează a fi preluată; antetul de răspuns (response header) oferă informații despre răspuns, precum locația sau serverul de pe care provine; antetul de reprezentare (representation header) include informații despre corpul resursei, cum ar fi tipul de conținut, compresia sau codarea; iar antetul cu conținut util (payload header) cuprinde informații referitoare la datele din corpul mesajului, descriind caracteristicile acestuia fără a depinde de modul în care este procesat sau trimis. În final, o linie goală indică faptul că metadata mesajului este completă.

Un corp (body) pune la dispoziție datele asociate mesajului, transmiterea căruia este

stabilită încă de la linia de start și din anteturile anterioare. De obicei corpul include fie o metodă trimisă de client către server, fie o resursă redirectionată de către server clientului.

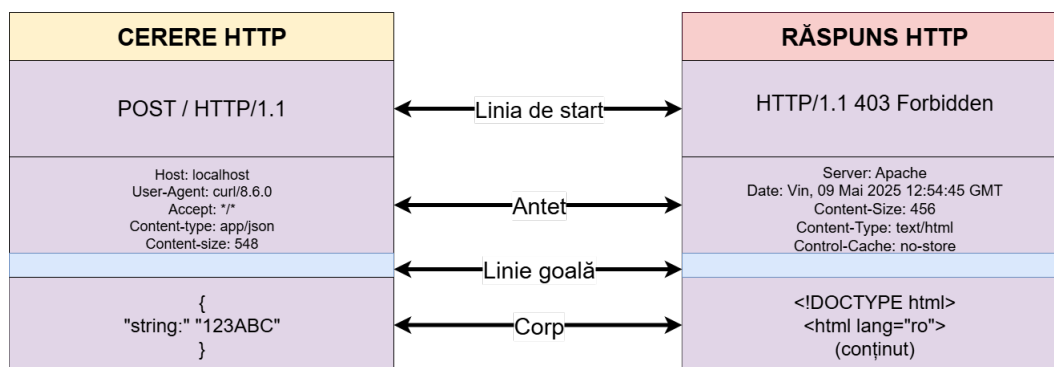


Figura 2.10: Structura mesajelor HTTP

Cereri HTTP

```

HTTP

POST /user HTTP/1.1
Host: exemplu.ro
Content-Type: application/x-www-form-urlencoded
Content-Length: 78

nume=nume%20&prenume&email=licenta%40exemplu.ro
    
```

Figura 2.11: Exemplu de cerere HTTP

O cerere HTTP este un mesaj trimis de client către server cu scop de a solicita o resursă. Structura acesteia este alcătuită dintr-o parte superioară ce cuprinde linia cererii (request line), antet (header), urmată de o linie goală și opțional, un corp [10].

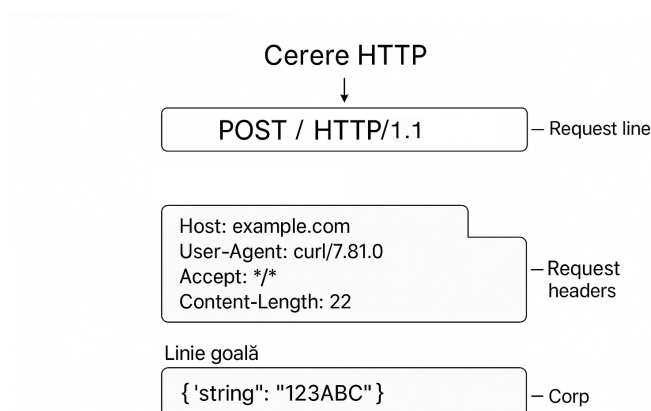


Figura 2.12: Structura cererii HTTP

Partea superioară a unei cereri HTTP începe cu linia de cerere, care conține trei elemente principale: metoda utilizată, resursa cerută și versiunea protocolului.

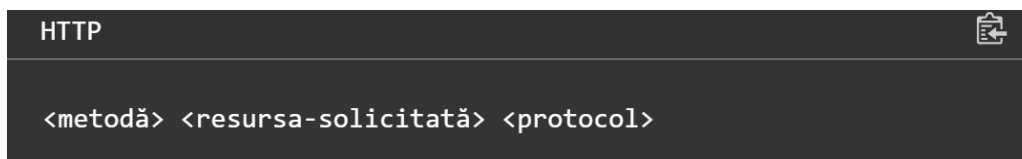


Figura 2.13: Structura unei linii de cerere HTTP

Metoda HTTP

O metodă HTTP indică scopul unei cererii trimise către un server și rezultatul așteptat în cazul în care cererea a fost realizată cu succes. De obicei fiecare metodă are propriul său rol, dar pot împărtăși anumite caracteristici comune. Există mai multe tipuri de metode precum ar fi:

- **GET:** Este folosită pentru doar a prelua date de pe server, fără ca acestea să sufere noi modificări.
- **HEAD:** Se comportă în același mod ca și metoda **GET**, dar fără a include conținut în răspuns.
- **POST:** Are rol în a trimite date către server, de exemplu încărcarea unui fișier.
- **PUT** înlocuiește toate instanțele curente cu noi instanțe specificate de către client.
- **DELETE:** Este utilizată cu scopul de a solicita ștergerea unei resurse specificate.

Resursa solicitată

Resursa solicitată este un URI care identifică resursa căreia i se trimite cererea. Există mai multe tipuri de formatare utilizate pentru a descrie o resursă solicitată:

- Forma de origine, utilizată cu metodele **GET**, **POST**, **HEAD** și **OPTIONS**, este cea mai folosită formatare pentru a identifica originea resursei dintr-un server sau gateway.
- Forma absolută este o sintaxă de tip URL complet ce include informațiile necesare pentru identificarea serverului.
- Forma de autoritate, o structură folosită în cererile HTTP formată dintr-un nume de gazdă și opțional un port delimitat de simbolul **:** împreună cu metoda **CONNECTION** pentru a seta un tunel către resursa țintă.
- Forma asterisk (*) aplicată alături de metoda **OPTIONS** atunci când cererea HTTP nu se aplică unei resurse particulare, ci serverul-ui ca un întreg.

Protocolul

Protocolul este un indicator care reprezintă versiunea protocolului HTTP utilizată în timpul procesului de trimitere a cererii folosit având rol în interpretarea corectă a mesajelor transmise. Singura versiune care trebuie declarată în acest câmp este HTTP/1.1, deoarece versiunile HTTP/0.9, HTTP/1.0 sunt învechite și nu se mai folosesc, iar versiunile mai noi de HTTP/2.0 sunt configurate în timpul conexiunii.

Răspunsuri HTTP

Răspunsurile HTTP sunt mesaje trimise de către server către clienți, în care răspunsul anunță clientul rezultatul determinat în urma cererii. Partea superioară (head) a mesajului HTTP este alcătuită din linia de status și anteturi, iar corpul, deși conține date transmise între client și server, este o componentă opțională [11].

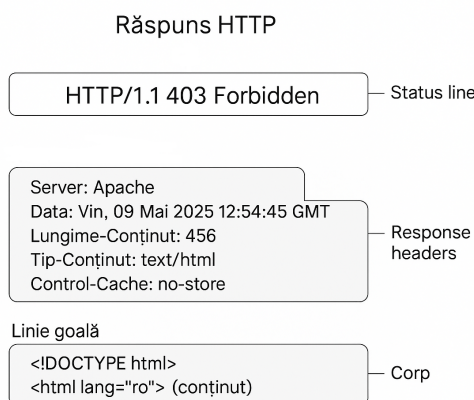


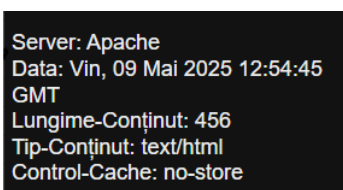
Figura 2.14: Structura răspunsului HTTP

Partea superioară a unui răspuns HTTP este structurată în trei componente principale, începând cu protocolul, care indică versiunea HTTP utilizată de server pentru a transmite mesajul către client. Urmează codul de stare, care reflectă rezultatul acestei transmisii și este organizat în cinci categorii: răspunsuri informaționale (100–199), răspunsuri de succes (200–299), mesaje de redirectionare (300–399), erori ale clientului (400–499) și erori ale serverului (500–599). Printre cele mai frecvente coduri întâlnite se numără 200 OK, ce indică un mesaj transmis cu succes; 404 Not Found, care semnalează că resursa solicitată nu a fost găsită și 302 Found, ce indică schimbarea temporară a URL-ului asociat resursei. În final, textul de stare oferă o explicație succintă a codului, facilitând astfel înțelegerea rezultatului răspunsului.



Figura 2.15: Componentele liniei de stare

Antetul unui răspuns conține metadate despre mesajul transmis clientului și se poate împărți în două categorii principale. Response header-ul reprezintă un șir de caractere care furnizează informații suplimentare despre mesaj și explică modul în care clientul ar trebui să gestioneze cererile ulterioare. Fiecare antet include un nume și o valoare, separate prin simbolul `:` și nu face diferență între majuscule și minuscule. Representation header-ul, pe de altă parte, descrie forma datelor din corpul mesajului și orice codare aplicată acestora. Acest tip de antet oferă destinatarului instrucțiuni despre modul în care resursa trebuie reconstruită înainte de a fi utilizată.



```
Server: Apache
Data: Vin, 09 Mai 2025 12:54:45
GMT
Lungime-Conținut: 456
Tip-Conținut: text/html
Control-Cache: no-store
```

Figura 2.16: Exemplu de response și representation header

Ultima componentă a unui răspuns HTTP este corpul mesajului, prezent în majoritatea cazurilor când serverul trimite un răspuns către client. În situația răspunsurilor reușite, corpul conține datele solicitate prin metoda GET. În caz de erori, corpul include detalii despre problemele care au împiedicat transmiterea corespunzătoare a mesajului.

2.7 Framework-ul Flask Python

Odată ce am înțeles modul în care cererile și răspunsurile sunt transmise prin HTTP, este esențial să analizăm componenta care le interpretează și implementează logica de business. În acest scop, pentru dezvoltarea serverului de aplicații, am optat pentru Flask, un micro-framework Python ale cărui caracteristici și rol în cadrul proiectului vor fi detaliate în cele ce urmează.

Prezentare generală

Flask este un framework web minimalist flexibil creat în limbajul Python, deseori numit „micro-framework” datorită accesibilității de a dezvolta aplicații web folosind doar componente esențiale, fără necesitatea folosirii unor biblioteci suplimentare sau structuri complicate și posibilitatea implementării rapide a unor noi funcționalități în aplicațiile web [12].

Flask a fost creat în anul 2004 de către Armin Ronacher, un membru al grupului de pasionați de Python, Pocoo. Inițial, crearea celebrului framework a fost doar o aluzie la celebra zi a glumelor, 1 aprilie, dar mai târziu a devenit realitate datorită cererii mari a publicului. Numele „flask” (în engleză înseamnă flacon) este o referință la precedentul framework Bottle (în engleză sticlă). În anul 2016, echipa Pocoo și-a încheiat activitatea, iar dezvoltarea Flask-ului și a librăriilor acestuia a fost preluată de către echipa nou fondată numită Pallets [13].

Rolul Flask în aplicația dezvoltată

În contextul acestei lucrări de licență, am ales să implementez serverul aplicației web în Flask datorită funcționalităților sale de construire personalizată a site-urilor, definirii rutelor care facilitează redirecționarea către pagini atunci când un link este accesat, gestionării sesiunilor de autentificare a utilizatorilor și procesării cererilor trimise către aplicația web. Framework-ul Flask prezintă numeroase avantaje precum:

- Minimalism și extensibilitate: Este un framework backend minimalist, care permite extinderea aplicației în funcție de cerințele impuse.
- Ușurință în învățare: API-ul său intuitiv oferă posibilitatea chiar și începătorilor să se familiarizeze rapid cu acesta.
- Flexibilitate: Permite personalizarea și extinderea aplicației conform nevoilor specifice.
- Integrare cu baze de date: Conectarea la baze de date precum MySQL este facilitată prin extensii sau biblioteci externe.
- Suport pentru routing HTTP: Definirea rutelor URL și asocierea acestora cu funcții care gestionează cererile și răspunsurile HTTP.
- Gestionarea securizată a sesiunilor: Stocarea informațiilor pe server și client folosind cookie-uri semnate.
- Integrare cu Jinja2: Oferă unelte pentru generarea paginilor HTML dinamice, permițând o dezvoltare rapidă și eficientă a interfeței web.

O primă funcționalitate pe care o are Flask este gestionarea logicii a aplicației web și bazei de date asociată. Când utilizatorul inițiază o acțiune de tipul **SELECT**, **INSERT**, **UPDATE** sau **DELETE** din interfața frontend, Flask acționează ca intermediar între acesta și baza de date. Framework-ul transformă cererea într-o instrucțiune SQL executabilă, iar rezultatul obținut este apoi returnat și afișat în frontend.

Cea de-a doua funcționalitate a aplicației constă în gestionarea autentificării și a sesiunilor de utilizator. Flask permite crearea conturilor noi, autentificarea utilizatorilor, monitorizarea stării de conectare și definirea permisiunilor pentru diferite acțiuni ale acestora.

O altă funcționalitate importantă este afișarea paginilor dinamice folosind template-uri HTML. Prin integrarea cu motorul de template-uri Jinja2, Flask permite separarea logicii aplicației de partea de prezentare. Astfel, datele preluate din baza de date sau introduse de utilizator pot fi afișate în mod dinamic în pagini web, fără a fi nevoie de modificarea codului sursă HTML. Această abordare contribuie la o dezvoltare mai rapidă și mai flexibilă a aplicației, asigurând totodată o structură clară și ușor de întreținut.

Ultima funcționalitate o reprezintă controlul fluxului aplicației în care utilizatorii după efectuarea anumitor acțiuni sunt redirecționați spre alte pagini. De exemplu, la crearea unui nou cont, utilizatorii sunt redirecționați către pagina de login unde se vor conecta la aplicație.

2.8 Sistemul de baze de date MySQL

O bază de date reprezintă o colecție de informații corelate organizate sub forma unui spațiu de stocare menite pentru a facilita procesele de gestionare, accesare a informațiilor de către utilizatori. Sistemul de gestiune a bazelor de date este un software fundamental ce stă la baza multor aplicații cum ar fi sisteme de rezervări, cumpărături online în care utilizatorii au posibilitatea să definească, să creeze, să întrețină și să controleze accesul la baza de date [14].

Conceptele bazelor de date relaționale și SGBD

O bază de date relațională este o bază de date ce se bazează pe modelul relațional al datelor organizat sub forma unor tabele. Un tabel (sau relație) este format din rânduri și coloane. Rândurile, denumite și tupluri, reprezintă o entitate sau o înregistrare individuală în baza de date. Coloanele sau atributele, constituie caracteristicile sau proprietățile entităților. Fiecare coloană are un identificator unic și un tip de date specific.

Baze de date distribuite și replicarea Master-Slave

O bază de date distribuită este o bază de date logică unică divizată în mai multe părți stocate fizic pe mai multe computere sau noduri interconectate în rețea. La nivel de frontend, o bază de date distribuită funcționează ca o singură bază de date centralizată, nu este nevoie ca utilizatorii să folosească mai multe interfețe, dar la nivel de backend datele sunt distribuite și replicate în diverse locații. Principalele obiective ale unei baze de date distribuite includ scalabilitatea, disponibilitatea, fiabilitatea și îmbunătățirea performanței, obținute prin distribuirea componentelor pe locații cu roluri specifice.

Configurația Master-Slave reprezintă o structură de replicare a bazei de date, în care datele de pe Master sunt copiate automat pe Slave. Master-ul este nodul principal, în care au loc toate operațiile de scriere, cum ar fi `INSERT`, `UPDATE` sau `DELETE`, iar orice modificare efectuată asupra acestuia este transmisă automat către Slave. Slave-ul, în schimb, este un nod secundar responsabil cu efectuarea operațiilor de citire (`SELECT`) și cu actualizarea datelor primite de la Master, asigurând astfel consistența și disponibilitatea informațiilor.

În contextul prezentei lucrări, s-a implementat o arhitectură de tip Master-Slave datorită următoarelor avantaje:

- Scalabilitatea operațiilor de citire: Într-un sistem de rezervări volumul operațiilor de citire depășește semnificativ volumul operațiilor de scriere. Așadar, o arhitectură de tip Master-Slave devine esențială prin direcționarea cererilor de citire către nodurile Slave, eliberarea resurselor consumate de către nodul Master și eficientizarea operațiilor de scriere importante. De asemenea, flexibilitatea sistemului asigură posibilitatea de extindere orizontală cu noi noduri Slave, distribuind uniform și eficient cererile de citire pe măsură ce volumul de date crește.
- Separarea sarcinilor: Arhitectura Master-Slave stabilește două roluri bine separate între cele două noduri ale bazei de date. Nodul Master se ocupă exclusiv cu procesarea și validarea operațiilor de scriere (`INSERT`, `UPDATE`, `DELETE`) pentru a asigura consistența și

integritatea datelor. Nodul Slave are rol în preluarea și executarea operațiilor de citire (**SELECT**). Această separare a rolurilor optimizează performanța operațiilor de citire și scriere prin minimizarea conflictelor de blocare și a suprapunerii resurselor.

- Îmbunătățirea disponibilității: Configurația Master-Slave este o metodă preventivă în cazul unei defecțiuni sau indisponibilități a unei resurse. În cazul în care Master-ul nu mai funcționează, Slave-ul preia atribuțiile de scriere și este promovat la rolul de Master. Acest mecanism, denumit **failover**, asigură continuitatea operațiilor de citire și scriere prin reducerea la minimum a perioadelor de nefuncționare a aplicației.

Limbaajul SQL

SQL este limbaajul principal utilizat pentru lucrul cu baze de date relaționale, fiind recunoscut ca standard în industrie. Acesta a fost inițial dezvoltat de IBM la începutul anilor '70, în cadrul proiectului System R, sub denumirea SEQUEL (Structured English Query Language). Ulterior, numele a fost abreviat la forma actuală, SQL, odată cu standardizarea sa. Limbaajul a fost creat pentru a permite interogarea și manipularea datelor într-un mod declarativ, punând accent pe rezultatul dorit, mai degrabă decât pe pașii necesari pentru a-l obține [14].

SQL este strâns legat de modelul relațional, servind drept interfață standard pentru interacțiunea cu sistemele de gestiune a bazelor de date relaționale. În acest model, datele sunt organizate sub formă de tabele (relații), fiecare rând corespunzând unui tuplu, iar fiecare coloană unui atribut. Limbaajul permite definirea structurii bazei de date, manipularea conținutului și gestionarea accesului, oferind astfel o abstracție eficientă asupra modului în care datele sunt stocate și accesate. Prin natura sa declarativă, SQL permite exprimarea interogărilor într-un mod logic, independent de modul fizic de stocare, ceea ce îl face portabil între diverse implementări de SGBD-uri.

SQL este organizat în mai multe subcomponente funcționale, fiecare având un rol bine definit în interacțiunea cu baza de date. Comenzile de tip DDL permit definirea și modificarea schemelor de date prin instrucțiuni precum **CREATE**, **ALTER** și **DROP**. Comenzile DML includ operații precum **SELECT**, **INSERT**, **UPDATE** și **DELETE**, care sunt utilizate pentru interogarea și manipularea efectivă a datelor. De asemenea, SQL oferă suport pentru controlul accesului prin comenzi de tip DCL, precum **GRANT** și **REVOKE**, și pentru gestionarea tranzacțiilor prin comenzi TCL, cum ar fi **COMMIT** și **ROLLBACK**. Această structurare clară a limbaajului contribuie la separarea responsabilităților în cadrul aplicațiilor care gestionează date relaționale.

În contextul aplicației Flask de rezervări, limbaajul SQL este utilizat ca principalul instrument pentru interacțiunea cu baza de date MySQL. Operațiile efectuate în backend, precum înregistrarea unui utilizator, autentificarea, crearea sau anularea unei rezervări sunt transpuse în comenzi SQL care sunt transmise serverului de baze de date pentru execuție. SQL pune la dispoziție o interfață standardizată pentru aceste operații, indiferent de complexitatea logicii aplicației, asigurând o gestionare uniformă și eficientă a datelor salvate. Prin folosirea unui limbaj standardizat, aplicația poate menține portabilitatea și compatibilitatea cu alte sisteme bazate pe același model relațional.

Deși SQL oferă mecanisme integrate pentru definirea permisiunilor de acces asupra

obiectelor din baza de date, responsabilitatea implementării unei securități eficiente nu se oprește la nivelul SGBD-ului. În mod special, aplicațiile web trebuie să adopte practici suplimentare pentru a preveni vulnerabilități precum SQL injection. Astfel, în cadrul aplicației, sunt utilizate interogări parametrizate care înlocuiesc concatenarea directă a datelor în instrucțiunile SQL, limitând posibilitatea introducerii de cod malițios. Acest aspect este esențial pentru protejarea integrității și confidențialității datelor gestionate [15].

2.9 Internet Information Services

Internet Information Services reprezintă serverul web dezvoltat de compania Microsoft, introdus pentru prima dată în anul 1995 ca parte a sistemului de operare Windows NT 3.51. De atunci, IIS a cunoscut numeroase îmbunătățiri, fiind integrat în versiunile ulterioare de Windows Server și devenind o soluție stabilă și sigură pentru găzduirea aplicațiilor și serviciilor web [16].

IIS operează pe principiul arhitecturii client-server. Un utilizator, prin intermediul unui browser sau al unei aplicații, trimite o solicitare de tip HTTP/HTTPS către server. IIS primește această solicitare și o procesează în funcție de tipul ei. În cazul fișierelor statice, precum HTML, CSS, imagini sau fișiere JavaScript, serverul le transmite direct către client. Dacă solicitarea vizează conținut dinamic, cum ar fi aplicații ASP.NET sau servicii API, aceasta este redirectionată către motorul de procesare corespunzător, care generează răspunsul ce va fi returnat clientului.

Integrarea IIS în arhitectura proiectului se justifică printr-o serie de avantaje precum:

- Gestionarea fișierelor statice: IIS permite servirea rapidă a fișierelor statice (HTML, CSS, JS, imagini) fără a încărca aplicațiile backend, contribuind astfel la reducerea timpului de răspuns către client.
- Integrarea cu aplicații dinamice: IIS acționează ca un „frontend” pentru aplicațiile ASP.NET sau alte servicii web, redirectionând cererile către motorul de procesare adecvat.
- Stabilitate și securitate: Fiind dezvoltat de Microsoft și integrat în Windows Server, IIS oferă un mediu securizat, cu mecanisme de autentificare, criptare și protecție împotriva atacurilor web uzuale.
- Flexibilitate în arhitecturi distribuite: IIS poate funcționa împreună cu Nginx, gestionând eficient resursele și cererile, facilitând scalarea aplicației și echilibrarea încărcării între servere multiple.

2.10 Nginx

Nginx (Engine-X) este un server web de înaltă performanță, un reverse proxy și un load balancer. A fost ales pentru a îndeplini rolurile de server web, cât și reverse proxy cu capabilități de load balancing. Această componentă este poziționată strategic în fața serverelor de aplicații Flask, acționând ca un „distribuitor de trafic” și un punct de intrare unic pentru toate cererile utilizatorilor [17].

Principiul de funcționare al load balancing-ului

Load balancing-ul reprezintă procesul de distribuire a traficului de rețea între mai multe servere, având ca obiective utilizarea eficientă a resurselor, creșterea debitului și reducerea timpului de răspuns. În arhitectura implementată în această lucrare, rolul de load balancer este îndeplinit de Nginx, care preia cererile HTTP de la clienți și le repartizează între instanțele aplicației Flask ce rulează pe stația de lucru principală și VM 2.

Pentru această distribuire a traficului s-a utilizat algoritmul Round Robin. Este cea mai simplă și comună strategie, prin care cererile sunt alocate secvențial fiecărui server disponibil. Concret, Nginx direcționează prima cerere către un server (de exemplu, stația principală), a doua cerere către următorul server din listă (VM 2), continuând acest ciclu pentru a asigura o repartizare uniformă și predictibilă a încărcării între cele două noduri ale sistemului.

Integrarea Nginx cu Flask ca reverse proxy

Pe lângă rolul său de load balancer, Nginx acționează și ca un reverse proxy pentru aplicația Flask. Un reverse proxy este un server care primește cereri de la clienți și le retransmite către serverele interne, gestionând traficul între clienți și infrastructura internă. Configurația Nginx implică definirea unui bloc upstream care listează adresele (IP și port) ale tuturor instanțelor Flask disponibile. Apoi, într-un bloc server care ascultă pe porturile HTTP/HTTPS standard (80/443), cererile sunt redirectionate către grupul upstream folosind directive precum `proxy_pass`. Un alt aspect esențial al integrării Nginx constă în capacitatea sa de a furniza fișiere statice, precum CSS, JavaScript sau imagini încărcate de utilizatori. Deși aplicațiile Flask pot livra fișiere statice, acestea nu sunt optimizate pentru această sarcină, iar gestionarea directă a fișierelor statice poate consuma resurse semnificative ale serverului de aplicații. Nginx este extrem de eficient în gestionarea și servirea fișierelor statice direct din sistemul de fișiere, reducând semnificativ sarcina serverelor de aplicație.

În arhitectura proiectului, cererile pentru resurse statice sunt redirectionate de Nginx către IIS, care se ocupă de livrarea rapidă a acestora către client. În schimb, toate celelalte solicitări dinamice sunt redirectionate către serverele Flask, care le procesează corespunzător. Această separare optimizează performanța generală a sistemului și permite scalarea eficientă a aplicației.

Impactul asupra fluxurilor de aplicație

Rolul Nginx ca load balancer și reverse proxy îmbunătățește performanța, scalabilitatea și reziliența tuturor fluxurilor de aplicație prezentate anterior

Chiar și în condiții de trafic intens, cererile de înregistrare și autentificare sunt distribuite eficient, astfel încât procesul de creare a conturilor noi și accesarea celor existente rămâne rapid și responsiv. În cazul în care un server Flask devine indisponibil, Nginx direcționează automat cererile către serverele active, menținând funcționarea continuă a aplicației.

Fluxul de căutare, care poate genera un număr semnificativ de interogări `SELECT` către

backend, este optimizat prin distribuirea încărcării de către Nginx. Astfel, timpii de răspuns pentru utilizatori se mențin constanți, indiferent de volumul cererilor.

Deși procesul de rezervare este o tranzacție complexă, compusă din mai mulți pași, distribuirea cererilor inițiale (provenite din formularele de căutare sau vizualizare a proprietății) către servere multiple contribuie la păstrarea receptivității sistemului. În acest context, Nginx asigură stabilitatea conexiunii cu instanța de server Flask care procesează efectiv rezervarea.

Chiar dacă modulele dedicate administratorilor și proprietarilor generează un trafic mai redus, ele beneficiază de aceeași reziliență. Operațiunile asupra proprietăților și rezervărilor pot fi realizate fără întârzieri, chiar și în condiții de încărcare ridicată a sistemului sau în cazul în care un server devine indisponibil. Nginx garantează astfel disponibilitatea, performanța și capacitatea aplicației de a scala în funcție de cerințele utilizatorilor.

3 Arhitectura si structura aplicatiei

3.1 Introducere

Creșterea exponențială a numărului de utilizatori și a volumului de date în mediul online a generat provocări tot mai complexe în dezvoltarea aplicațiilor web. Platformele moderne, în special cele care oferă servicii prin internet, trebuie să susțină un număr ridicat de interacțiuni simultane, fără a compromite performanța, securitatea sau fiabilitatea. Trecerea procesului de rezervare din mediul tradițional în cel digital, accelerată de extinderea accesului la internet și utilizarea pe scară largă a dispozitivelor conectate, impune o serie de constrângeri tehnice în special în ceea ce privește gestionarea eficientă a cererilor multiple și a volumelor mari de date.

Pentru a răspunde acestor cerințe aplicațiile web trebuie proiectate ținând cont de mai multe principii fundamentale:

1. Performanță: Optimizarea timpilor de răspuns pentru interacțiunile utilizatorului, atât în cazul afișării rapide a informațiilor, cât și în contextul proceselor mai complexe, precum căutarea proprietăților și efectuarea rezervărilor.
2. Modularitate: Organizarea logicii aplicației în componente distincte, fiecare cu un rol clar definit, pentru a facilita testarea independentă, întreținerea eficientă și actualizările rapide.
3. Scalabilitate: Proiectarea unei arhitecturi capabile să susțină creșteri semnificative ale numărului de utilizatori, fără degradarea performanței, prin implementarea unor soluții flexibile de extindere pe orizontală.
4. Capacitatea de testare: Dezvoltarea componentelor software astfel încât acestea să permită aplicarea eficientă a testelor unitare și de integrare, contribuind la asigurarea unui comportament robust și previzibil al aplicației.

Deși aplicația a fost implementată ca un sistem **multi-utilizator**, unde fiecare utilizator își gestionează propriile date, arhitectura a fost proiectată având în vedere principiile **multi-tenant**. Separarea clară a datelor la nivelul bazei de date, unde fiecare resursă (proprietate, cameră, rezervare) este asociată direct cu un `owner_id` sau `user_id`, oferă o structură solidă pentru o eventuală extindere către un model complet multi-tenant. Într-un astfel de model, datele aparținând unor entități organizaționale distincte ar putea fi izolate complet, asigurând un grad ridicat de securitate și control.

3.2 Arhitectura generală a sistemului distribuit

Soluția arhitecturală propusă pentru îndeplinirea acestor obiective este una distribuită multi-nod. Sistemul nu se bazează pe o singură mașină, ci pe un ansamblu de componente interconectate, fiecare cu un rol specializat. Această structură, prezentată în figura 3.1, stă la baza strategiei de disponibilitate și echilibrare a sarcinilor.

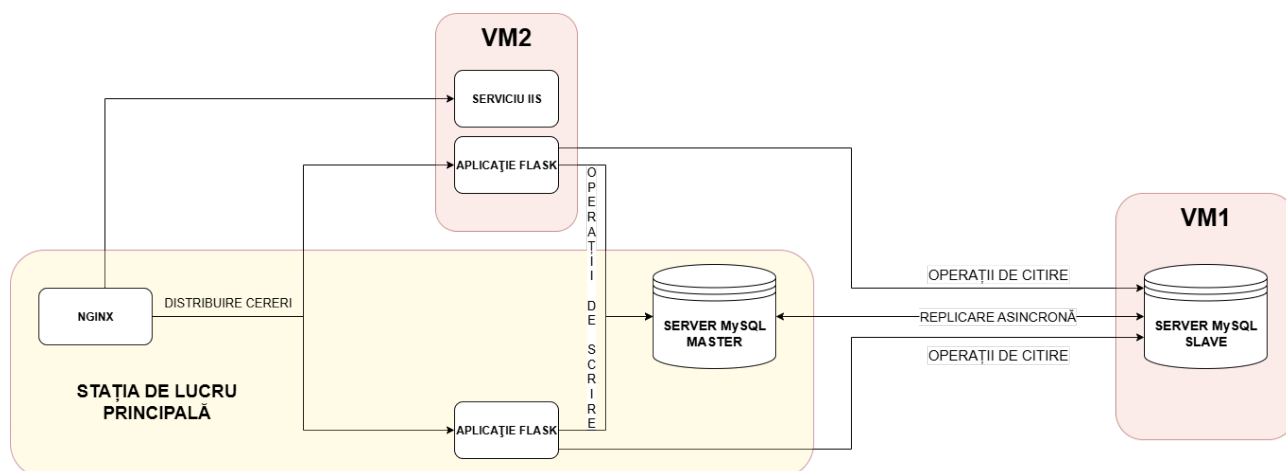


Figura 3.1: Diagrama arhitecturală a componentelor sistemului distribuit

1. **Stația de lucru principală – Host:** Reprezintă mediul principal de dezvoltare și rulare a aplicației. Aici funcționează o instanță a aplicației Flask (care implementează logica de business), serverul de baze de date MySQL Master, responsabil pentru operațiile de scriere, și componenta Nginx, configurată ca load balancer. Aceasta distribuie traficul HTTP către instanțele aplicației aflate pe Host și VM 2.
2. **VM 1:** Această mașină virtuală este dedicată replicii bazei de date, găzduind serverul MySQL Slave. Ea preia exclusiv operațiile de citire, replicând în timp real modificările efectuate pe serverul Master. Astfel, se îmbunătățește performanța accesului la date și se asigură redundanța acestora în caz de eroare.
3. **VM 2:** Găzduiește o instanță secundară a aplicației Flask, care procesează cererile direcționate de Nginx prin mecanismul de load balancing. Astfel, aplicația beneficiază de o capacitate crescută de procesare și de scalabilitate orizontală. Tot pe această mașină rulează și un server web IIS, responsabil cu servirea fișierelor statice către care Nginx redirectionează automat cererile corespunzătoare.
4. **Nginx - load balancer:** Rulat pe Host Nginx acționează ca un reverse proxy ce preia cererile utilizatorilor și le redistribuie către instanțele Flask de pe Host și VM 2 conform algoritmului Round Robin. Această distribuție eficientă permite echilibrarea încărcării și asigură disponibilitatea serviciului.
5. **Aplicația Flask Python:** Reprezintă stratul de logică de business al platformei. Rulând pe Host și VM 2, aceasta gestionează cererile utilizatorilor, interacționează cu baza de date (Master și Slave) și se ocupă de procesele de autentificare, sesiuni și logica funcțională specifică aplicației.
6. **Baza de date MySQL Master/Slave:** Configurația de tip Master-Slave permite împărțirea sarcinilor între scriere (pe Host) și citire (pe VM 1). Această arhitectură optimizează performanța, permite scalarea accesului la date și oferă redundanță prin replicare, reducând riscul de pierdere a datelor.

Împreună cele trei mașini constituie o rețea IP/LAN privată formând un mediu izolat și consistent, esențial pentru testarea scenariilor de eșec și a rezilienței arhitecturii.

3.3 Arhitectura bazei de date

Introducere

Baza de date a aplicației „Platforma Rezervări” este concepută pentru a stoca și gestiona eficient toate informațiile legate de utilizatori, proprietăți, camere, rezervări și plățile aferente. Structura sa relațională asigură integritatea datelor și permite interogări complexe necesare funcționalităților aplicației. Baza de date este compusă din cinci tabele principale interconectate prin chei primare și străine asigurând integritatea referențială și organizarea logică a datelor.



Figura 3.2: Structura bazei de date „rezervări”

Tabela users

Tabela **users** reprezintă componenta centrală a aplicației, stocând informațiile esențiale despre utilizatorii platformei. Aceasta conține date necesare pentru autentificare, autorizare și personalizarea experienței utilizatorilor.

Structura principală include următoarele câmpuri:

- **user_id**: cheie primară auto-incrementată, identificatorul unic pentru fiecare utilizator.
- **username**: numele de utilizator ales la înregistrare, unic pe platformă și utilizat pentru login.
- **email**: adresa de email a utilizatorului folosită pentru înregistrarea și recuperarea contului.
- **password**: parola utilizatorului stocată sub formă de hash securizat, protejând credențialele chiar și în cazul unei breșe de securitate.
- **is_admin**: flag boolean (0 pentru utilizator normal, 1 pentru administrator) care definește rolul și nivelul de acces.

- **created_at**: timestamp-ul la care a fost creat contul utilizatorului.

Relațiile dintre entitățile din baza de date sunt fundamentale pentru organizarea și funcționarea aplicației. Un utilizator poate deține zero sau mai multe proprietăți, relație realizată prin cheia străină **owner_id** din tabela **properties**, care face referire la **user_id**. De asemenea, un utilizator poate efectua zero sau mai multe rezervări, relație realizată prin cheia străină **user_id** din tabela **reservations**, fiind astfel posibilă urmărirea istoricului rezervărilor fiecărui client.

Tabela **properties**

Tabela **properties** stochează detalii complete despre fiecare unitate de cazare (hotel, pensiune, apartament) înregistrată pe platformă. Coloanele principale sunt:

- **property_id**: cheia primară auto-incrementată, identificatorul unic al proprietății.
- **owner_id**: cheia străină care face legătura cu tabela **users**, specificând proprietarul.
- **name**: numele proprietății, de exemplu „Hotel Continental” sau „Pensiunea La Cetate”.
- **description**: descrierea detaliată a proprietății utilă pentru a oferi informații complete utilizatorilor.
- **address, city, country**: coloane pentru detalii de localizare, esențiale pentru căutări și orientare.
- **created_at**: timestamp-ul adăugării proprietății în sistem.

O proprietate poate conține una sau mai multe camere, relație realizată prin cheia străină **property_id** din tabela **rooms**. De asemenea, relația cu tabela **users** este bidirecțională: proprietățile sunt listate în funcție de proprietarul lor, iar ștergerea unui utilizator șterge în cascadă și proprietățile deținute.

Tabela **rooms**

Tabela **rooms** stochează informații detaliate pentru fiecare cameră sau tip de cameră disponibilă pentru rezervare, în funcție de coloana **number_of**. Aceasta permite platformei să gestioneze corect disponibilitatea și caracteristicile camerelor.

Coloanele principale sunt:

- **idrooms**: cheia primară auto-incrementată pentru fiecare cameră înregistrată.
- **property_id**: cheia străină către proprietatea căreia îi aparține camera.
- **name**: numele tipului de cameră (de exemplu, „Cameră single” sau „Apartament cu 2 dormitoare”).
- **description**: scurtă descriere a facilităților camerei.
- **capacity**: numărul maxim de persoane pe care camera le poate găzdui, esențial pentru validarea rezervărilor.

- **price**: prețul standard pe noapte.
- **available**: flag boolean care indică disponibilitatea generală a camerei.

O cameră poate fi inclusă în zero sau mai multe rezervări, relație realizată prin cheia străină `room_id` din tabela `reservations`. De asemenea, ștergerea unei proprietăți duce la ștergerea în cascadă a camerelor asociate.

Tabela `reservations`

Tabela `reservations` reprezintă componenta centrală a sistemului de rezervări, înregistrând fiecare tranzacție efectuată în cadrul platformei. Aceasta permite monitorizarea istoricului rezervărilor și gestionarea procesului complet de rezervare.

Coloanele principale sunt:

- **reservation_id**: cheia primară auto-incrementată pentru identificarea unică a fiecărei rezervări.
- **user_id**: cheia străină care indică utilizatorul care a inițiat rezervarea.
- **room_id**: cheia străină către camera specifică rezervată.
- **start_date**, **end_date**: intervalul de timp al rezervării, esențial pentru verificarea disponibilității și prevenirea rezervărilor duble.
- **status**: starea curentă a rezervării (de exemplu: „pending”, „confirmed”, „cancelled”).
- **created_at**: timestamp-ul creării rezervării.

Fiecare rezervare este asociată cu un singur set de detalii suplimentare de contact stocate în tabela `reservation_details`. Relațiile cu tabelele `users` și `rooms` sunt definite prin cheile străine `user_id` și `room_id`, iar ștergerea unui utilizator sau a unei camere determină, prin ștergere în cascadă, eliminarea automată a rezervărilor aferente.

Tabela `reservation_details`

Tabela `reservation_details` stochează detalii de contact ale clientului care a efectuat rezervarea, precum și prețul total al acesteia. Separarea acestor informații respectă principiile de normalizare, reducând redundanța și facilitând gestionarea datelor sensibile.

Coloanele principale includ:

- **detail_id**: cheia primară auto-incrementată pentru fiecare intrare de detalii.
- **reservation_id**: cheia străină care leagă direct aceste detalii la o rezervare specifică din tabela `reservations`.
- **full_name**: numele complet al persoanei care a efectuat rezervarea.
- **phone**: numărul de telefon de contact.
- **email**: adresa de email de contact.

- **total_price**: prețul total final calculat pentru rezervarea respectivă.

Relația dintre tabela `reservation_details` și `reservations` este de tip 1:1 astfel încât fiecare rezervare are un singur set de detalii asociate. Atributul `reservation_id` din `reservation_details` este o cheie străină cu regula ON DELETE CASCADE, ceea ce înseamnă că, atunci când o rezervare este ștearsă, detaliile sale sunt eliminate automat asigurând integritatea datelor în baza de date.

3.4 Fluxul datelor și funcționarea bazei de date

Funcționalitatea aplicației se bazează pe interacțiuni cu structura bazei de date, gestionând toate etapele procesului de rezervare.

Procesul începe cu înregistrarea unui utilizator. Când un client nou se înscrie, o înregistrare este creată în tabela `users` atribuindu-i un `user_id` unic, iar parola este stocată sub formă hash pentru securitate.

Următorul pas este adăugarea proprietăților. Un utilizator cu rol de proprietar poate introduce o proprietate în sistem, iar o înregistrare corespunzătoare este inserată în tabela `properties` cu `owner_id` referind la `user_id`-ul proprietarului. Camerele disponibile sunt apoi adăugate în tabela `rooms` fiecare fiind legată de `property_id`-ul proprietății.

Căutarea proprietăților implică interogări către tabelele `properties` și `rooms` pentru identificarea unităților care corespund criteriilor specificate și pentru verificarea disponibilității camerelor în perioada selectată.

Crearea unei rezervări este un proces complex și tranzacțional. Etapele implicate sunt:

- Inserarea unei noi înregistrări în tabela `reservations` legată de `user_id`-ul clientului și de `room_id`-ul camerei selectate, cu datele de start și end.
- Inserarea detaliilor de contact și a prețului total în tabela `reservation_details` legate de `reservation_id`-ul nou creat.
- Executarea tuturor acestor instrucțiuni în cadrul unei tranzacții ACID pentru a asigura consistența datelor și a preveni situații precum rezervări fără detalii de contact.

Gestionarea rezervărilor permite utilizatorilor și proprietarilor să vizualizeze rezervările interogând tabela `reservations` și folosind JOIN-uri către `rooms`, `properties` și `reservation_details`.

În cazul anulării, statusul rezervării este actualizat în tabela `reservations` la „cancelled”, păstrând astfel istoricul complet al tranzacțiilor.

3.5 Asigurarea consistenței datelor prin integritate referențială

Utilizarea cheilor străine (**FOREIGN KEY**) cu reguli precum **ON DELETE CASCADE** și **ON UPDATE CASCADE** este esențială pentru menținerea integrității referențiale a bazei de date. De exemplu, ștergerea unui utilizator din tabela **users** determină eliminarea automată a tuturor proprietăților și rezervărilor asociate din tabelele **properties** și **reservations**. Acest mecanism previne apariția înregistrărilor neasociate și simplifică logica aplicației, deoarece consistența datelor este gestionată direct de către sistemul de baze de date. În situațiile în care **FOREIGN KEY** nu este definită la nivel de schemă, aplicația trebuie să asigure manual integritatea datelor.

3.6 Structura internă a aplicației Flask

Dacă arhitectura generală descrie interacțiunile la nivel de sistem, arhitectura internă detaliază modul de construcție al componentei centrale, aplicația web. Pentru implementarea acestui nucleu s-a utilizat framework-ul Flask, o alegere justificată de caracterul său minimalist și flexibil.

Organizarea codului sursă

Aplicația Flask realizată este bine structurată și se bazează pe o separare clară a responsabilităților, ceea ce o face scalabilă și ușor de întreținut. Fiecare componentă a aplicației este concepută să gestioneze sarcini distincte, având posibilitatea de a dezvolta, testa și implementa scalarea independentă fără a afecta funcționarea celorlalte părți ale sistemului.

Această organizare modulară este reflectată direct în structura de directoare a proiectului, care separă logica aplicației de interfața cu utilizatorul și de resursele statice, așa cum reiese din Anexa 1.

Structura aplicației este următoarea:

- **app.py**: Conține logica centrală a aplicației (backend), inclusiv definirea rutelor, gestionarea cererilor și interacțiunea cu baza de date.
- **templates/**: Găzduiește toate fișierele HTML reprezentând stratul de prezentare (frontend) al aplicației.
- **static/**: Conține resursele care nu necesită procesare, precum fișierele CSS, fiind servite direct utilizatorului.

Inițializarea aplicației începe prin crearea unei instanțe standard Flask, unde este setată o cheie secretă, esențială pentru gestionarea securizată a sesiunilor și protejarea operațiunilor sensibile. Baza de date este configurată într-o arhitectură Master-Slave, utilizând două motoare SQLAlchemy distincte (**master_engine** și **slave_engine**) pentru a implementa o strategie de separare a sarcinilor (read/write splitting). Instrucțiunile de scriere (precum înregistrarea unui cont sau adăugarea unei proprietăți) sunt direcționate către baza de date principală (Master), în timp ce majoritatea instrucțiunilor de citire (afișarea proprietăților, căutările) sunt preluate de

baza de date secundară (Slave). Această separare eliberează serverul Master să se concentreze pe operațiuni critice, optimizând performanța generală.

Pentru a ilustra concret această separare, au fost create funcții *helper* care direcționează interogările către motorul corespunzător, așa cum se observă în fragmentul de cod de mai jos:

```
# Definirea motoarelor pentru arhitectura Master-Slave
master_engine = create_engine(db_url_master)
slave_engine = create_engine(db_url_slave)

--- Funcții Helper pentru baza de date ---
# Funcție care execută instrucțiuni de citire de pe serverul SLAVE
def fetch_all(engine, query, params=None):
    with engine.connect() as cnx:
        result = cnx.execute(text(query), params or {})
    return result.mappings().all()

# Funcție care execută instrucțiuni de scriere pe serverul MASTER
def execute_commit(engine, query, params=None):
    with engine.connect() as cnx:
        # ... logica de execuție și commit ...

--- Exemplu de utilizare în rute ---
@app.route('/')
def home():
    # Se folosește SLAVE_ENGINE pentru o instrucțiune de citire
    properties = fetch_all(slave_engine, "SELECT ...")
    return render_template("home.html", properties=properties)

@app.route('/register', methods=['POST'])
def register():
    # Se folosește MASTER_ENGINE pentru a scrie datele noului utilizator
    execute_commit(master_engine, "INSERT INTO users ...")
    return redirect(url_for('login'))
```

Fragmentul 1: Implementarea separării citire/scriere (Master-Slave)

Interacțiunea cu baza de date este gestionată sigur și eficient prin utilizarea context manager-ului `with engine.connect() as cnx`: care asigură deschiderea și închiderea automată și sigură a conexiunilor, chiar și în cazul apariției unor erori. Managementul cursorilor este realizat implicit de către SQLAlchemy în cadrul acestui context, simplificând logica de acces la date. Fiecare rută este configurată să folosească deliberat fie conexiunea către Master pentru scriere, fie cea către Slave pentru citire, asigurând integritatea datelor. O excepție de menționat este ruta `/login`, care utilizează conexiunea Master chiar și pentru citire, o practică recomandată pentru a preveni erorile cauzate de întârzierea de replicare (replication lag).

Structura rutelor este organizată logic pe funcționalități, cele pentru autentificare, gestionarea utilizatorilor, administrarea proprietăților și rezervări fiind grupate și ușor de identificat. Fiecare funcție asociată unei rute urmează un model consistent: verifică autentificarea utilizatorului, gestionează cererile GET și POST, interacționează cu baza de date prin funcțiile helper, afișează mesaje flash pentru feedback și, în final, redă template-uri HTML cu datele

necesare.

Pe lângă logica principală aplicația include funcții utilitare care sporesc claritatea și reutilizarea codului. Funcția `is_logged_in()` (Anexa 2) verifică rapid dacă un utilizator este autentificat, iar filtrul Jinja2 `format_ro_date` (Anexa 3) afișează datele într-un format prietenos pentru utilizator. Motorul de recomandări `get_recommendations` (Anexa 4) realizează logica complexă pentru selectarea camerelor potrivite pe baza criteriilor de căutare, iar funcția `check_server_status` (Anexa 5) permite monitorizarea stării serverelor în panoul de administrare.

În ceea ce privește implementarea pentru producție, aplicația rulează pe serverul waitress, un server WSGI robust, capabil să gestioneze multiple cereri concurente. Configurarea acestuia permite accesul aplicației pe rețea și asigură stabilitate și performanță ridicată chiar și în condiții de trafic intens.

4 Funcționalitățile aplicației

4.1 Introducere în mecanismele funcționale ale aplicației

În această secțiune sunt prezentate componentele funcționale fundamentale ale aplicației, cu accent pe modul în care sunt gestionate cererile utilizatorilor – de la interacțiunea cu interfața grafică, la prelucrarea pe server și până la salvarea datelor în baza de date. Analiza este structurată pe etape distincte, pentru a evidenția clar modul în care aceste componente colaborează. Am urmărit să subliniez procesele care contribuie direct la asigurarea unei funcționări stabile, sigure și coerente a aplicației.

4.2 Înregistrarea și autentificarea utilizatorilor

Mecanismele de înregistrare și autentificare reprezintă punctul de acces principal în cadrul sistemului, asigurând atât prima interacțiune a utilizatorilor noi cu aplicația, cât și accesul celor existenți la funcționalitățile dedicate. În realizarea acestui modul, s-a urmărit asigurarea unui echilibru între simplitatea în utilizare și protecția solidă a datelor. Componenta de autentificare este un element critic în arhitectura aplicației întrucât orice vulnerabilitate poate compromite atât confidențialitatea datelor, cât și funcționarea generală a sistemului.

4.2.1 Formularul de înregistrare

Procesul de înregistrare marchează primul punct de contact al utilizatorului cu aplicația și combină elementele de interfață cu mecanismele din backend care se ocupă de validarea datelor și asigurarea securității acestora.

Prezentarea frontend

Interfața de înregistrare este un formular HTML simplu, conceput pentru a colecta informațiile esențiale:

- Nume de utilizator: Un câmp text (`<input type="text">`) pentru identificarea unică în platformă.
- Email: Un câmp specific pentru email (`<input type="email">`) care beneficiază de validarea automată a formatului la nivel de browser.
- Parolă: Un câmp de tip parolă (`<input type="password">`) al cărui conținut este mascat pentru a preveni expunerea vizuală (fenomenul de „shoulder surfing”).
- Autentifică-te: Pentru utilizatorii care deja au cont și nu mai este necesară autentificarea.

Pentru a garanta o experiență de utilizare corectă toate câmpurile sunt marcate ca fiind obligatorii prin atributul `required`, oferind astfel o validare preliminară la nivel de client. Totuși, validarea finală și securizarea datelor se realizează întotdeauna pe server. Această separare între validarea din interfață și cea din backend este esențială pentru prevenirea atacurilor bazate pe manipularea cererilor HTTP și pentru menținerea corectă a funcționării aplicației.

Din punct de vedere vizual, formularul este stilizat folosind clase CSS pentru a crea o interfață modernă, clară și responsivă. O astfel de abordare este importantă pentru menținerea interesului și confortului utilizatorilor pe durata interacțiunii cu aplicația oferind o experiență coerentă și intuitivă. De asemenea, interfața include un link vizibil către pagina de autentificare destinat utilizatorilor care dețin deja un cont.

Figura 4.1: Formularul de înregistrare a utilizatorilor

Logica backend

Procesarea datelor se realizează în funcția asociată rutei `/register` care gestionează cererile `POST`. Fluxul de validare este strict și include verificarea unicității utilizatorului, securizarea parolei prin hashing cu biblioteca `bcrypt` și inserarea noii înregistrări în baza de date. După succes utilizatorul este informat printr-un mesaj `flash` și redirecționat către pagina de login.

Logica de business implementată în codul prezentat mai jos poate fi descrisă ca un flux de instrucțiuni structurat în mai mulți pași.

```
@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        username = request.form['username']
        email = request.form['email']
        password = request.form['password']

        # Verifică dacă utilizatorul există deja (operațiune de citire)
        existing_user = fetch_one(master_engine,
                                   "SELECT user_id FROM users WHERE username = :username OR
                                   email = :email",
                                   {'username': username, 'email': email})
```

```

    if existing_user:
        flash("Numele de utilizator sau emailul există deja.", "
danger")
        return redirect(url_for('register'))

    # Securizarea parolei folosind bcrypt înainte de salvare
    hashed_password = bcrypt.hashpw(password.encode('utf-8'),
bcrypt.gensalt())

    # Salvarea datelor securizate în baza de date pe serverul
MASTER
    execute_commit(master_engine,
        "INSERT INTO users (username, email, password) VALUES (:u,
:e, :p)",
        {'u': username, 'e': email, 'p': hashed_password})

    flash("Contul a fost creat cu succes!", "success")
    return redirect(url_for('login'))
return render_template('register.html')

```

Fragmentul 2: Cod pentru logica de înregistrare (ruta /register)

În primul rând la primirea unei cereri de tip POST informațiile transmise prin formular precum numele de utilizator, adresa de email și parola sunt extrase din obiectul `request.form`. Această etapă asigură preluarea corectă a datelor introduse de utilizator și constituie punctul de plecare al procesului de înregistrare.

Ulterior, se realizează validarea unicității datelor. Se execută o interogare SQL `SELECT` către serverul Master pentru a verifica dacă numele de utilizator sau adresa de email există deja în tabelul `users`. În cazul în care se identifică o înregistrare existentă, procesul de înregistrare este întrerupt, iar utilizatorul este informat printr-un mesaj `flash()` și redirecționat către pagina curentă pentru corectarea datelor.

O etapă critică în proces este securizarea parolei. Dacă datele sunt valide și unice, parola introdusă de utilizator este procesată prin funcția `bcrypt.hashpw()`, care combină parola cu un „salt” generat aleatoriu, obținându-se astfel un hash securizat și non-reversibil. Această metodă garantează că parola nu este stocată niciodată în formă clară și protejează sistemul împotriva accesului neautorizat.

După securizarea parolei datele sunt salvate în baza de date. Numele de utilizator, adresa de email și hash-ul parolei sunt inserate în tabelul `users` printr-o comandă SQL `INSERT` executată pe serverul Master, iar operațiunea este finalizată printr-un `commit`, ceea ce garantează salvarea permanentă a informațiilor în baza de date.

În final procesul se încheie cu etapa de feedback și redirecționare. După inserarea cu succes a datelor, utilizatorului i se afișează un mesaj de confirmare folosind `flash()`, iar acesta este redirecționat către pagina de `/login`, unde se poate autentifica cu contul nou creat.

4.2.2 Formularul de autentificare

Procesul de autentificare este mecanismul prin care un utilizator înregistrat obține acces la funcționalitățile protejate ale platformei. Acesta este implementat prin intermediul rutei `/login` și implică atât o componentă de interfață, cât și o logică de validare robustă în backend.

Prezentarea frontend

Interfața de autentificare este un formular HTML minimalist care solicită utilizatorului introducerea a două date esențiale:

- Nume de utilizator: câmpul de identificare al contului.
- Parolă: câmpul de verificare a identității, de tip parolă.

La trimitere datele sunt transmise serverului printr-o cerere de tip **HTTP POST**. Interfața este, de asemenea, capabilă să afișeze mesaje de eroare (de exemplu: „Date de autentificare incorecte”) prin sistemul `flash` al Flask oferind un feedback clar utilizatorului în cazul unui eșec.

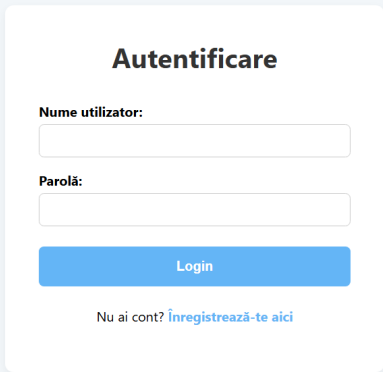
The image shows a login form titled "Autentificare" centered on a light blue background. The form is a white card with a subtle shadow. It contains two input fields: "Nume utilizator:" and "Parolă:". Below the password field is a blue "Login" button. At the bottom of the form, there is a link that says "Nu ai cont? Înregistrează-te aici".

Figura 4.2: Formularul de autentificare a utilizatorilor

Logica backend

Funcția `login()` din backend este responsabilă cu validarea credențialelor. Procesul este securizat și eficient, urmând mai mulți pași logici:

1. Preluarea credențialelor: Datele (nume de utilizator și parolă) sunt extrase din cererea `POST`.
2. Căutarea utilizatorului în baza de date: Se execută o comandă `SQL SELECT` pentru a găsi înregistrarea din tabelul `users` care corespunde numelui de utilizator introdus. Dacă nu se găsește niciun utilizator, autentificarea eșuează.

3. Verificarea criptografică a parolei: Acesta este pasul critic de securitate. Parola în clar, introdusă de utilizator, este comparată cu hash-ul stocat în baza de date folosind funcția `bcrypt.checkpw()`. Această funcție extrage automat „salt”-ul din hash-ul stocat, aplică operația de hashing parolei introduse și compară rezultatele. Procesul este încetinit intenționat pentru a preveni atacurile de tip brute-force.
4. Crearea sesiunii de utilizator: Dacă parola este corectă se inițiază o sesiune pentru utilizator. Informații cheie, precum `user_id`, `username` și rolul `is_admin`, sunt stocate în obiectul `session` al Flask. Aceste date sunt trimise clientului sub forma unui cookie semnat criptografic care va fi folosit pentru a identifica utilizatorul la cererile ulterioare.
5. Redirecționare: După succesul autentificării utilizatorul este redirecționat către pagina principală a aplicației.

4.2.3 Mecanismele de securitate

Securitatea conturilor de utilizator reprezintă o componentă esențială a aplicației asigurată prin metode criptografice bine implementate. Controlul accesului se realizează prin două rute principale definite în fișierul `app.py`: `/register` și `/login`. Aceste rute nu se limitează la preluarea și procesarea datelor trimise din partea frontend-ului, ci includ și mecanisme avansate menite să protejeze informațiile sensibile ale utilizatorilor.

În momentul înregistrării unui cont nou, parola este prelucrată printr-un algoritm de hashing securizat, astfel încât nicio variantă lizibilă a acesteia să nu fie salvată în baza de date. Această operațiune este realizată cu ajutorul bibliotecii **bcrypt**.

Funcția `bcrypt.hashpw()` combină parola (codificată în format binar) cu o valoare „salt” unică, generată de `bcrypt.gensalt()`. La autentificare, parola introdusă este verificată cu hash-ul stocat folosind funcția `bcrypt.checkpw()`. Astfel, protecția credențialelor este menținută la un nivel foarte înalt, conform bunelor practici din securitatea modernă.

Unul dintre cele mai critice aspecte de securitate în orice aplicație web modernă este modul în care sunt stocate parolele utilizatorilor. Salvarea acestora în forma lor originală, lizibilă, este o vulnerabilitate fundamentală care în cazul unei breșe de securitate ar expune imediat toate conturile. Pentru a contracara acest risc aplicația folosește un mecanism robust de hashing bazat pe standarde criptografice recunoscute și verificate.

Funcționalitățile de hashing sunt furnizate de biblioteca **bcrypt**, o soluție modernă și foarte sigură, recunoscută ca standard de facto pentru protecția parolelor. Spre deosebire de funcțiile de hash rapide, precum **SHA-256**, concepute pentru viteză și inadecvate pentru parole, **bcrypt** este un algoritm **adaptiv**, proiectat să fie intenționat lent și costisitor din punct de vedere computațional. Această caracteristică, denumită factor de lucru (*work factor*), poate fi ajustată pentru a contracara creșterea puterii de calcul a hardware-ului modern, menținând astfel relevanța sa pe termen lung. Mecanismul de securitate al **bcrypt** se bazează pe doi factori esențiali: integrarea unei valori „salt” și adoptarea unui „factor de lucru” adaptiv.

- **Sarea (salt-ul)** este un șir de date aleatoriu, generat automat de funcția `bcrypt.gensalt()` pentru fiecare parolă în parte. Acesta este combinat cu parola înainte

de hashing, asigurând că două parole identice vor avea hash-uri finale complet diferite. Această tehnică anulează eficiența atacurilor bazate pe tabele pre-calculate, cum ar fi **rainbow tables**.

- **Factorul de lucru (work factor)** este caracteristica adaptivă a **bcrypt**. Acesta controlează complexitatea computațională, determinând numărul de iterații interne ale algoritmului. Un factor de lucru mai mare înseamnă un timp de calcul mai lung pentru fiecare hash. Acest „cost” computațional ridicat face ca atacurile de tip **brute-force** să fie extrem de lente și, în consecință, nepracticabile din punct de vedere economic și temporal pentru un atacator.

În practică, biblioteca **bcrypt** gestionează eficient această complexitate, oferind o soluție simplificată și sigură. Funcția `bcrypt.hashpw()`, combinată cu `bcrypt.gensalt()`, gestionează automat generarea salt-ului și aplicarea factorului de lucru, producând la final un singur șir de caractere. Ulterior, la autentificare, funcția `bcrypt.checkpw()` utilizează aceste informații. Ea extrage automat salt-ul din hash-ul stocat în baza de date pentru a procesa parola introdusă de utilizator și a valida corectitudinea acesteia. Implementarea acestui mecanism este esențială pentru integritatea sistemului și pentru încrederea utilizatorilor în platformă.

Procesul de înregistrare

Atunci când un utilizator își creează un cont nou, parola introdusă este imediat procesată printr-un algoritm robust de hashing. Procesul utilizează funcția `bcrypt.hashpw()` din biblioteca **bcrypt**.

Inițial, parola introdusă de utilizator este un șir de caractere (**string**). Deoarece **bcrypt** operează pe date binare, aceasta este mai întâi codificată în format **UTF-8** utilizând `password.encode('utf-8')`. Ulterior funcția `bcrypt.gensalt()` este apelată pentru a genera un salt unic și aleatoriu. Acest salt este esențial pentru securitate deoarece face ca două parole identice care aparțin unor utilizatori diferiți generează hash-uri complet diferite, reducând astfel eficiența atacurilor de tip **rainbow table**. Procesul de hashing este realizat de funcția `bcrypt.hashpw()` care combină parola codificată cu salt-ul generat rezultând un hash securizat ce încorporează salt-ul în structura sa și care poate fi stocat în siguranță în baza de date.

Principiul fundamental al acestui mecanism constă în faptul că doar hash-ul calculat este stocat în baza de date niciodată parola în forma sa originală. Aceasta garantează faptul că în cazul unei breșe de securitate care ar compromite baza de date atacatorii ar avea acces doar la șirurile criptografice rezultate. Chiar dacă ar fi obținute aceste hash-uri sunt proiectate pentru a fi practic ireversibile din punct de vedere computațional protejând astfel în mod eficient credențialele reale ale utilizatorilor.

Codul implementează atât mecanisme de securizare a parolei prin **bcrypt**, cât și validări pentru unicitatea numelui de utilizator și a adresei de email, prevenind astfel crearea conturilor duplicate și menținând consistența datelor din sistem.

Procesul de autentificare

La autentificare serverul verifică identitatea utilizatorului printr-un proces securizat desfășurat în mai mulți pași gestionat de ruta `/login`. Logica backend este proiectată să asigure atât corectitudinea datelor, cât și protecția contului.

Procesul începe prin căutarea utilizatorului în baza de date folosind numele de utilizator furnizat în formular. Se execută o interogare SQL `SELECT` pentru a găsi înregistrarea corespunzătoare. Această căutare este trimisă intenționat către serverul Master al bazei de date, o alegere de design care previne problemele cauzate de întârzierea replicării și asigură că un utilizator recent înregistrat se poate autentifica imediat.

Urmează etapa critică de securitate: verificarea criptografică a parolei. Parola în clar introdusă de utilizator este comparată cu hash-ul stocat în baza de date prin apelul funcției `bcrypt.checkpw()`. Metoda extrage automat valoarea unică „salt” încorporată în hash, aplică procesul de hashing asupra parolei introduse și compară rezultatele obținute. Mecanismul este esențial pentru securitatea procesului, garantând că parola în clar nu este niciodată procesată sau comparată într-un format nesecurizat.

Dacă identitatea este confirmată autentificarea reușește și se inițiază o sesiune securizată. Obiectul `session` din Flask este populat cu informații esențiale despre utilizator, precum identificatorul unic (`user_id`), numele de utilizator și rolul acestuia. Aceste date sunt transmise către browser sub forma unui cookie semnat criptografic care validează identitatea utilizatorului la fiecare cerere ulterioară. În paralel, ruta `/logout` are rolul de a încheia sesiunea curentă eliminând toate datele asociate pentru a încheia în mod sigur accesul utilizatorului.

4.3 Mecanismul de gestiune a sesiunii

Odată ce un utilizator se autentifică cu succes, aplicația trebuie să mențină starea de autentificare pe parcursul interacțiunilor ulterioare. Acest mecanism este implementat prin sistemul de sesiuni al framework-ului Flask.

La o autentificare reușită, informații esențiale despre utilizator, precum identificatorul unic (`user_id`), numele de utilizator (`username`) și rolul de acces (`is_admin`) sunt stocate în obiectul `session`. Flask codifică aceste date și le trimite către browser sub forma unui cookie. Acest cookie nu este stocat în clar, ci este **semnat criptografic** de către server folosind o cheie secretă unică, definită în configurația aplicației prin `app.secret_key`. Această semnătură digitală previne orice tentativă de falsificare sau manipulare a datelor din sesiune.

Mecanismul criptografic utilizat asigură două aspecte fundamentale de securitate:

- **Integritatea Datelor:** Previne orice tentativă de modificare a datelor stocate în cookie de către un utilizator. De exemplu, un utilizator nu își poate atribui singur drepturi de administrator prin modificarea valorii `is_admin = True`, deoarece orice alterare a conținutului ar invalida semnătura criptografică, iar serverul ar respinge sesiunea.

- Autenticitatea: Garantează că respectivul cookie provine de la serverul aplicației și nu a fost fabricat de o terță parte. Doar serverul care deține cheia secretă poate genera o semnătură validă.

Confidențialitatea valorii `app.secret_key` este prin urmare critică pentru securitatea întregului sistem. Compromiterea acesteia ar permite unui atacator să genereze sesiuni false și valide pentru orice utilizator ocolind complet mecanismul de autentificare.

Momentul cheie în care sesiunea este creată este imediat după validarea credențialelor în ruta `/login`. Odată ce cheia `'user_id'` este prezentă în dicționarul de sesiune, aplicația consideră utilizatorul ca fiind autentificat. Securizarea rutelor protejate se simplifică astfel la o singură verificare: dacă un utilizator neautentificat (a cărui sesiune nu conține `'user_id'`) încearcă să acceseze o resursă protejată, acesta este redirecționat automat către pagina de autentificare, prevenind astfel accesul neautorizat.

4.4 Interacțiunea cu baza de date

Mecanismele de înregistrare și autentificare utilizează baza de date pentru a garanta stocarea sigură și regăsirea fiabilă a informațiilor utilizatorilor.

La înregistrarea unui utilizator nou informațiile acestuia (nume de utilizator, email și parola deja procesată prin hashing) sunt salvate în tabelul `users` printr-o comandă `INSERT`. Folosirea placeholder-elor (`%s`) în locul concatenării directe a valorilor în interogare este necesară deoarece reprezintă metoda standard prin care driver-ul bazei de date previne atacurile de tip **SQL injection**.

În timpul autentificării, după ce utilizatorul introduce numele de utilizator și parola, se execută o interogare `SELECT` pentru a găsi înregistrarea corespunzătoare din baza de date. Scopul este de a compara hash-ul parolei stocate cu cel generat din parola introdusă. Dacă interogarea returnează un rezultat sunt preluate și alte informații relevante, precum ID-ul și rolul utilizatorului, necesare pentru crearea sesiunii.

După ce procesul de autentificare s-a realizat cu succes și sesiunea utilizatorului este creată pe baza informațiilor validate este la fel de important să asigurăm o metodă sigură și corectă de încheiere a acesteia. Astfel, ruta `/logout` gestionează terminarea sesiunii, garantând că utilizatorul se deconectează într-un mod controlat și fără riscuri de securitate. Deși implementarea acestei funcționalități este simplă ea este esențială pentru menținerea integrității și protecției datelor utilizatorilor. Funcția asociată acestei rute execută următoarele operații:

- Invalidarea completă a sesiunii: Logica fundamentală constă în apelarea metodei `session.clear()`. Această funcție golește complet dicționarul de sesiune, eliminând dintr-o singură operațiune toate cheile stocate la autentificare (precum `user_id`, `username` și `is_admin`). Abordarea este robustă și asigură că nicio dată reziduală de sesiune nu este lăsată accidental în urmă.
- Feedback pentru utilizator: După terminarea sesiunii, se utilizează funcția `flash()` pentru a trimite un mesaj de confirmare („Deconectare reușită”), informând utilizatorul că acțiunea s-a finalizat cu succes.

- Redirecționare: La final, utilizatorul este redirecționat către pagina de login folosind `redirect(url_for('login'))`.

Prin ștergerea completă a datelor din sesiune, ruta `/logout` garantează că nicio acțiune ulterioară nu poate fi realizată în numele utilizatorului deconectat, fiind o componentă esențială a arhitecturii de securitate a aplicației.

4.5 Pagina principală

Pagina principală, servită de calea `/`, reprezintă punctul central de intrare în aplicație. Rolul său este de a oferi o primă impresie vizuală, de a facilita navigarea și de a prezenta o listă relevantă de proprietăți disponibile.

Prezentarea frontend

Interfața paginii principale este structurată în jurul a trei componente cheie care sunt randate dinamic folosind motorul de template-uri Jinja2:

1. Bara de navigație dinamică: Aceasta este o componentă esențială pentru experiența utilizatorului. Conținutul său se adaptează în funcție de starea de autentificare a utilizatorului.
 - Pentru un utilizator neautentificat (figura 4.3), bara afișează link-uri către paginile de `/login` și `/register`.

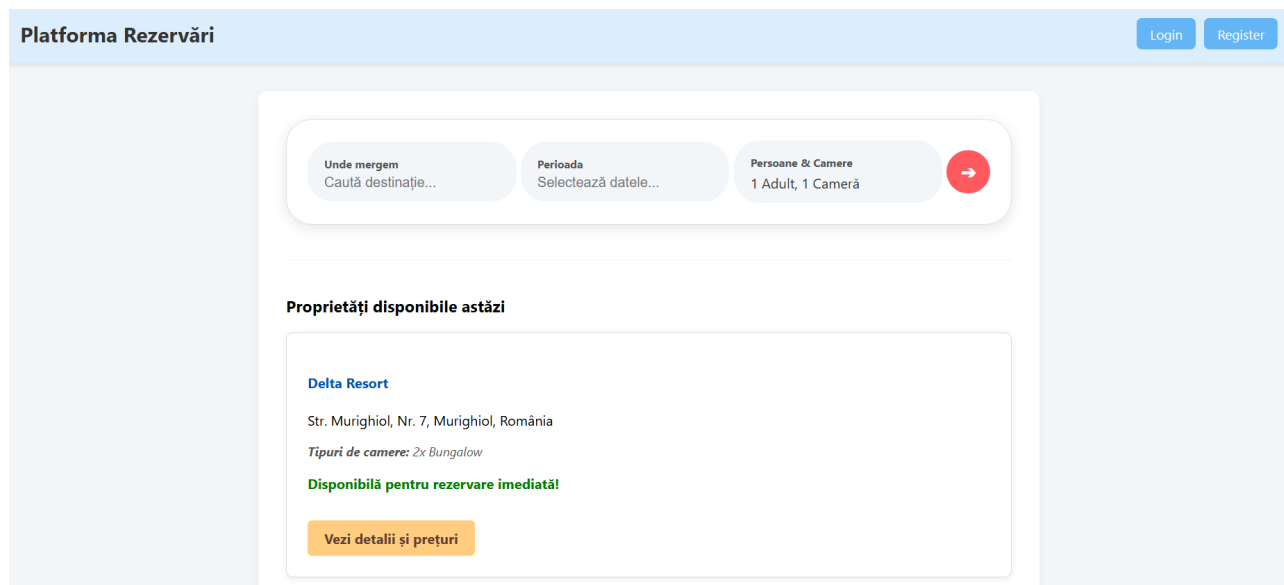


Figura 4.3: Pagina home - utilizator neautentificat

- Pentru un utilizator autentificat (figura 4.4), link-urile de autentificare sunt înlocuite cu opțiuni specifice contului: un buton pentru a adăuga o proprietate nouă, un meniu dropdown cu acces rapid către profil, rezervări și proprietățile personale și un buton de deconectare (`/logout`). Această logică condițională este implementată în template folosind un bloc `{% if logged_in %}`.

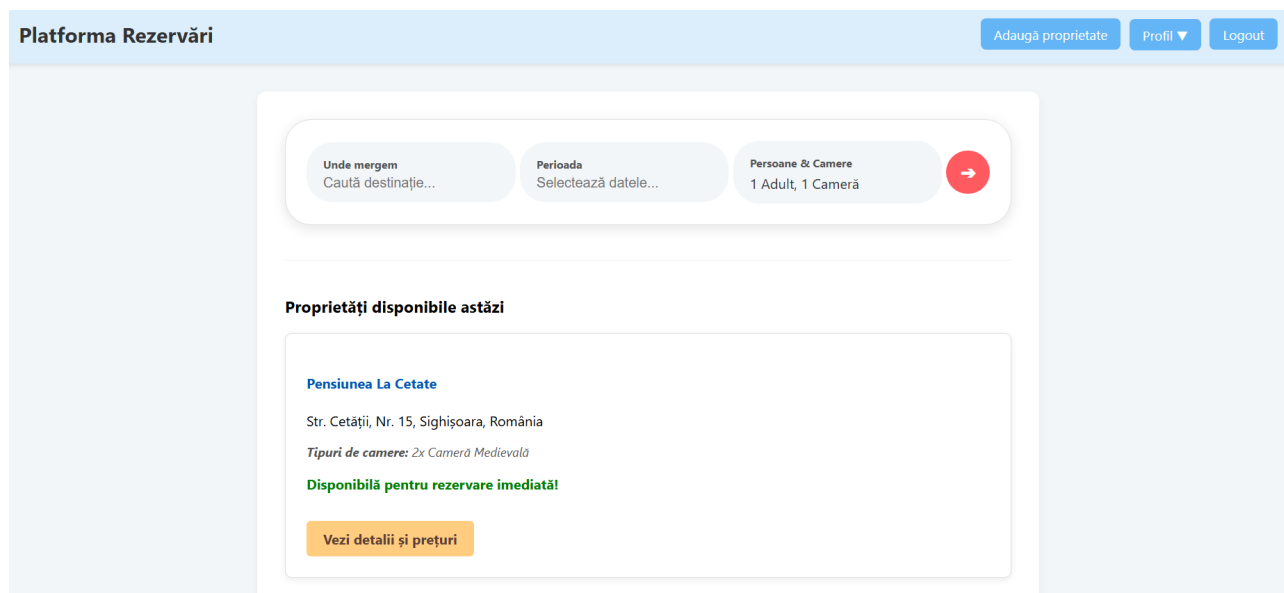


Figura 4.4: Pagina home - utilizator autentificat

2. Componenta de căutare: Formularul de căutare este amplasat în partea superioară a paginii pentru a oferi vizitatorilor un „call-to-action” clar și imediat accesibil.
3. Zona de afișare a proprietăților: Sub bara de căutare, pagina prezintă o listă de proprietăți sub formă de carduri. Fiecare card afișează informații esențiale precum numele proprietății, locația și un sumar al tipurilor de camere disponibile. Aplicația indică în timp real dacă o proprietate este disponibilă pentru rezervare imediată.

Logica backend

Funcția `home()` are rolul de a pregăti datele care urmează să fie afișate pe pagina principală. În primul rând, se realizează preluarea proprietăților printr-o interogare SQL `SELECT`, care utilizează o instrucțiune `JOIN` pentru a uni tabela `properties` cu tabela `users` asigurând afișarea numelui proprietarului alături de fiecare cazare.

În al doilea rând, logica backend include o caracteristică de personalizare a conținutului: dacă utilizatorul este autentificat, interogarea este ajustată dinamic printr-o clauză `WHERE` pentru a exclude proprietățile deținute de acesta, astfel încât un proprietar să nu își vadă propriile oferte în lista principală de sugestii. Aceste date colectate sunt transmise către template-ul `home.html`, unde sunt randate dinamic conform regulilor descrise anterior.

4.6 Pagina de profil

Pagina de profil, accesibilă prin ruta `/profile`, reprezintă un centru de comandă personalizat pentru fiecare utilizator autentificat. Aceasta nu este o pagină statică, ci un „dashboard” interactiv care îi permite utilizatorului să își gestioneze toate activitățile pe platformă.

Prezentarea frontend

Interfața web este proiectată pentru a afișa date agregate într-un mod structurat și intuitiv, folosind motorul de template-uri Jinja2. Pagina întâmpină utilizatorul cu un mesaj de bun venit personalizat și prezintă două carduri principale interactive:

- Cardul „Proprietățile Mele”: Afișează numărul total de proprietăți deținute de utilizator. Cardul conține un buton care direcționează utilizatorul către o pagină dedicată gestionării proprietăților sale.
- Cardul „Rezervările Mele”: În mod similar, afișează un sumar al rezervărilor efectuate și oferă un link către o pagină cu istoricul detaliat al acestora.

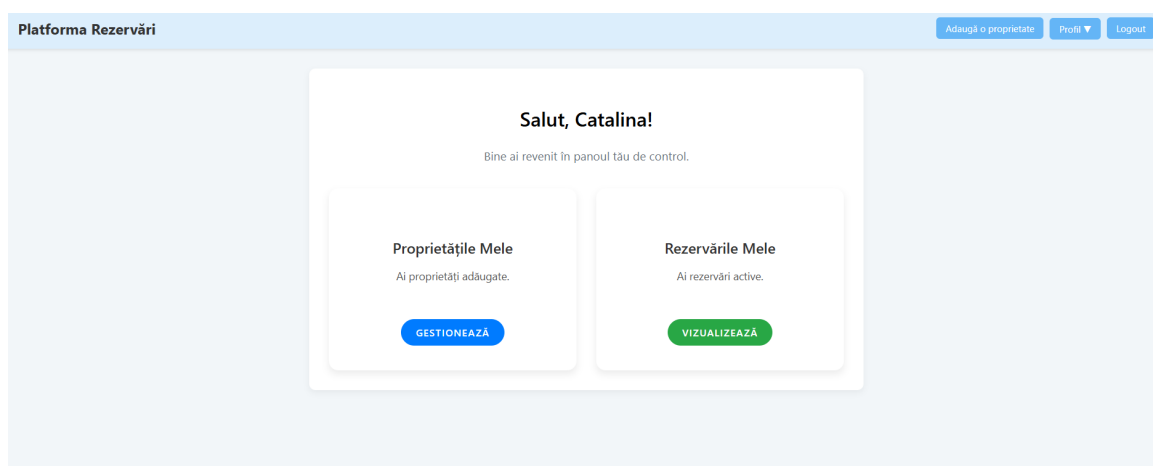


Figura 4.5: Interfața paginii de profil

Logica backend

Pentru a popula dinamic aceste secțiuni ale interfeței, funcția asociată rutei `/profile` are un rol esențial de **agregator de date**. Folosind ID-ul utilizatorului stocat în `session` funcția execută mai multe interogări `SELECT` specializate către baza de date:

1. O interogare pentru a prelua detaliile personale ale utilizatorului din tabelul `users`.
2. O a doua interogare pentru a extrage toate proprietățile pe care utilizatorul le deține (`owner_id`).
3. O a treia interogare, mai complexă, care implică operațiuni de `JOIN` între tabelele `reservations`, `rooms` și `properties`, pentru a lista detaliile complete ale rezervărilor făcute de utilizator.

Toate aceste seturi de date sunt apoi transmise ca și context la randarea template-ului `profile.html`.

4.7 Modificarea datelor de profil

Funcționalitatea de editare a profilului permite utilizatorilor să își actualizeze informațiile personale. Întregul proces este gestionat de ruta `/edit-profile` care se ocupă atât de afișarea formularului, cât și de procesarea sigură a datelor modificate.

Prezentarea frontend

Interfața de editare constă într-un formular HTML care la o cerere de tip `GET` este pre-populat cu datele curente ale utilizatorului (nume de utilizator, email), extrase din obiectul `user` trimis de backend. Formularul este împărțit logic în mai multe secțiuni:

- Date de bază: Câmpuri pentru modificarea numelui de utilizator și a adresei de email.
- Schimbarea parolei (opțional): O secțiune dedicată actualizării parolei, care conține câmpuri pentru noua parolă și confirmarea acesteia. Aceste câmpuri sunt opționale.
- Verificarea identității: Un câmp obligatoriu pentru introducerea parolei curente. Această măsură de securitate asigură că doar proprietarul contului, care cunoaște parola actuală, poate efectua modificări asupra datelor sensibile.

La trimitere datele sunt transmise serverului printr-o cerere de tip `POST`.

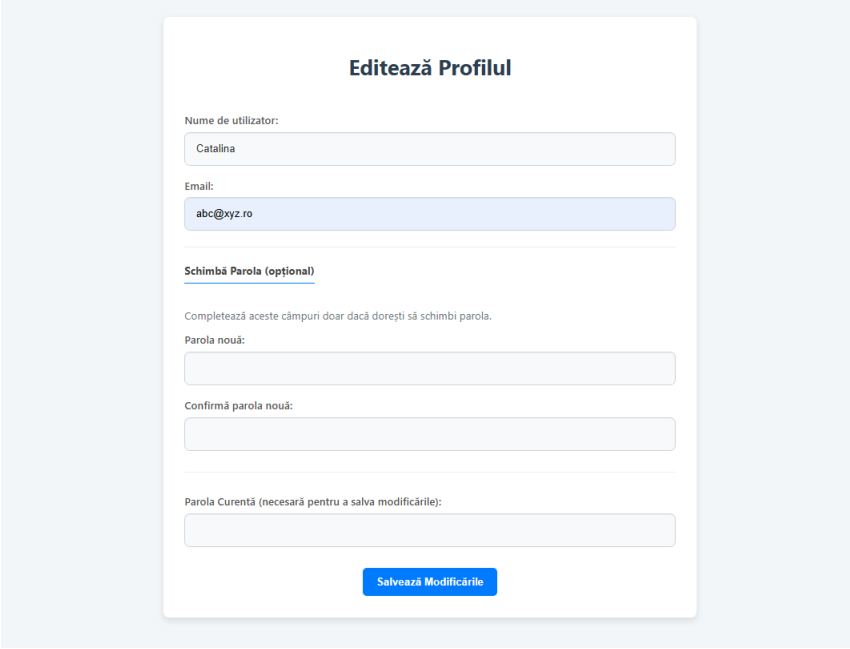


Figura 4.6: Pagina Editează Profilul

Logica backend

Logica din spatele acestei interfețe, gestionată printr-o cerere de tip `POST`, constă într-un proces multi-etapă care prioritizează securitatea și integritatea datelor.

Primul pas este verificarea identității utilizatorului. Aplicația recuperează hash-ul parolei curente din baza de date și utilizează funcția `bcrypt.checkpw()` pentru a asigura potrivirea cu parola introdusă în formular. În cazul unei neconcordanțe, procesul este oprit, iar utilizatorul primește un mesaj de eroare.

Următorul pas constă în validarea unicității datelor noi. Înainte de a efectua orice modificare, se execută interogări `SQL` pentru a verifica dacă numele de utilizator sau adresa de email propuse nu sunt deja asociate unui alt cont prevenind astfel crearea de duplicate.

Dacă utilizatorul a completat câmpurile pentru o parolă nouă și aceasta corespunde confirmării, parola este procesată prin `bcrypt.hashpw()` pentru a genera un hash securizat. Ulterior, se construiește și se execută o comandă `SQL UPDATE` care actualizează înregistrarea corespunzătoare din tabela `users`, modificând doar câmpurile efectiv schimbate.

În cazul modificării numelui de utilizator se actualizează și valoarea corespunzătoare din `session['username']`, astfel încât schimbarea să fie imediat vizibilă în interfața aplicației.

La finalul procesului utilizatorul este redirecționat către pagina de profil însoțit de un mesaj de confirmare a modificărilor.

4.8 Panoul de administrare

Panoul de administrare, accesibil prin ruta protejată `/admin`, este o componentă cu acces restricționat, proiectată pentru a oferi o imagine de ansamblu asupra întregii platforme. Spre deosebire de profilul de utilizator, care oferă o perspectivă personalizată și limitată la datele proprii, panoul de administrare operează la un nivel global servind ca un instrument de monitorizare și analiză a sistemului.

Accesul la această secțiune este securizat printr-un mecanism de control al accesului bazat pe roluri (RBAC), implementat direct în logica aplicației. La autentificare, sesiunea unui utilizator este marcată cu un indicator boolean `is_admin` preluat din baza de date. Fiecare cerere către ruta `/admin` validează prezența și veridicitatea acestui indicator în sesiune. În absența sa, accesul este imediat refuzat cu un cod de eroare `HTTP 403 (Forbidden)` asigurând o separare strictă și sigură a privilegiilor între utilizatorii normali și administratori. Mai mult, un administrator care navighează către ruta standard `/profile` este automat redirecționat către panoul `/admin`, susținând separarea clară a fluxului de lucru.

Panoul de administrare oferă funcționalități dedicate monitorizării și analizei globale a platformei. Administratorul poate vizualiza statistici agregate reprezentând indicatori cheie de performanță (KPIs) calculați la nivelul întregului sistem. Interogările `SQL` pentru această secțiune nu sunt filtrate după `user_id`, ci rulează pe întreg setul de date, oferind astfel o imagine completă a platformei. Printre metricile afișate se numără numărul total de utilizatori înregistrați, proprietăți listate, rezervări active și anulate, camere existente în sistem și sesiuni

active în timp real.

De asemenea administratorul beneficiază de instrumente de urmărire a numărului de rezervări. Prin intermediul unui formular interactiv, utilizatorul poate verifica gradul de ocupare a camerelor pentru o perioadă aleasă.

Panoul de administrare include, totodată, componente pentru monitorizarea stării sistemului. Acestea permit verificarea statusului serverelor Flask, facilitând identificarea rapidă a eventualelor probleme tehnice și garantând funcționarea neîntreruptă a platformei.

Prin aceste unelte rolul de administrator este clar definit ca unul de supervizare și management strategic, distinct de rolul tranzacțional al utilizatorului standard care este limitat strict la gestionarea propriilor date.

Prezentarea frontend

Interfața panoului de administrare este concepută ca un „dashboard” de business intelligence, prezentând administratorului o imagine clară a stării curente a platformei. Componentele vizuale includ cardurile de statistici care afișează date agregate esențiale despre platformă, precum numărul total de utilizatori înregistrați, activi, totalul proprietăților existente și numărul rezervărilor active sau anulate, oferind administratorului o imagine rapidă și completă asupra activității pe platformă.

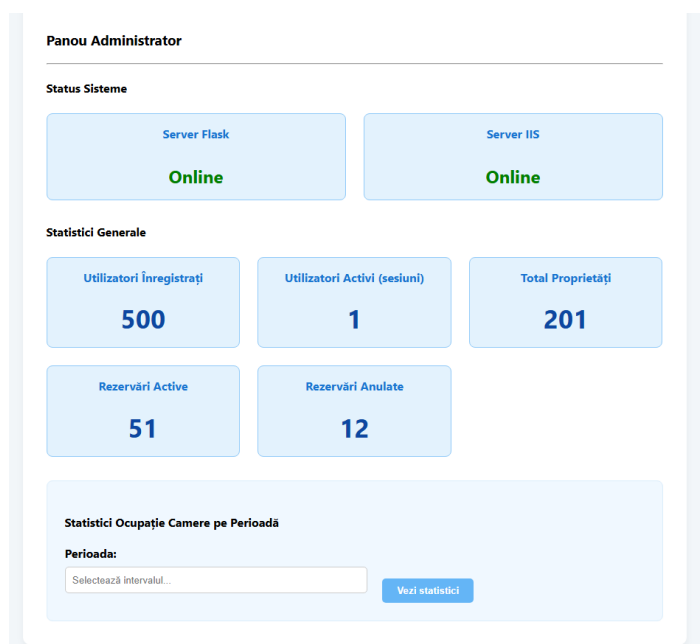


Figura 4.7: Panou Administrator

Analiza camerelor ocupate într-un anumit interval se face printr-un formular interactiv ce include un selector de dată (datepicker), permițând administratorului să stabilească o perioadă personalizată pentru raportare. La trimiterea formularului, backend-ul procesează informațiile și calculează atât numărul camerelor ocupate, cât și pe cel al camerelor disponibile în perioada selectată, oferind astfel o perspectivă detaliată și utilă asupra gradului de ocupare al platformei.

Indicatorii privind statusul serverelor oferă informații despre starea infrastructurii care

susține platforma. În primul rând, statusul serverului Flask indică dacă serverul de aplicații care rulează logica Python funcționează corect și răspunde cererilor utilizatorilor. În al doilea rând, statusul serverului IIS oferă date despre starea instanței secundare Flask, asigurând astfel o monitorizare completă a ambelor componente critice ale sistemului și permițând identificarea rapidă a eventualelor probleme tehnice.

Logica backend

Funcția `admin()` din backend este protejată printr-un mecanism de control al accesului și are un rol esențial în agregarea și prezentarea datelor relevante pentru administratori.

1. Controlul accesului bazat pe rol: La începutul execuției, funcția verifică dacă utilizatorul logat are rolul de administrator prin expresia `if not session.get('is_admin')`. Dacă valoarea `is_admin` din sesiune nu este `True`, accesul este refuzat imediat, iar serverul returnează codul de status HTTP 403 (`Forbidden`), prevenind accesul neautorizat la informațiile sensibile.
2. Agregarea statisticilor generale: Funcția execută o serie de interogări SQL simple, de tip `SELECT COUNT(*)`, pentru a calcula rapid statisticile de bază, precum numărul total de utilizatori, numărul total de proprietăți și alte date agregate relevante. Aceste informații oferă o imagine clară asupra activității platformei.
3. Calcularea ocupanței: Dacă administratorul trimite un interval de date prin formular funcția procesează aceste date și execută o interogare SQL mai complexă. Aceasta numără camerele distincte care au rezervări ce se suprapun cu intervalul selectat determinând gradul de ocupare și oferind o perspectivă utilă pentru gestionarea proprietăților.
4. Randarea datelor: Toate statisticile și informațiile calculate sunt transmise către template-ul `admin.html` unde sunt afișate într-un mod clar și accesibil, permițând administratorului să vizualizeze rapid datele esențiale și să ia decizii informate.

4.9 Fluxul de căutare și recomandări personalizate

Funcționalitatea de căutare este nucleul interactiv al platformei ce permite utilizatorilor să găsească și să rezerve cazare conform unor criterii specifice. Fluxul de căutare nu se limitează la o simplă filtrare, ci include un algoritm de recomandare care propune pachete optimizate de camere. Acest proces se desfășoară în mai multe etape gestionate de multiple rute și template-uri.

Etapa 1: Căutarea inițială

Procesul de căutare începe pe pagina principală prin intermediul formularului disponibil care colectează toate informațiile necesare pentru identificarea proprietăților disponibile. Formularul preia patru date esențiale: destinația dorită, care poate fi exprimată fie printr-un nume de locație, fie printr-un oraș, perioada călătoriei selectată cu ajutorul calendarului interactiv *Litepicker*, numărul de adulți și numărul de camere dorite. Odată completate și trimise aceste informații sunt transmise prin metoda `POST` către ruta `/search_results`, declanșând procesul de filtrare și prelucrare a datelor pe backend.

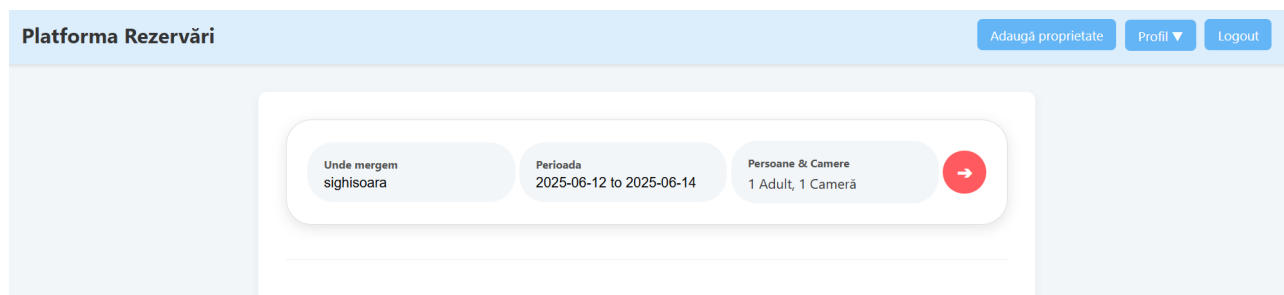
The screenshot shows a web interface for a reservation platform. At the top, there's a header with the title "Platforma Rezervări" and three buttons: "Adaugă proprietate", "Profil", and "Logout". Below the header is a search form with three input fields: "Unde mergem" (containing "sighisoara"), "Perioada" (containing "2025-06-12 to 2025-06-14"), and "Persoane & Camere" (containing "1 Adult, 1 Camera"). A red arrow button is positioned to the right of these fields.

Figura 4.8: Formularul de autentificare a utilizatorilor

La nivelul serverului funcția asociată rutei `/search_results` din fișierul `app.py` acționează ca un filtru inițial și ca mecanism de preprocesare a datelor introduse de utilizator. Mai întâi criteriile de căutare sunt extrase, iar o interogare SQL identifică toate proprietățile care corespund destinației selectate. Pentru fiecare proprietate identificată se realizează o interogare mai complexă care verifică disponibilitatea camerelor în perioada specificată de utilizator. În această etapă se utilizează funcția de recomandare `get_recommendations()` care validează posibilitatea construirii unui pachet de cazare complet și corect. Numai proprietățile care trec cu succes prin acest proces de filtrare și validare sunt trimise înapoi către frontend unde sunt afișate utilizatorului într-o formă clară și accesibilă, facilitând astfel luarea unei decizii informate privind rezervarea dorită.

Etapa 2: Afișarea rezultatelor

Template-ul frontend primește lista proprietăților eligibile și le afișează utilizatorului sub formă de carduri, oferind o prezentare clară și ușor de parcurs. Un aspect tehnic important constă în modul în care se construiește link-ul pentru fiecare rezultat. Fiecare link către pagina de vizualizare a proprietății (`view_property`) include nu doar `property_id`, ci și toți parametrii căutării inițiale, precum perioada selectată, numărul de adulți și numărul de camere. Această abordare asigură transmiterea completă a contextului căutării către etapa următoare, oferind posibilitatea utilizatorului să navigheze între pagini fără pierderea informațiilor introduse anterior.

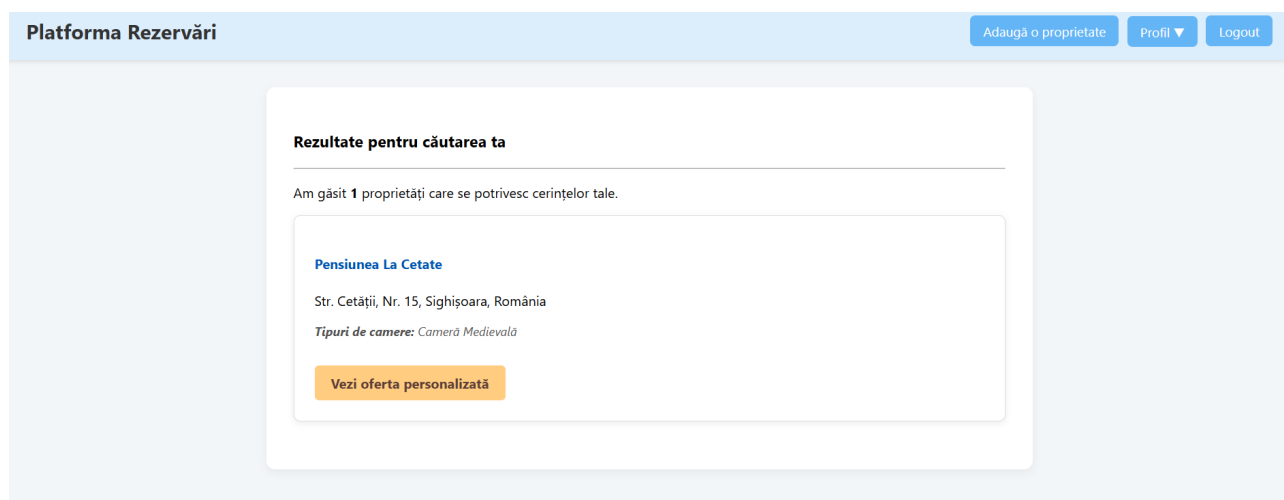
The screenshot shows the search results page. The header is the same as in Figure 4.8. The main content area has a title "Rezultate pentru căutarea ta" and a message "Am găsit 1 proprietăți care se potrivesc cerințelor tale." Below this is a card for a property named "Pensiunea La Cetate". The card displays the address "Str. Cetății, Nr. 15, Sighișoara, România" and the room type "Tipuri de camere: Cameră Medievală". At the bottom of the card is a button labeled "Vezi oferta personalizată".

Figura 4.9: Rezultatele căutării

Etapa 3: Calculatorul de disponibilitate și recomandări

Pagina de vizualizare a proprietății reprezintă componenta centrală a fluxului de căutare și rezervare. În partea superioară aceasta include un formular denumit „calculator”, care este pre-populat automat cu parametrii primiți de la pagina de rezultate. Utilizatorul are posibilitatea să modifice aceste criterii cum ar fi perioada dorită sau numărul de persoane și să re-trimită formularul către aceeași rută pentru a recalcula oferta disponibilă.

Sub calculator pagina afișează o listă de pachete de cazare personalizate generate de backend pe baza parametrilor introduși. În lipsa unei căutări active se afișează o listă generică cu toate tipurile de camere disponibile pentru proprietatea respectivă oferind utilizatorului o prezentare completă a opțiunilor.

The screenshot displays the 'Platforma Rezervări' (Reservation Platform) interface. At the top, there's a header with the platform name and user actions like 'Adaugă proprietate', 'Profil', and 'Logout'. The main content area features a card for 'Pensiunea La Cetate' with its address and a description. Below this is a search calculator with fields for 'Perioada' (2025-06-09 to 2025-06-10), 'Persoane' (1), and 'Camere' (1), along with a 'Caută / Actualizează' button. Underneath the calculator, it shows 'Opțiuni de rezervare pentru 1 persoane' (Reservation options for 1 person). A highlighted green box contains the recommendation: 'Opțiunea optimă (1 cameră)' (Optimal option (1 room)), specifically '1x Cameră Medievală (capacitate 2)' (1x Medieval Room (capacity 2)), priced at '380.00 lei / noapte' (380.00 lei / night), with a 'Rezervă Pachetul' (Reserve the package) button.

Figura 4.10: Calculatorul de disponibilitate și recomandări

La nivel de backend ruta `/property/<id>` gestionează logica cea mai complexă a aplicației. Aceasta primește parametrii de căutare și extrage toate camerele disponibile pentru proprietatea și perioada specificate. Ulterior apelează algoritmul de recomandare `get_recommendations()` care construiește și sortează pachetele optime, ținând cont de capacitatea necesară și preferințele utilizatorului. Pachetele rezultate împreună cu detaliile proprietății sunt transmise către template pentru afișare.

Algoritmul de recomandare (`get_recommendations()`)

Această funcție Python reprezintă nucleul inteligent al sistemului de căutare și recomandare. Ea primește ca intrare lista camerelor disponibile și cerințele utilizatorului și folosește `itertools.combinations` pentru a genera toate combinațiile posibile de camere. Combinările care nu satisfac capacitatea totală necesară sunt filtrate, iar combinațiile valide sunt sortate după un scor calculat pe baza priorităților: numărul de camere solicitat, capacitatea totală minimă necesară și, în final, prețul cel mai mic. Funcția returnează cele mai bune cinci pachete oferind utilizatorului opțiuni optimizate pentru rezervarea sa.

4.10 Adăugarea unei proprietăți noi

O funcționalitate esențială pentru proprietari este posibilitatea de a adăuga noi unități de cazare în platformă. Acest proces este gestionat de ruta `/add_property` și implică o interfață dinamică și o logică de backend capabilă să proceseze date structurate complex.

Prezentarea frontend

Interfața pentru adăugarea unei proprietăți este un formular care permite definirea atât a detaliilor generale ale proprietății, cât și a diverselor tipuri de camere disponibile în cadrul acesteia. Prima parte a formularului de adăugare a proprietății conține câmpuri text standard pentru completarea informațiilor generale, precum numele proprietății, adresa, orașul și țara. Aceste date oferă cadrul de bază necesar pentru identificarea și înregistrarea corectă a fiecărei proprietăți în sistem.

The image shows a web form titled "Adaugă o proprietate nouă". It contains four text input fields, each with a label and an example value:

- Numele proprietății:** Ex: Pensiunea Soarelui
- Adresa:** Ex: Str. Principală, Nr. 10
- Localitate:** Ex: Brașov
- Țară:** Ex: România

Figura 4.11: Formular pentru înregistrarea unei proprietăți noi

Secțiunea de gestiune a camerelor reprezintă componenta cea mai complexă a interfeței. Utilizatorul nu este limitat la un număr fix de tipuri de camere, ci poate adăuga dinamic noi tipuri de camere prin intermediul unui buton **Adaugă un tip de cameră**. Acest buton activează o funcție JavaScript care adaugă dinamic în pagină un nou set de câmpuri pentru definirea unui tip de cameră. Fiecare set permite introducerea unui tip de cameră distinct, cum ar fi „Cameră Dublă Standard”, împreună cu toate atributele sale: capacitatea, prețul pe noapte, numărul de camere identice și disponibilitatea generală. Astfel, utilizatorul poate adăuga oricâte tipuri de camere dorește și le poate elimina individual, oferind o flexibilitate completă în configurarea și gestionarea structurii de cazare.

Această metodă dinamică le oferă proprietarilor o flexibilitate sporită oferind posibilitatea să configureze cu precizie structura oricărei unități de cazare.

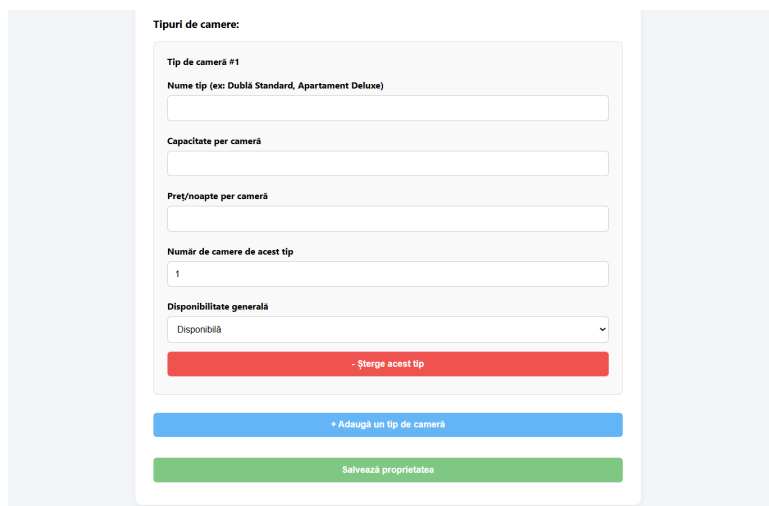
The image shows a web form titled "Tipuri de camere:" (Room Types:). It contains several input fields and buttons. The fields are: "Tip de cameră #1" (Room type #1), "Nume tip (ex: Dublu Standard, Apartament Deluxe)" (Type name), "Capacitate per cameră" (Capacity per room), "Preț/noapte per cameră" (Price/night per room), "Număr de camere de acest tip" (Number of rooms of this type) with a value of 1, and "Disponibilitate generală" (General availability) with a dropdown menu showing "Disponibilă" (Available). Below the fields are three buttons: a red button labeled "- Șterge acest tip" (Delete this type), a blue button labeled "+ Adaugă un tip de cameră" (Add a room type), and a green button labeled "Salvează proprietatea" (Save property).

Figura 4.12: Editarea avansată a caracteristicilor camerelor

Logica backend

Logica de backend pentru adăugarea unei noi proprietăți, gestionată de ruta `/add_property` la o cerere de tip POST, reprezintă una dintre cele mai complexe instrucțiuni de scriere din aplicație. Aceasta implică inserarea coordonată a datelor în multiple tabele (`properties` și `rooms`) menținând în același timp integritatea referențială. Pentru a garanta consistența datelor întreaga operație este încapsulată într-o tranzacție de bază de date evitând apariția unor înregistrări incomplete. Fragmentul de cod reprezentativ acestei funcții se poate regăsi la Anexa ??

Procesul de salvare a datelor, așa cum este implementat în codul prezentat anterior, respectă un flux logic bine definit menit să asigure atomicitatea operațiunilor în baza de date.

Întregul proces este încapsulat într-un bloc `with cnx.begin()`, care garantează că toate comenzile SQL executate în interiorul său sunt tratate ca o singură unitate atomică. Prima operațiune constă într-un `INSERT` în tabelul `properties` în care sunt adăugate detaliile generale ale proprietății, cum ar fi numele, adresa și alte informații relevante. ID-ul proprietarului este preluat din sesiunea curentă, iar imediat după inserare, cheia primară a noii înregistrări, `property_id`, este obținută pentru a fi utilizată în pașii următori.

Datele corespunzătoare diferitelor tipuri de camere adăugate dinamic în formularul frontend sunt primite sub formă de liste. Funcția `request.form.getlist()` permite extragerea eficientă a tuturor valorilor asociate fiecărui câmp, precum numele camerei, capacitatea, prețul pe noapte și alte atribute specifice.

Pentru inserarea camerelor care respectă relația 1:N între proprietate și camere, aplicația parcurge lista de tipuri de camere primite. Pentru fiecare tip de cameră se execută o buclă de un număr de ori egal cu numărul de camere specificat de utilizator. În cadrul acestei bucle fiecare cameră este inserată individual în tabela `rooms` fiind asociată proprietății-mamă prin `property_id` obținut anterior.

Finalizarea tranzacției se face automat la încheierea blocului `with cnx.begin()`: care efectuează un `COMMIT` dacă toate operațiunile s-au realizat fără erori salvând astfel atomic toate modificările în baza de date. În cazul apariției unei excepții, cum ar fi o eroare de constrângere sau o problemă de conectivitate, execuția blocului este întreruptă și se efectuează automat un `ROLLBACK`, anulând toate modificările din cadrul tranzacției și prevenind astfel salvarea inconsistentă a datelor.

4.11 Gestiunea proprietăților personale

Pagina „Cazările mele”, accesibilă prin ruta `/my-properties`, este un panou de control dedicat utilizatorilor care au rolul de proprietari. Această secțiune le permite să vizualizeze și să gestioneze toate unitățile de cazare pe care le-au înregistrat pe platformă.

Prezentarea frontend

Interfața acestei pagini este concepută pentru a oferi utilizatorului o imagine de ansamblu clară asupra proprietăților pe care le deține și un acces rapid la principalele acțiuni de management. Componenta centrală este reprezentată de o listă de carduri, unde fiecare card corespunde unei proprietăți. Informațiile esențiale, precum numele și adresa proprietății, sunt afișate pentru a facilita identificarea rapidă a fiecărei înregistrări.

Fiecare card include un set de butoane care permit utilizatorului să interacționeze direct cu proprietatea respectivă. Astfel, există un link pentru editarea proprietății care deschide formularul dedicat rutei `/edit_property/<id>`, un link către pagina de vizualizare și gestionare a rezervărilor unde proprietarul poate vedea toate rezervările active sau viitoare pentru acea proprietate și un buton de ștergere care declanșează un dialog de confirmare JavaScript (`onclick="return confirm(...)"`) pentru a preveni acțiunile accidentale.

În cazul în care utilizatorul nu a adăugat încă nicio proprietate pagina afișează un mesaj informativ și un link direct către formularul de adăugare ghidând utilizatorul către pasul următor și oferind o experiență clară și intuitivă.

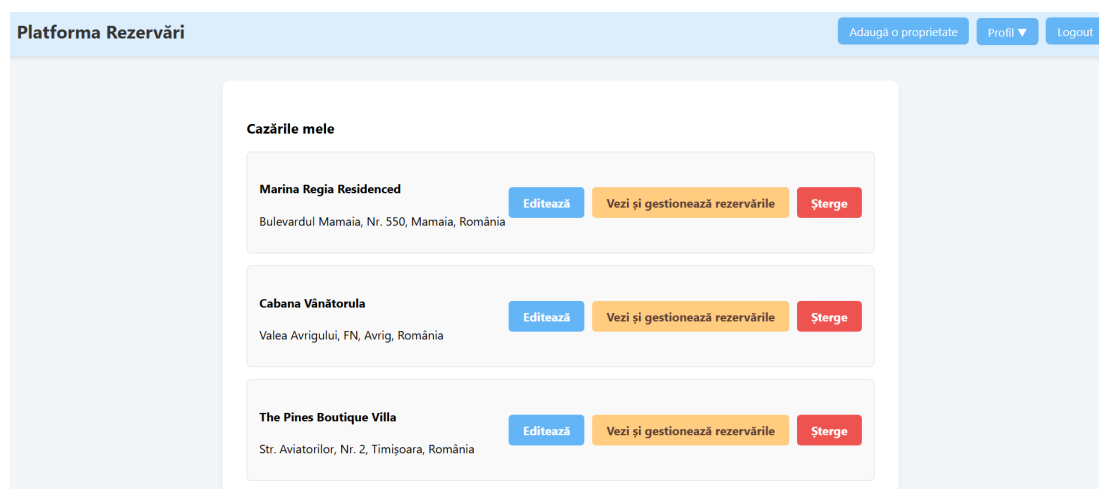


Figura 4.13: Prezentarea generală a proprietăților utilizatorului

Logica backend

Funcția din backend care servește această pagină are un rol simplu, dar esențial în menținerea securității și corectitudinii datelor. Prima verificare efectuată este aceea a autentificării utilizatorului, în cazul în care acesta nu este logat, este redirecționat către pagina de login.

Ulterior datele sunt filtrate astfel încât fiecare utilizator să poată vizualiza și gestiona doar proprietățile pe care le deține. Acest lucru se realizează printr-o interogare SQL de tip `SELECT` asupra tabelului `properties` care include clauza `WHERE owner_id = %s` unde valoarea este preluată din `session['user_id']`. Această filtrare este critică pentru a asigura separarea strictă a datelor între conturi.

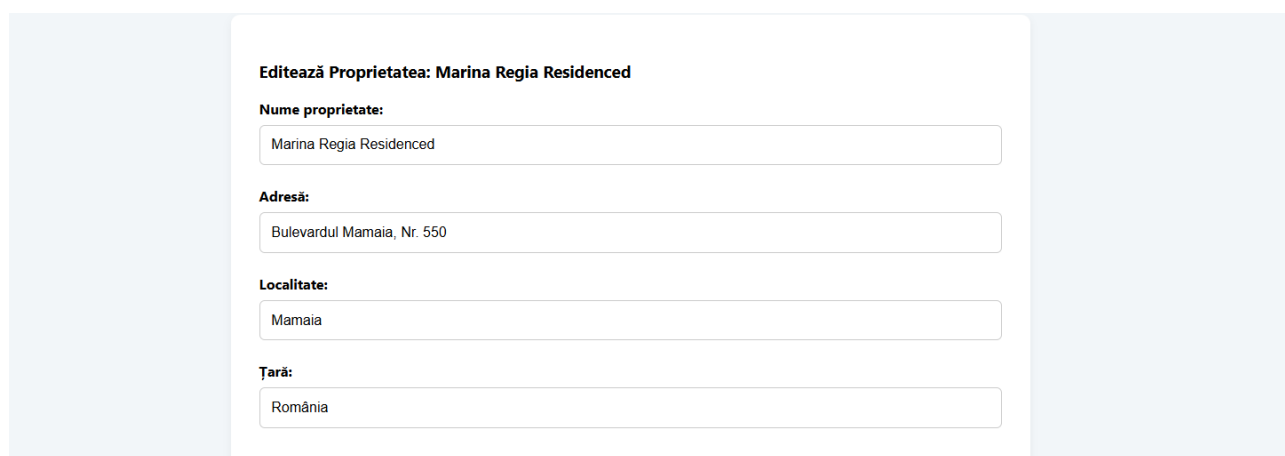
În final lista rezultată de proprietăți este transmisă către template-ul `my_properties.html` unde este afișată dinamic utilizatorului oferind o prezentare clară și interactivă a tuturor înregistrărilor disponibile.

4.12 Modificarea unei proprietăți existente

Funcționalitatea de editare a unei proprietăți este una dintre cele mai complexe din aplicație care asigură proprietarilor un control precis asupra unităților de cazare listate. Procesul este gestionat de ruta dinamică `/edit_property/<property_id>` și implică o interfață interactivă, validări asincrone și o logică tranzacțională robustă în backend.

Prezentarea frontend

Interfața de editare (figura 4.14) este un formular avansat care la încărcare este pre-populat cu toate datele existente ale proprietății și ale camerelor asociate. Prima parte a formularului include câmpurile pentru detaliile generale ale proprietății, cum ar fi numele și adresa, oferind utilizatorului posibilitatea de a le modifica după nevoie.



Editează Proprietatea: Marina Regia Residenced

Nume proprietate:

Adresă:

Localitate:

Țară:

Figura 4.14: Interfața de editare a detaliilor generale ale proprietății

Componenta centrală a interfeței este secțiunea de gestionare a camerelor. Tipurile de camere existente sunt afișate în grupuri distincte unde fiecare grup include atribute precum prețul, capacitatea și numărul de camere disponibile. Utilizatorul poate modifica aceste atribute, poate adăuga tipuri noi de camere prin intermediul butonului **Adaugă un tip nou de cameră** care declanșează o funcție JavaScript sau poate șterge un tip de cameră existent (figura 4.15).

Figura 4.15: Gestionarea dinamică a tipurilor de camere pentru o proprietate

O caracteristică importantă a interfeței este mecanismul de validare asincronă la ștergerea unui tip de cameră. La apăsarea butonului de ștergere se execută o funcție JavaScript care trimite o cerere asincronă (AJAX/Fetch) către endpoint-ul `/check_room_type_reservations/`. Acest endpoint verifică în timp real dacă există rezervări active pentru tipul de cameră selectat, iar doar dacă nu există rezervări, interfața permite eliminarea vizuală a grupului de câmpuri oferind feedback instantaneu și prevenind erorile.

Logica backend

La nivel de backend procesul este gestionat prin două rute principale: `/edit_property/<id>` și `/check_room_type_reservations/`.

Ruta `/edit_property/<id>` gestionează atât cererile de tip `GET`, cât și cele de tip `POST`. Pentru cererile `GET`, se verifică dacă utilizatorul autentificat este proprietarul proprietății. Dacă această condiție este îndeplinită toate informațiile despre proprietate și camerele asociate sunt preluate și trimise către frontend pentru a popula formularul.

În cazul cererilor `POST` se aplică o logică tranzacțională care actualizează detaliile proprietății și ale camerelor existente, adaugă tipuri noi de camere și le elimină pe cele șterse. Dacă o operație va eșua tranzacția este anulată printr-un `ROLLBACK`, asigurând astfel integritatea bazei de date.

Ruta AJAX `/check_room_type_reservations/` gestionează cererile venite din partea frontend-ului, primind ca parametri un ID de proprietate și un nume de tip de cameră. Aceasta interoghează baza de date pentru rezervările active corespunzătoare și returnează un răspuns în format JSON care indică dacă ștergerea tipului de cameră este sigură.

4.13 Ștergerea unei proprietăți

Funcționalitatea de ștergere a unei proprietăți reprezintă o operațiune distructivă care necesită un nivel ridicat de securitate și validare pentru a preveni pierderea accidentală a datelor și pentru a menține integritatea bazei de date. Procesul este inițiat din interfața de management a proprietarului și este gestionat de o rută dedicată în backend.

Prezentarea frontend

Pe pagina „Cazările mele”, fiecare proprietate afișată are atașat un buton de ștergere. Pentru a preveni ștergerile accidentale, acest buton nu declanșează imediat acțiunea, ci folosește o măsură de siguranță la nivelul clientului. Astfel la apăsarea butonului este apelată funcția nativă `return confirm(...)` a browser-ului care afișează o fereastră modală ce solicită confirmarea explicită a utilizatorului. Ștergerea este trimisă către server doar dacă utilizatorul confirmă acțiunea prin apăsarea butonului „OK”. În caz contrar, procesul este anulat.

Logica backend

La nivel de backend ștergerea este tratată ca o operațiune securizată și tranzacțională cu verificări riguroase înainte de a modifica baza de date. Prima verificare constă în validarea autentificării utilizatorului și a dreptului de proprietate. Se verifică dacă utilizatorul este logat și se execută o interogare `SELECT` pentru a confirma că `user_id`-ul curent corespunde cu `owner_id`-ul proprietății ce urmează a fi ștersă. În lipsa acestei potriviri, operațiunea este imediat oprită, prevenind astfel accesul neautorizat.

O altă verificare importantă constă în respectarea regulilor de business. Proprietățile care au rezervări active sau viitoare nu pot fi șterse. Se execută o interogare suplimentară în tabela `reservations` pentru a identifica rezervările confirmate a căror dată de sfârșit este în viitor. Dacă se găsește cel puțin o astfel de rezervare, ștergerea este anulată, iar proprietarul este informat.

În cazul în care toate verificările sunt satisfăcute se inițiază operațiunile de ștergere. Deși schema bazei de date include constrângeri `ON DELETE CASCADE`, codul implementează explicit ștergerea datelor din tabelele dependente (`reservation_details`, `reservations`, `rooms`) înainte de eliminarea înregistrării principale din tabela `properties`. Toate aceste operații sunt efectuate într-o singură tranzacție. La final se apelează `cnx.commit()` pentru a confirma ștergerile, iar în caz de eroare, context managerul anulează automat întreaga tranzacție pentru a menține baza de date într-o stare consistentă.

4.14 Procesul de rezervare

Procesul de rezervare, de la alegerea unei proprietăți până la confirmarea finală, este un flux complex care implică mai multe rute și componente menite să asigure atât integritatea datelor, cât și o experiență clară și intuitivă pentru utilizator. Fluxul începe cu afișarea rezultatelor căutării unde utilizatorul poate examina proprietățile disponibile și poate alege una

dintre acestea. Odată selectată proprietatea, pagina de vizualizare a acesteia permite inițierea procesului de rezervare prin selectarea perioadei dorite și a numărului de persoane.

Pagina `search_results.html` afișează proprietățile relevante sub formă de carduri, fiecare card având un link dedicat care direcționează utilizatorul către pagina de vizualizare detaliată a proprietății. Această organizare oferă o imagine clară a opțiunilor disponibile și facilitează navigarea rapidă către proprietatea dorită.

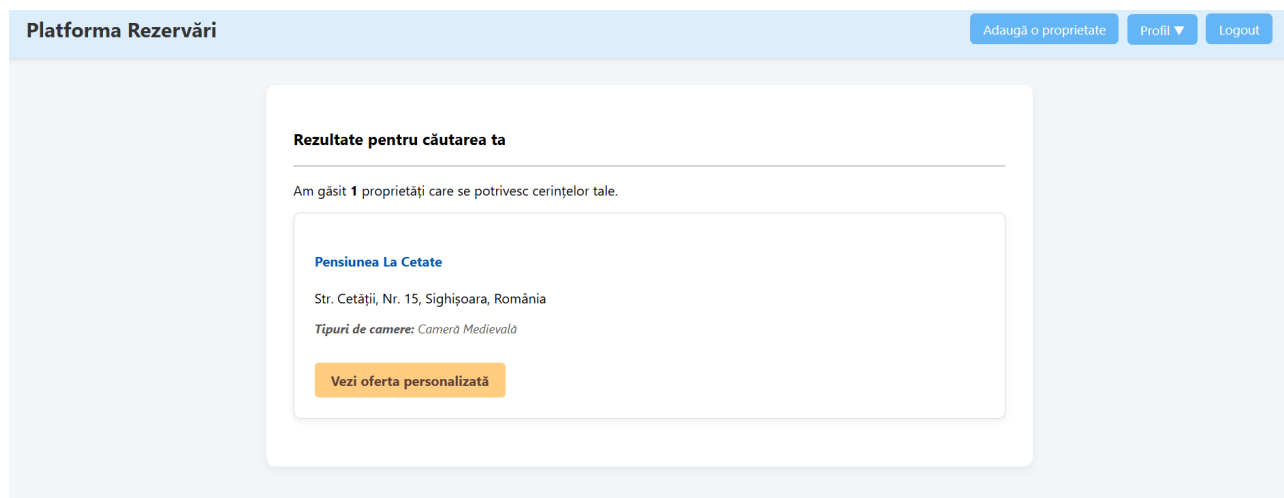


Figura 4.16: Lista proprietăților rezultate în urma căutării

Pagina `view_property.html` include „calculatorul” de disponibilitate, pre-populat automat cu parametrii căutării inițiale. Utilizatorul poate vizualiza pachetele de cazare recomandate sau poate ajusta criteriile pentru a recalcula oferta. Atunci când utilizatorul alege un pachet și apasă butonul de rezervare, acesta este redirecționat către ruta de confirmare a rezervării.

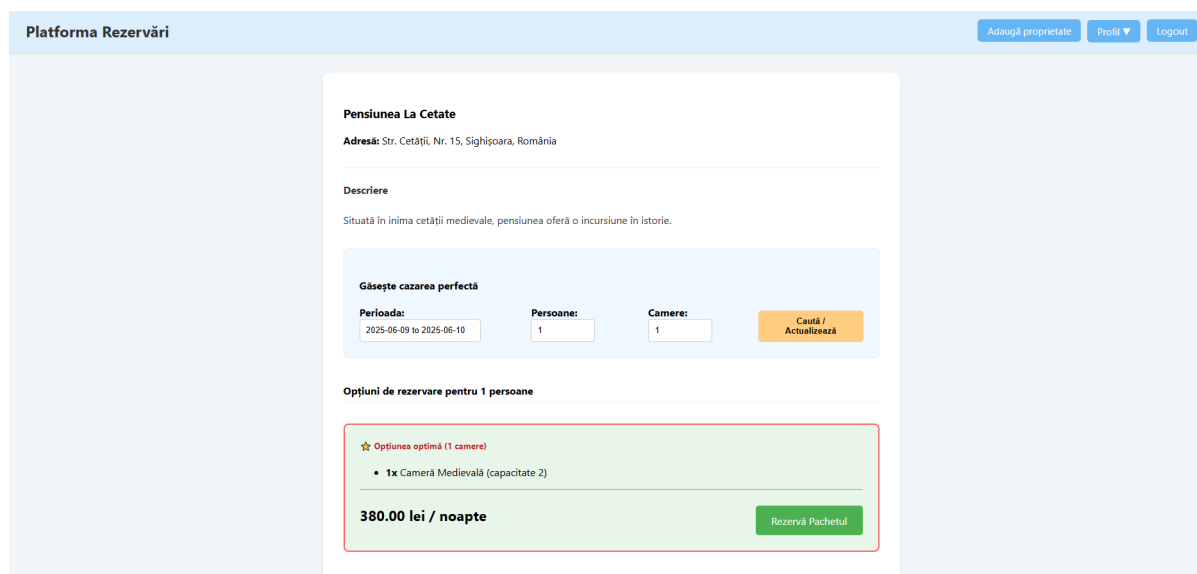


Figura 4.17: Pagina cu detalii și opțiuni de rezervare pentru o proprietate

La nivel de backend funcția `view_property()` preia parametrii de căutare transmiși prin URL sau formular. Aceasta extrage toate camerele disponibile pentru proprietatea și perioada specificată și apelează algoritmul `get_recommendations()` pentru a construi și sorta pachetele optime de cazare. Rezultatul, împreună cu detaliile proprietății, este transmis către template pentru afișare. În cazul în care nu se găsește nicio combinație de camere care să satisfacă cerințele utilizatorului, pagina afișează un mesaj informativ pentru menține claritatea și transparența procesului de rezervare.

4.15 Confirmarea și finalizarea rezervării

Odată ce utilizatorul a ales un pachet de camere este redirecționat către pagina de rezervare unde completează detaliile de contact și finalizează tranzacția.

Prezentarea frontend

Pagina de confirmare a rezervării afișează un sumar detaliat al pachetului de camere selectat. Aceasta include perioada rezervării, numărul de nopți, detaliile fiecărei camere, precum numele, capacitatea și prețul pe noapte, precum și prețul total de plată. De asemenea, pagina conține un formular pentru introducerea datelor de contact ale clientului, cum ar fi numele complet, telefonul și adresa de email, pentru a finaliza rezervarea.

Confirmare și Finalizare Rezervare

Sumar Pachet

Perioada: **09 iunie 2025 - 10 iunie 2025** (1 noapte)

1x Cameră Medievală (2 pers.) - 380.00 lei/noapte

Total de plată: **380.00 lei**

Date de contact

Nume complet:

Ion Gigel

Telefon:

0712345678

Email:

abc@xyz.ro

Confirmă și Trimite Rezervarea

Figura 4.18: Formular de rezervare

Logica backend

Logica backend pentru ruta `/booking/confirm` gestionează atât afișarea formularului de confirmare prin metoda `GET`, cât și procesarea finală a rezervării prin metoda `POST`. Ruta necesită ca utilizatorul să fie autentificat și preia ID-urile camerelor și datele de start și end din URL, validând perioada selectată. Pentru cererile de tip `GET`, backend-ul preia detaliile camerelor și ale proprietății, calculează prețul total și trimite aceste informații către template pentru afișare. În cazul cererilor de tip `POST` se inițiază o tranzacție în baza de date pentru a asigura atomicitatea operațiunilor. Backend-ul re-verifică disponibilitatea camerelor și prețurile direct în baza de date, iar pentru fiecare cameră din pachet se înserează o înregistrare în tabelul `reservations`, în timp ce detaliile de contact și prețul total sunt salvate într-un tabel separat. Dacă toate operațiunile se execută cu succes, tranzacția este confirmată prin `cnx.commit()`, iar în caz de eroare este anulată prin `cnx.rollback()`, prevenind astfel rezervările parțiale. La final utilizatorul este redirecționat către pagina de succes unde poate vizualiza confirmarea și detaliile rezervării.

4.16 Pagina de confirmare a rezervării

Această pagină este responsabilă pentru afișarea unui mesaj de succes și a detaliilor rezervării după ce aceasta a fost confirmată.

Prezentarea frontend

Pagina de succes afișează un mesaj vizual de confirmare cu mesajul „Mulțumim, rezervarea a fost realizată cu succes!”. Aceasta detaliază complet rezervarea și include informații despre proprietate, perioada rezervării, numărul de nopți, prețul total și datele clientului. De asemenea, pagina include un buton care permite utilizatorului să fie redirecționat către secțiunea „Rezervările mele” pentru a vizualiza toate rezervările sale.

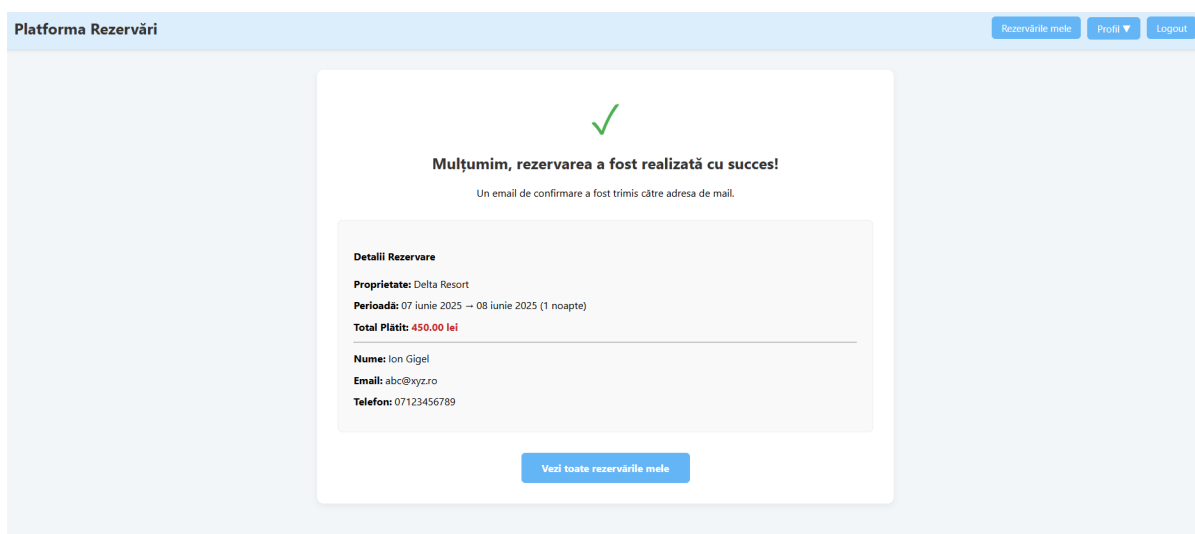


Figura 4.19: Pagina de confirmare a rezervării

Logica backend

La nivel de backend ruta `/booking/success` gestionează afișarea detaliilor rezervării după ce aceasta a fost confirmată cu succes. În primul rând, se verifică dacă utilizatorul este autentificat și dacă există un `last_booking_id` stocat în sesiune. Pe baza acestui ID, backend-ul preia toate detaliile rezervării din baza de date, inclusiv camerele, proprietatea, perioada și informațiile clientului. În final aceste date sunt transmise către template-ul `reservation_success.html` care le afișează într-un format clar și accesibil pentru utilizator.

4.17 Vizualizarea și gestionarea rezervărilor

Secțiunea „Rezervările mele” este un panou de control dedicat utilizatorilor non-administratori, oferindu-le posibilitatea de a vizualiza și gestiona toate rezervările pe care le-au efectuat în cadrul platformei.

Prezentarea frontend

Pagina „Rezervările mele” oferă o vedere clară și acces rapid la detalii esențiale despre rezervările utilizatorului. Fiecare rezervare este afișată într-un card separat care conține numele proprietății, numele camerei rezervate, perioada completă a rezervării cu datele formate prietenos, statusul curent (de exemplu, „confirmed” sau „cancelled”), prețul total (dacă este disponibil) și detaliile de contact ale clientului, precum numele complet, emailul și numărul de telefon. Pentru rezervările cu status „confirmed” este afișat un buton „Anulează Rezervarea” care include o confirmare JavaScript prevenind anulările accidentale și sporind siguranța.

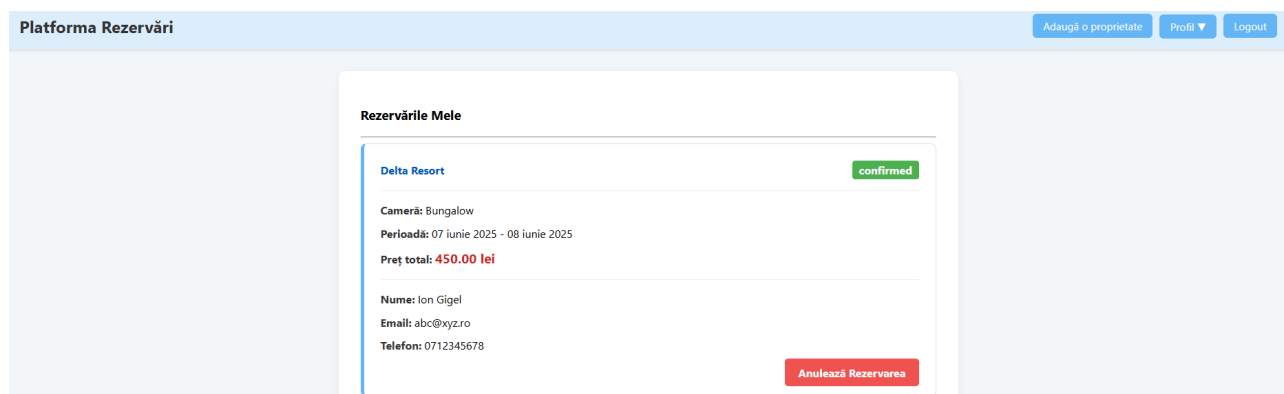


Figura 4.20: Interfața paginii „Rezervările mele”

Logica backend

Funcția de backend asociată rutei `/my-reservations` preia și pregătește datele pentru afișarea în interfață. Mai întâi, verifică dacă utilizatorul este autentificat. Dacă nu, este redirecționat către pagina de login pentru a proteja datele personale. Pentru utilizatorii autentificați, funcția realizează o interogare complexă în baza de date folosind instrucțiuni JOIN pentru a prelua toate rezervările legate de `user_id`-ul din sesiune. Aceasta combină informații din tabelele `reservations`, `rooms`, `properties` și `reservation_details`. Interogarea aduce

toate detaliile necesare pentru afișare, precum numele camerei, al proprietății, datele de contact ale clientului și prețul total. Rezultatele sunt ordonate descrescător după ID-ul rezervării pentru a oferi o vizualizare cronologică. Datele sunt apoi transmise către template-ul `my_reservations.html` care construiește dinamic interfața afișată utilizatorului.

4.18 Anularea rezervării

Această pagină permite utilizatorilor să anuleze o rezervare existentă cu verificări stricte ale drepturilor și ale stării rezervării.

La nivel de backend ruta responsabilă pentru anularea unei rezervări necesită ca utilizatorul să fie autentificat. Funcția verifică mai întâi drepturile de acces, asigurându-se că utilizatorul logat este fie creatorul rezervării, fie proprietarul proprietății asociate. În caz contrar, anularea rezervării nu este permisă.

Dacă rezervarea este deja anulată sistemul afișează un mesaj informativ pentru utilizator. Dacă toate verificările sunt trecute cu succes, statusul rezervării este actualizat în baza de date la valoarea „cancelled”. Modificarea este confirmată printr-un `cnx.commit()`, iar în final utilizatorul este redirecționat fie către pagina „Rezervările mele”, fie către pagina de gestionare a rezervărilor proprietății, în funcție de context.

4.19 Gestionarea rezervărilor proprietăților personale

Această secțiune este un panou de control dedicat proprietarilor de proprietăți, oferindu-le o vizualizare centralizată a tuturor rezervărilor active și viitoare aferente unei proprietăți specifice. Spre deosebire de pagina generală „Rezervările mele” (care afișează rezervările făcute de utilizator), aceasta listează rezervările primite pentru una dintre proprietățile sale.

Prezentarea frontend

Interfața este concepută pentru a oferi proprietarului o imagine clară asupra stării rezervărilor pentru o anumită proprietate. Pe lângă numele, adresa și orașul proprietății afișate, pagina prezintă o listă de carduri, fiecare reprezentând o rezervare individuală. Fiecare card include numele și ID-ul camerei rezervate, perioada rezervării — cu data de început și data de sfârșit, statusul rezervării, numele complet al clientului, adresa sa de email și numărul de telefon, precum și prețul total al rezervării.

Dacă statusul rezervării este „confirmed” este afișat butonul „Anulează Rezervarea”, care include o confirmare JavaScript pentru a preveni anulările accidentale. Acest buton trimite către backend un parametru de redirecționare (`redirect_to_prop_id`), astfel încât utilizatorul să fie readus pe pagina de gestionare a rezervărilor proprietății curente. De asemenea, pagina oferă un buton „Înapoi la proprietățile mele” pentru o navigare facilă.

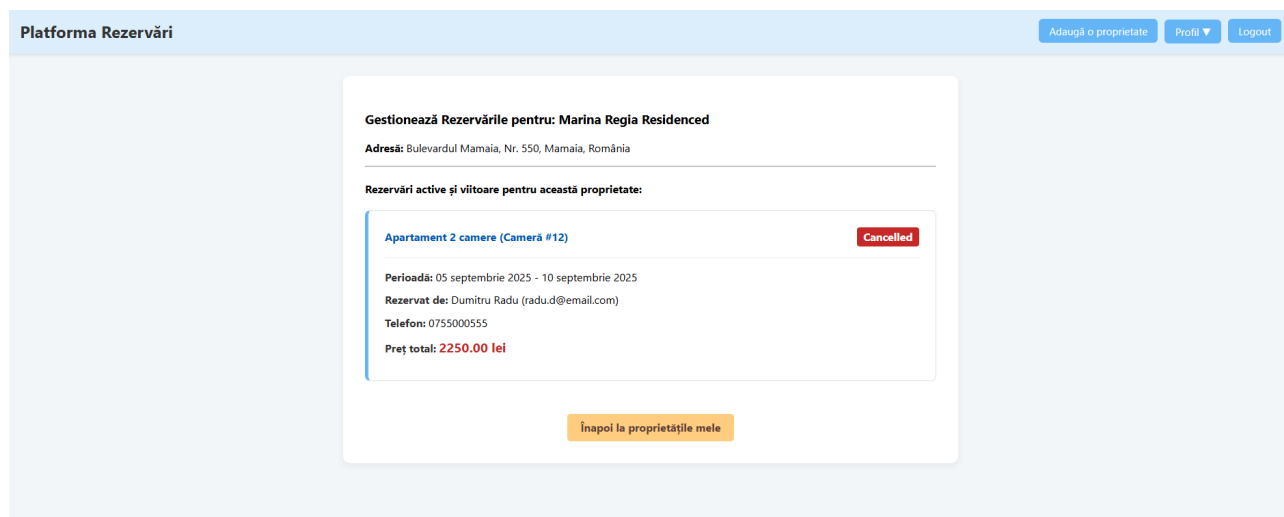


Figura 4.21: Vizualizarea rezervărilor pentru o proprietate

Logica backend

Logica de backend asociată acestei rute se ocupă de preluarea și afișarea datelor relevante pentru proprietar.

În primul rând, se verifică autentificarea utilizatorului și se asigură că acesta este proprietarul proprietății specificate în URL. În caz contrar, accesul este interzis și utilizatorul este redirectionat. Apoi se extrag detaliile complete ale proprietății vizate, incluzând numele, adresa, orașul și țara, pentru a fi afișate în interfață. În continuare, se execută o interogare în baza de date pentru a obține toate rezervările asociate proprietății, folosind JOIN-uri cu tabelele `rooms` și `reservation_details` pentru a aduna informații despre camere, datele clientului și preț. Se filtrează doar rezervările active sau viitoare, adică acelea al căror `end_date` este mai mare sau egal cu data curentă, iar rezultatele sunt sortate după data de început a rezervării. Toate datele colectate sunt transmise către template-ul `manage_property_reservations.html` unde sunt afișate dinamic pentru proprietar.

5 Arhitectura de suport și înaltă disponibilitate

5.1 Introducere în arhitectura de suport

Acest capitol se concentrează pe elementele fundamentale ale infrastructurii și arhitecturii de suport care asigură scalabilitatea, reziliența și performanța necesare unei aplicații web moderne, capabile să gestioneze un trafic considerabil și să funcționeze fără întreruperi. La nivel infrastructural, modul în care aplicația este găzduită și administrată este la fel de important pentru succesul său ca și logica de afaceri. Lucrarea analizează mecanisme cheie precum **load balancing**-ul, replicarea bazelor de date și rolul virtualizării în testarea și implementarea unei astfel de arhitecturi.

5.2 Rolul Nginx în contextul arhitecturii implementate

Așa cum a fost prezentat în secțiunea 2.10, Nginx are un rol esențial în menținerea scalabilității și rezilienței aplicației web, prin funcționalitățile sale de load balancing și reverse proxy. În arhitectura implementată pentru sistemul de rezervări, Nginx este configurat să gestioneze eficient traficul către instanțele aplicației Flask, contribuind direct la optimizarea performanței și disponibilității platformei.

Aplicarea load balancing-ului în arhitectura proiectului

În arhitectura distribuită prezentată Nginx configurat pe stația de lucru principală (Host), acționează ca punct unic de intrare pentru cererile utilizatorilor. Aceste cereri sunt distribuite între cele două instanțe ale aplicației Flask, una locală (Host) și cealaltă pe VM 2, asigurând o repartizare echilibrată a încărcării și prevenind supraîncărcarea unei singure instanțe. Distribuția activă a traficului permite gestionarea unui volum ridicat de cereri concurente și menținerea unor timpi de răspuns rapizi, chiar în condiții de trafic intens. Totodată, Nginx poate detecta instanțele indisponibile și redirecționa traficul către cele funcționale, contribuind astfel la reziliența și disponibilitatea ridicată a stratului de aplicație.

Integrarea Nginx cu Flask ca reverse proxy și servirea fișierelor statice

Nginx este integrat ca reverse proxy pentru a facilita comunicarea securizată și eficientă între utilizatori și serverele Flask. Configurația Nginx este adaptată pentru a recunoaște cererile dinamice destinate logicii aplicației Flask și a le transmite către grupul de servere definite, asigurând propagarea corectă a antetelor HTTP esențiale (**Host**, **X-Real-IP**, **X-Forwarded-For**). Un aspect crucial al acestei integrări este rolul Nginx în servirea fișierelor statice (CSS, JavaScript, imagini). Prin configurarea Nginx să servească direct aceste resurse, se reduce semnificativ sarcina pe serverele Flask, care pot astfel dedica mai multe resurse procesării logicii de business și a cererilor dinamice. Această optimizare îmbunătățește considerabil viteza de încărcare a paginilor și experiența generală a utilizatorului.

Pentru o exemplificare concretă a implementării, fragmentul de cod de mai jos, extras din fișierul de configurare Nginx, definește grupul de servere backend și regulile de proxy.

```
# Definirea grupului de servere (instanțe Flask)
upstream flask_backend {
    # Distribuie cererile prin algoritmul Round Robin (implicit)
    server 192.168.50.1:5000;    # Instanța Flask de pe Host
    server 192.168.50.3:5000;    # Instanța Flask de pe VM 2
}

server {
    listen 80;
    server_name 192.168.50.1;

    # Regula pentru cererile dinamice
    location / {
        proxy_pass http://flask_backend;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Regula pentru servirea fișierelor statice (proxy către IIS)
    location /static {
        proxy_pass http://192.168.50.3;
    }
}
```

Fragmentul 3: Configurația Nginx pentru load balancing și reverse proxy

Blocul `upstream` definește cele două instanțe Flask între care Nginx va distribui traficul. Directiva `proxy_pass` din blocul `location /` redirecționează toate cererile dinamice către acest grup, realizând funcția de load balancing.

5.3 Impactul pozitiv al arhitecturii distribuite asupra aplicației

Folosirea Nginx ca load balancer și reverse proxy aduce beneficii semnificative pentru fiecare flux de lucru al aplicației de rezervări.

În cazul înregistrării și autentificării, chiar și atunci când traficul este intens, cererile critice sunt distribuite uniform, asigurând un proces fluid și responsiv. Reziliența asigurată de Nginx previne întreruperile de funcționare cauzate de defectarea unei instanțe Flask.

Pentru căutarea și vizualizarea proprietăților, flux care poate genera un volum ridicat de cereri de citire, distribuirea încărcării contribuie la menținerea unor timpi de răspuns reduși. În plus, servirea fișierelor statice direct de către Nginx reduce presiunea asupra serverelor de aplicație.

În ceea ce privește crearea unei rezervări, un proces tranzacțional complex, distribuirea cererilor inițiale către servere diferite asigură capacitatea aplicației de a răspunde rapid. Nginx contribuie la păstrarea unei conexiuni stabile cu serverul Flask responsabil de procesarea

rezervării.

De asemenea, fluxurile de management, atât pentru proprietari, cât și pentru administratori, beneficiază de aceeași reziliență, asigurând un control eficient al platformei chiar și în condiții de încărcare ridicată.

În concluzie, Nginx acționează ca un dirijor al traficului, asigurând nu doar o distribuție eficientă a sarcinii, ci și o experiență de utilizare fluidă și neîntreruptă.

5.4 Importanța replicării MySQL (Master-Slave) în optimizarea performanței

Scalabilitatea și disponibilitatea ridicată a stratului de date sunt esențiale pentru performanța și reziliența sistemului de rezervări, în special datorită caracterului predominant „read-heavy” al aplicațiilor web de acest tip. Pentru a satisface aceste cerințe arhitectura implementată utilizează un mecanism robust de replicare MySQL Master-Slave, cu nodul Master găzduit pe stația de lucru principală (Host) și nodul Slave pe VM 1. Această configurație permite optimizarea operațiilor de citire și scriere, precum și asigurarea continuității serviciului în caz de defecțiuni.

Implementarea separării operațiunilor de citire și scriere

Unul dintre beneficiile majore ale configurației Master-Slave, esențial pentru optimizarea performanței, este posibilitatea de a separa instrucțiunile de citire (**SELECT**) de cele de scriere (**INSERT**, **UPDATE**, **DELETE**). În cadrul lucrării, nodul MySQL Master (pe Host) este configurat să gestioneze exclusiv toate operațiile de scriere asigurând consistența și integritatea datelor. Concomitent, nodul MySQL Slave (pe VM 1) preia operațiile de citire. Această distribuție a sarcinilor maximizează debitul și scade încărcarea nodului Master.

Impactul replicării asupra fiabilității și performanței sistemului

Replicarea MySQL și separarea citirilor de scrieri au un impact major asupra fluxurilor de lucru din aplicație.

Fluxurile de căutare și vizualizare a proprietăților sunt în principal orientate spre citiri. Toate aceste operații sunt direcționate către nodul Slave, ceea ce reduce considerabil sarcina pe nodul Master și garantează timpi de răspuns rapizi chiar și în condiții de încărcare ridicată.

În cazul înregistrării și autentificării, fluxurile sunt gestionate strategic. Instrucțiunile de scriere la înregistrare vizează nodul Master, iar pentru a garanta consistența imediată a datelor, tot nodul Master este interogată și la autentificare, o decizie de proiectare menită să evite erorile cauzate de întârzierea de replicare. În schimb, operațiunile frecvente de citire, precum afișarea profilului, sunt direcționate către nodul Slave.

Procese de creare, modificare și anulare a rezervărilor sunt operații critice care implică exclusiv scrieri și sunt gestionate în totalitate de nodul Master. Performanța acestuia este esențială, iar faptul că nu este încărcat suplimentar cu cereri de citire contribuie la succesul

tranzacțiilor. Mai mult, în cazul unei defecțiuni, nodul Slave poate fi promovat ca Master printr-un mecanism de failover, asigurând continuitatea serviciului.

În ceea ce privește fluxurile de management pentru proprietari și administratori, operațiile de adăugare sau modificare sunt efectuate pe Master, în timp ce citirile necesare pentru afișarea listelor de proprietăți sunt preluate de pe Slave. Această abordare garantează o interfață de administrare rapidă și eficientă. Astfel, arhitectura Master-Slave nu doar că optimizează viteza, ci adaugă și un strat esențial de reziliență și fiabilitate la nivelul datelor.

5.5 Rolul virtualizării în testarea arhitecturii

Implementarea și testarea arhitecturii distribuite a sistemului de rezervări a presupus un mediu flexibil și izolat. Infrastructura de virtualizare a fost esențială în acest proces oferind posibilitatea de a crea un mediu de testare care reproduce fidel condițiile de producție, fără costurile și complexitatea asociate hardware-ului fizic.

Crearea unui mediu de testare

Virtualizarea a permis crearea de mașini virtuale care rulează sisteme de operare și aplicații într-un mediu complet izolat de hardware-ul gazdă. Această izolare asigură un nivel înalt de consistență și reproductibilitate, permițând recrearea cu ușurință a oricărei configurații sau erori pentru depanare. De asemenea, oferă independență față de hardware, asigurând că dezvoltarea și testarea se desfășoară într-un mediu uniform și controlat.

Testarea scenariilor de eșec și a rezilienței

Un avantaj major al virtualizării este capacitatea de a testa scenarii de eșec într-un mod controlat. Aceste teste sunt esențiale pentru validarea mecanismelor de **failover** și a rezilienței sistemului:

- Verificarea rezilienței componentelor: Se poate opri intenționat o instanță **Flask** pentru a observa dacă **Nginx** detectează indisponibilitatea și redirecționează traficul către celelalte instanțe. De asemenea, oprirea serverului **MySQL Master** permite testarea procesului de promovare a unui **Slave**.
- Evaluarea scalabilității sistemului: Prin lansarea rapidă a unor noi mașini virtuale se poate simula o creștere a traficului și se poate analiza modul în care sistemul se adaptează la cerințele crescute.
- Izolarea și controlul problemelor: Mediul virtual limitează impactul defectelor unei componente asupra restului sistemului, contribuind la menținerea stabilității globale.

Această abilitate de a genera defecțiuni controlate este extrem de valoroasă pentru identificarea și remedierea punctelor slabe înainte de lansarea în producție.

Flexibilitatea în configurarea mediilor de dezvoltare

Virtualizarea oferă o flexibilitate semnificativă în configurarea mediilor de dezvoltare. Utilizarea snapshot-urilor permite salvarea stării unei mașini virtuale și revenirea rapidă la o versiune stabilă în caz de erori. De asemenea, clonarea mediilor permite mai multor dezvoltatori să lucreze simultan pe copii identice ale sistemului. Nu în ultimul rând, virtualizarea se integrează perfect în procesele moderne de CI/CD, contribuind la menținerea calității codului.

6 Testarea performanței și analiza scalabilității

6.1 Introducere în testarea de performanță

Asigurarea scalabilității și fiabilității a constituit un obiectiv central al acestei lucrări, având în vedere specificul unei aplicații web de rezervări, care trebuie să susțină un număr mare de utilizatori și interacțiuni simultane. Testarea de performanță (load testing) reprezintă o etapă esențială în ingineria software, deoarece permite analizarea modului în care sistemul răspunde la sarcini ridicate. Aceasta implică simularea unui trafic intens, cu utilizatori și cereri concurente, pentru a măsura capacitatea de procesare, stabilitatea și timpii de răspuns ai aplicației. Prin aplicarea unor scenarii apropiate de utilizarea reală, pot fi identificate punctele critice, evaluate limitele de scalabilitate și determinate pragurile maxime de trafic gestionabil. Scopul acestei etape a fost validarea soluțiilor de scalare implementate, respectiv echilibrarea traficului cu Nginx și replicarea MySQL Master-Slave într-un context funcțional și realist.

6.1.1 Alegerea instrumentului de testare: Locust

Pentru realizarea testelor de performanță, s-a optat pentru instrumentul open-source Locust. Alegerea Locust a fost justificată de următoarele avantaje care se aliniază cerințelor proiectului:

Locust este un instrument de testare a performanței care se integrează natural în cadrul proiectului, deoarece este programabil în Python, un limbaj deja folosit prin framework-ul Flask. Această caracteristică permite definirea comportamentului utilizatorilor virtuali direct în cod, oferind flexibilitatea de a construi scenarii complexe și realiste, adaptate exact fluxurilor aplicației de rezervări.

Un alt aspect important este faptul că Locust nu se bazează pe înregistrarea și redarea simplă a cererilor HTTP, ci permite descrierea programatică a acțiunilor utilizatorilor. Astfel, se pot modela pași concreți, cum ar fi căutarea unei proprietăți, vizualizarea detaliilor și inițierea unei rezervări, ceea ce conduce la o simulare mult mai apropiată de interacțiunile reale ale utilizatorilor.

Instrumentul este conceput și pentru distribuție și scalabilitate. Testele pot rula la scară largă, deoarece Locust suportă un mod distribuit în care mai multe instanțe „workers” pot fi lansate pe mașini diferite și coordonate de o instanță „master”. În acest mod se poate genera un volum masiv de cereri, depășind cu mult capacitatea unei singure mașini de testare.

În plus, Locust oferă o interfață web intuitivă care permite monitorizarea în timp real a testelor. Prin intermediul acesteia, pot fi urmărite metrici cheie precum numărul de cereri pe secundă (RPS), timpii medii de răspuns, deviația standard, rata de eșec și distribuția timpilor de răspuns pe percentilă. Această vizibilitate imediată facilitează analiza performanței sistemului și luarea rapidă a deciziilor în timpul testării.

Pentru a evalua performanța aplicației într-un mod cât mai realist, a fost construit un set de scenarii de testare folosind Locust. Fiecare scenariu, definit ca un task (`@task`) distinct

în fișierul `locustfile.py`, simulează un comportament specific al utilizatorului. Frecvența executării acestor scenarii a fost calibrată astfel încât să reflecte un mix de activități comparabil cu cel dintr-un mediu de producție.

Primul scenariu este cel de `browse_and_search`, care reproduce navigarea generală pe platformă. Utilizatorul virtual accesează pagina principală (`/home`), vizualizează rezervările proprii (`/my-reservations`) și efectuează o căutare de proprietăți (`/search_results`). Acest flux, folosit cel mai des în testare, permite evaluarea performanței **operațiunilor de citire**, a generării paginilor dinamice și a direcționării corecte a interogărilor către nodul Slave al bazei de date.

Al doilea scenariu, `profile_and_edit`, simulează gestionarea contului de utilizator. Acesta acoperă atât vizualizarea profilului (`/profile`), cât și actualizarea datelor personale (`/edit-profile`), inclusiv schimbarea parolei. În acest mod sunt testate atât operațiuni de citire, cât și de scriere, cu accent pe subsistemul de management al utilizatorilor.

Un alt scenariu relevant este `full_booking_flow`, care simulează întregul proces de rezervare. Utilizatorul caută o proprietate, o selectează, vizualizează detaliile acesteia și finalizează rezervarea prin transmiterea datelor de contact. Acest test evaluează integritatea și performanța proceselor de scriere complexe, precum și modul în care nodul Master gestionează tranzacțiile atomice sub sarcină.

Scenariul `manage_properties` simulează activitățile unui proprietar, cum ar fi adăugarea unei noi proprietăți (`/add_property`), vizualizarea listei existente (`/my-properties`) și editarea unei proprietăți deja definite. Acest flux acoperă un set complet de operațiuni **CRUD**, testând atât operațiunile de scriere, cât și cele de citire din baza de date.

În final, sunt incluse și scenarii pentru autentificare și delogare. La începutul fiecărei sesiuni, utilizatorul virtual se conectează prin metoda `on_start`, iar ulterior task-ul `test_logout` simulează ieșirea din cont. Aceste acțiuni permit verificarea constantă a performanței mecanismelor de autentificare și gestionare a sesiunilor.

Un aspect esențial pentru realismul simulării este includerea funcției `get_static_files` în toate scenariile. Aceasta imită comportamentul JS, (imagini) aferente fiecărei pagini accesate. Astfel, testarea nu se limitează la logica de business din backend, ci verifică și capacitatea serverului web (Nginx/IIS) de a furniza eficient conținut static.

6.2 Procesul de execuție și monitorizare

Execuția testelor de performanță cu **Locust** a urmat un proces structurat, desfășurat în mai multe etape. În primul rând, **Locust** a fost lansat fie în mod master-slave, pentru rularea pe mai multe mașini virtuale, fie în mod standalone pentru teste de dimensiuni reduse. Instanțele worker au fost pornite pe VM-uri dedicate, iar traficul generat a fost direcționat către serverul Nginx.

Configurarea testului s-a realizat din interfața web a **Locust**, accesibilă la adresa `http://localhost:8089` pe Host. Aici au fost introduse parametrii principali ai testului, precum numărul de utilizatori virtuali simulați („Number of users to simulate”), rata de creștere a acestora („Spawn rate”) și alte opțiuni specifice scenariilor definite.

Pe durata execuției, monitorizarea performanței s-a făcut în timp real prin intermediul graficelor și tabelelor oferite de **Locust**. Printre indicatorii urmăriți s-au numărat rata de cereri pe secundă (RPS), timpii medii de răspuns, deviația standard a acestora, rata de eșec (procentul de cereri ce au returnat erori HTTP sau de rețea) și distribuția timpilor de răspuns exprimată prin percentile (de exemplu, 90% dintre cereri răspund în mai puțin de un anumit număr de milisecunde).

În paralel cu monitorizarea din **Locust**, resursele hardware ale fiecărei mașini implicate (gază, VM 1 și VM 2) au fost urmărite cu ajutorul Task Manager, concentrându-se asupra utilizării procesorului și a memoriei RAM.

La final, pentru a completa analiza, log-urile generate de Nginx, Flask și MySQL au fost investigate. Acestea au oferit informații utile privind erorile, avertismentele sau eventualele anomalii apărute sub sarcină, contribuind la identificarea posibilelor probleme de performanță sau securitate.

6.3 Arhitectura mediului de testare

Mediul de testare a fost proiectat pentru a reproduce cât mai fidel arhitectura de producție propusă, asigurând relevanța și acuratețea rezultatelor obținute. Componentele au fost distribuite pe mașini virtuale, pentru a simula un mediu real de tip cloud și pentru a permite izolarea resurselor.

Pentru a simula condiții cât mai apropiate de cele din producție, mediul de testare a fost alcătuit din trei mașini interconectate într-o rețea locală (LAN) configurată manual. Fiecare dintre aceste noduri a îndeplinit un rol bine definit în arhitectura distribuită, după cum este detaliat în continuare.

Stația de lucru principală (Host):

- Procesor: Intel Core i5-9300H quad-core cu 8 fire de execuție, frecvență de bază 2.4 GHz și mod turbo până la 4.1 GHz.

- Memorie RAM: 16 GB DDR4.
- Sistem de operare: Windows 10 Pro.
- Servicii active: Serverul **MySQL Master** (responsabil cu operațiunile de scriere și replicare), o instanță a aplicației **Flask**, serverul **Nginx** configurat ca load balancer și instanța principală **Locust**, pentru coordonarea testelor de performanță.

VM1:

- Sistem de operare: Windows Server 2022.
- Resurse alocate: 4 GB RAM și 4 nuclee CPU.
- Servicii active: Serverul **MySQL Slave** (dedicat operațiunilor de citire și replicării datelor de la Master) și workerii **Locust** care generează trafic concurent către aplicație pentru testarea încărcării.

VM2:

- Sistem de operare: Windows Server 2022.
- Resurse alocate: 4 GB RAM și 2 nuclee CPU.
- Servicii active: O instanță suplimentară a aplicației **Flask** care preia cererile distribuite prin serverul **Nginx**.

Această configurație hardware și software a fost aleasă pentru a oferi un echilibru optim între performanță și consumul de resurse, asigurând în același timp condiții reale de testare a arhitecturii distribuite a sistemului.

6.3.1 Pregătirea și rularea testelor Locust

Pentru a evalua scalabilitatea și reziliența arhitecturii sistemului de rezervări a fost configurat un set de scenarii de testare folosind **Locust** cu scopul de a simula cât mai fidel comportamentul real al utilizatorilor. Testele au acoperit un parcurs reprezentativ al interacțiunii cu platforma ce includ autentificarea, navigarea între pagini, căutarea de proprietăți, gestionarea contului și realizarea rezervărilor. În plus, fiecare interacțiune declanșată a fost completată cu solicitarea automată a resurselor statice aferente (precum fișiere CSS și JavaScript) pentru a reproduce cât mai exact încărcarea reală asupra serverului în condiții de utilizare normală. Execuția testelor de performanță a urmat un proces de configurare și rulare bine definit, menit să asigure relevanța și reproductibilitatea rezultatelor.

Pe parcursul etapei preliminare de testare am evaluat comportamentul ambelor arhitecturi sub sarcini progresive simulând succesiv 50, 70, 100 și 150 de utilizatori concurenți. Rezultatele au evidențiat o diferență clară de performanță și stabilitate. În timp ce arhitectura monolitică a început să prezinte instabilitate și erori încă de la pragul de 70 de utilizatori, arhitectura distribuită a gestionat această sarcină eficient, fără a înregistra erori sau întreruperi. Numărul de 70 de utilizatori a fost selectat ca punct de referință maxim pentru analiza comparativă detaliată, întrucât a reprezentat limita superioară la care arhitectura distribuită a menținut o funcționare stabilă, fără erori. În același timp, acest prag a evidențiat în mod clar

vulnerabilitățile arhitecturii monolitice, care a început să manifeste instabilitate sub aceeași încărcare.

Odată ce mediul de testare a fost pregătit, **Locust** a fost inițiat din terminal prin comanda `locust -f locustfile.py` care a pornit interfața web de control accesibilă la adresa `http://localhost:8089`. În interfața web a fost configurat testul de stres final cu următorii parametri:

- Număr total de utilizatori simulați: 70
- Rată de generare (Spawn rate): 10 utilizatori pe secundă
- Host țintă: `http://192.168.50.1`

Testul a fost lăsat să ruleze pentru o perioadă de cel puțin 15 minute după atingerea numărului maxim de utilizatori, pentru a permite sistemului să se stabilizeze și pentru a colecta date de performanță relevante pe termen lung.

6.4 Scenarii de Testare Implementate

Evaluarea performanței a fost realizată printr-o serie de teste menite să evidențieze comportamentul sistemului în condiții de utilizare intensă. Această secțiune detaliază scenariile de testare realizate cu ajutorul **Locust**, metodologia de execuție adoptată, precum și indicatorii utilizați pentru evaluarea comportamentului sistemului în condiții de încărcare crescută. Obiectivul a fost de a simula un trafic cât mai realist și de a colecta date precise care să permită o comparație obiectivă între arhitectura monolitică și cea distribuită. Analiza performanței s-a concentrat pe trei categorii principale de indicatori:

- Debitul (Requests Per Second): Reprezintă numărul de cereri pe care sistemul le poate procesa într-o secundă și reflectă capacitatea totală de procesare.
- Fiabilitatea (exprimată prin rata de eșec): Indică procentul de cereri care nu au fost finalizate cu succes, constituind un parametru critic pentru stabilitatea sistemului.
- Timpul de răspuns: Măsoară latența aplicației adică intervalul de timp dintre transmiterea unei cereri și primirea răspunsului. Pentru o caracterizare mai detaliată, au fost analizate două percentile:
 - Mediana (percentila 50): Descrie experiența utilizatorului tipic, astfel încât jumătate dintre cereri au un timp de răspuns mai mic decât această valoare.
 - Percentila 95: Reflectă performanța în condiții mai puțin favorabile, indicând că 95% dintre cereri sunt mai rapide decât această valoare.
 - Percentila 99: Oferă o perspectivă asupra celor mai lente răspunsuri din sistem, fiind un indicator sensibil la vârfurile de încărcare și la eventuale blocaje. Practic, doar 1% dintre cereri depășesc acest timp.

Pentru a evalua atât comportamentul inițial, cât și stabilitatea pe termen lung, analiza datelor s-a realizat pe două momente cheie: **după 2-3 minute** de la începutul testului, în timpul

perioadei de încărcare și **după 15 minute** odată ce sistemul a atins un regim de funcționare stabil.

6.4.1 Cazul 1: arhitectura monolitică

Acest scenariu de testare are rolul de a stabili performanța de referință, simulând comportamentul unei arhitecturi tradiționale, monolitice, pentru aplicația web de rezervări, fără mecanisme active de scalare sau load balancing. Obiectivul principal constă în înregistrarea metricilor de performanță inițiale care vor servi ulterior drept punct de comparație pentru evaluarea îmbunătățirilor aduse de soluțiile distribuite implementate.

Pentru a reproduce configurația monolitică, cadrul de testare a fost simplificat plasând toate componentele esențiale pe un singur sistem.

O singură instanță a aplicației web Flask a fost pornită rulând pe portul implicit 5000, fiind responsabilă atât de logica funcțională, cât și de procesarea directă a cererilor HTTP. Serverul principal MySQL, funcțional pe sistemul gazdă, a gestionat toate operațiile cu baza de date.

Pe durata testării, instanțele virtuale VM 1 și VM 2 au fost dezactivate, serverul MySQL secundar nu a fost utilizat, iar a doua instanță Flask nu a rulat. Componenta Nginx a fost de asemenea oprită, astfel încât cererile generate de Locust au fost direcționate direct către aplicația Flask de pe sistemul gazdă, fără implicarea unui load balancer sau proxy invers. Baza de date MySQL principală a fost populată cu un set de date de test (100 de utilizatori, 200 de proprietăți și 1150 de unități).

Rezultatele după 2-3 minute

După primele 2-3 minute de rulare a testului sistemul a atins rapid un debit de aproximativ 191 RPS cu un total de 19 532 de cereri procesate și 0% eșecuri în acest interval.

Timpii de răspuns analizați în primele minute de test evidențiază o discrepanță semnificativă între diferitele tipuri de cereri, indicând puncte critice de stres în arhitectura monolitică. Mediana agregată pentru toate cererile a fost de 16 ms, ceea ce sugerează că experiența tipică a utilizatorului a fost rapidă. În schimb, percentila 95 agregată a atins 180 ms, iar percentila 99 a fost de 440 ms, demonstrând că o parte semnificativă a cererilor a întâmpinat întârzieri considerabile, cu unele cereri extrem de lente. Acest comportament general al sistemului este ilustrat vizual în graficele de performanță din Anexa 6.

Discrepanța se datorează comportamentului diferit al endpoint-urilor, în special al operațiunilor de scriere:

Cererile simple de citire și pentru resurse statice, precum `GET /static/...` sau `GET /my-reservations` au fost procesate foarte rapid cu o mediană de 6 ms și, respectiv, 26 ms.

Endpoint-ul tranzacțional `POST /login` a înregistrat o mediană extrem de ridicată de 60 000 ms (60 de secunde) reprezentând un punct de blocaj major ce a afectat resursele serverului. Percentilele 95 și 99 pentru acest endpoint au fost de 61 000 ms, confirmând persistența blocajului.

Impactul acestui blocaj s-a resimțit și la alte rute. De exemplu, `GET /profile` a avut o mediană de 32 ms, însă media a fost de 611,47 ms. Această diferență indică faptul că multe cereri au fost blocate temporar, așteptând eliberarea resurselor ocupate de operațiunile lente de login.

Utilizarea resurselor a fost, de asemenea, monitorizată. CPU-ul a funcționat la aproximativ 83% cu frecvența de 2,71 GHz, iar memoria RAM s-a situat la 86% din totalul disponibil, utilizând 13,6 GB din 15,8 GB.

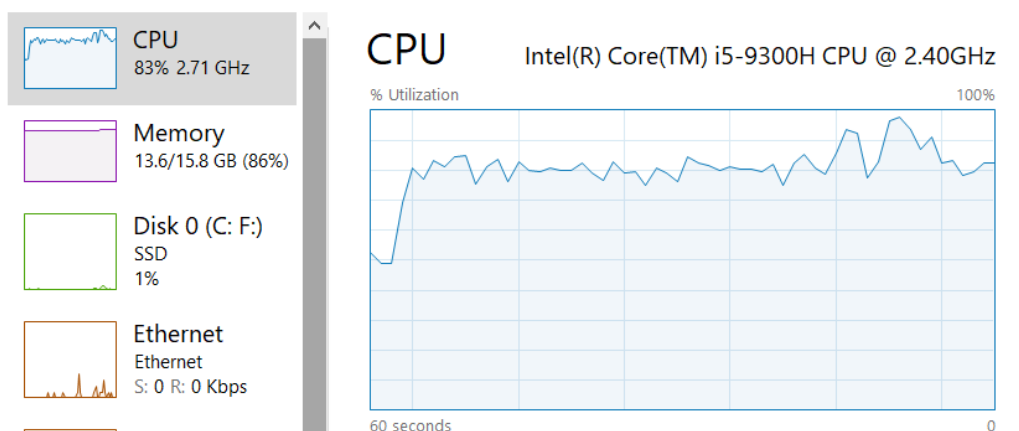


Figura 6.1: Utilizarea resurselor după 2-3 minute de test

Rezultatele după 15 minute

După primele 15 minute instabilitatea sistemului monolitic a devenit clară demonstrând incapacitatea acestuia de a susține o sarcină constantă. Deși debitul mediu raportat a fost de 217,2 RPS această valoare este înșelătoare deoarece maschează perioadele semnificative în care sistemul a înregistrat erori. Cel mai relevant indicator al acestei instabilități este rata de eșec de 4% calculată pe parcursul celor 168 533 de cereri agregate (Anexa 7).

În ceea ce privește timpii de răspuns s-au observat diferențe semnificative între cereri. Endpoint-urile rapide care implică predominant operațiuni de citire și acces la resurse statice, precum `GET /static/[...]` cu o mediană de 8 ms sau `GET /my-reservations` cu o mediană de 11 ms, au continuat să înregistreze performanțe foarte bune. Percentila 95 pentru aceste rute a rămas de asemenea la parametri de funcționare optimi - 27 ms, respectiv 48 ms.

Problema principală a persistat la endpoint-ul `POST /login` care a continuat să înregistreze o mediană de **60 000 ms (60 de secunde)**. Valorile percentilelor 95 și 99 de 61 000 ms, indică faptul că acest blocaj a rămas constant și a afectat majoritatea încercărilor de autentificare.

Consecința directă a acestui blocaj a dus la instabilitatea aplicației, ceea ce a generat eșecuri periodice și a condus la o experiență de utilizare îngreunată pentru un număr semnificativ de utilizatori.

Utilizarea resurselor a rămas constantă. CPU-ul s-a stabilizat la 65%, iar memoria RAM a continuat să fie utilizată în proporție de 72-73%.

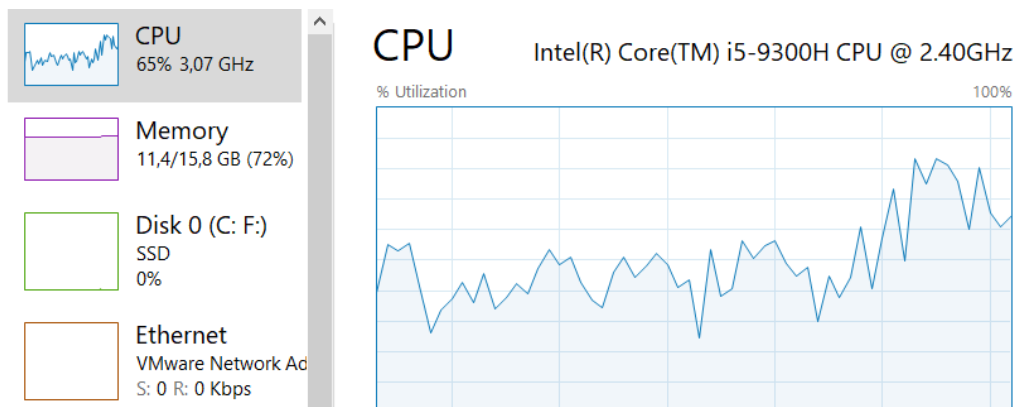


Figura 6.2: Resurse utilizate după 15 minute

Interpretarea rezultatelor pe baza graficelor

Monitorizarea testului de performanță a fost realizată utilizând graficele generate de Locust care au evidențiat principalii indicatori: numărul de cereri pe secundă (RPS), timpii de răspuns și numărul de utilizatori activi.

Analiza graficului pentru RPS prezentat în Figura 6.3 evidențiază o instabilitate semnificativă a arhitecturii monolitice sub sarcină. Deși sistemul atinge un debit de peste 200 RPS, se observă scăderi bruște și periodice în care rata de procesare se apropie de zero, în special în intervalele 11:34 și 11:36. Aceste episoade de colaps coincid cu vârfurile erorilor reprezentate de linia roșie și indică un număr crescut de cereri eșuate pe secundă.

Comportamentul sistemului este caracterizat prin perioade de funcționare normală urmate de eșecuri în cascadă. Acesta reflectă incapacitatea arhitecturii monolitice de a menține performanța sub sarcină constantă și evidențiază punctele de stres generate de operațiile blocante.

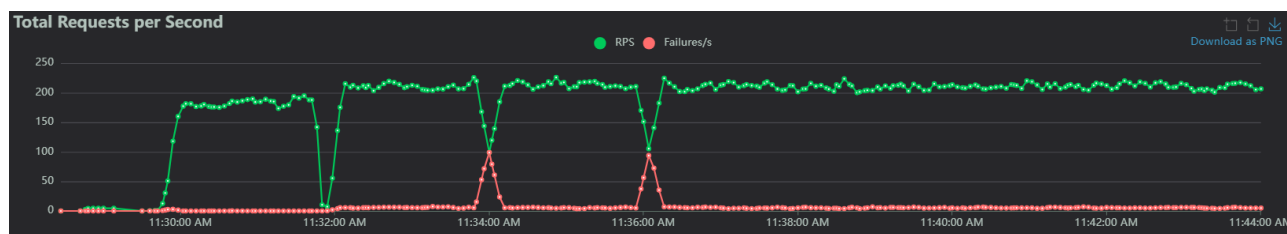


Figura 6.3: Număr de cereri pe secundă (RPS)

Graficul timpilor de răspuns prezentat în Figura 6.4 scoate în evidență ineficiența arhitecturii monolitice sub sarcină. Se remarcă un vârf inițial pronunțat al latenței, în care percentila 95 (linia violet) atinge valoarea critică de 60 000 ms (60 de secunde). Acest vârf este

cauzat de cererile de tip `POST /login` care, așa cum s-a demonstrat în analiza rezultatelor după primele 2–3 minute, blochează resursele sistemului.

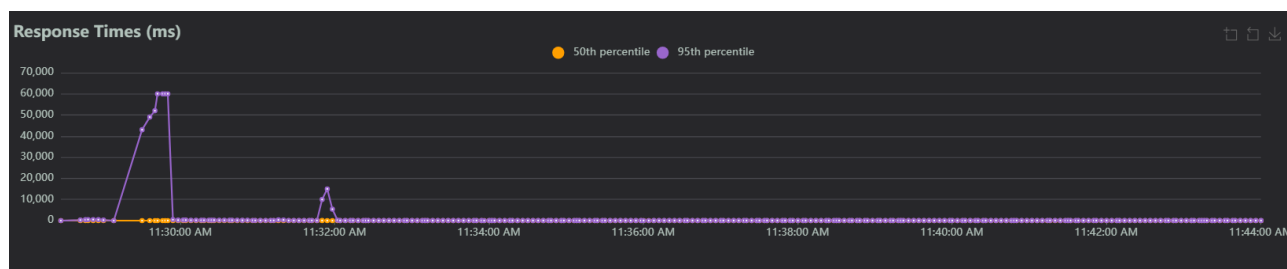


Figura 6.4: Timpii de răspuns (ms)

După depășirea acestui vârf inițial timpii de răspuns pentru majoritatea cererilor se stabilizează la valori relativ scăzute reflectate de linia galbenă a medianei care rămâne aproape de zero. Cu toate acestea percentila 95 înregistrează ocazional creșteri ale latenței care, deși mai puțin severe, indică persistența unor blocaje intermitente.

Aceste variații corelate cu vârfurile periodice de eșecuri vizibile în graficul RPS sugerează faptul că arhitectura monolitică nu gestionează eficient instrucțiunile intensive de scriere. În consecință, o parte semnificativă a utilizatorilor experimentează întârzieri considerabile ceea ce duce la o degradare a experienței pentru utilizatorii afectați de trafic.

Graficul numărului de utilizatori activi prezentat în Figura 6.5 arată că testul a rulat constant cu 70 de utilizatori simultan confirmând faptul că instabilitatea nu a fost cauzată de fluctuații ale numărului de utilizatori, ci de incapacitatea sistemului de a gestiona sarcina impusă de aceștia.

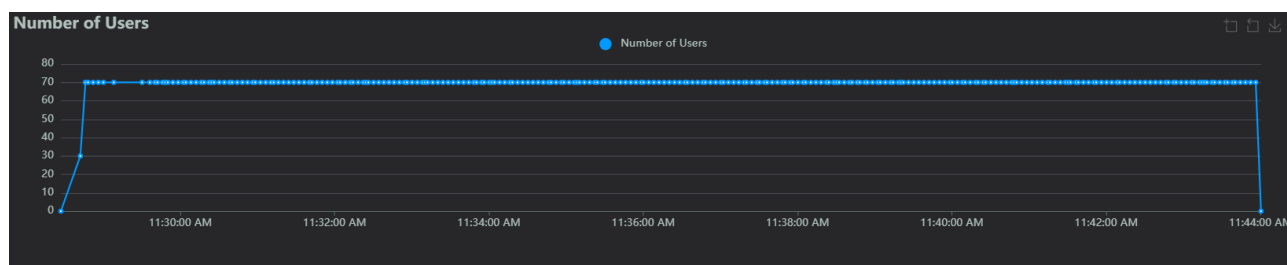


Figura 6.5: Numărul de utilizatori

Analiza timpilor de răspuns

Pentru evaluarea performanței aplicației sub sarcină s-au colectat timpii de răspuns ai tuturor cererilor, precum și ai principalelor endpoint-uri. Valorile prezentate în tabele includ mediana (50%), percentila 95 și percentila 99, oferind o imagine completă asupra performanței pentru utilizatorul tipic și pentru cazurile critice. Fiecare tabel este însoțit de o scurtă interpretare a semnificației datelor.

Rezultatele agregate pentru toate cererile

Datele agregate (tabel 6.1 arată o discrepanță mare între mediană și celelalte percentile, semnalând o performanță inconsistentă. Mediana foarte joasă (8-16 ms) arată că majoritatea cererilor simple de tip GET sunt rapide.

Totuși, valorile ridicate ale percentilelor 95 și 99, în special la începutul testului (după 2-3 minute), sugerează existența unor întârzieri semnificative pentru o parte a cererilor. Aceasta indică existența unor perioade cu creșteri temporare ale latenței care pot influența negativ calitatea serviciului pentru un număr restrâns de utilizatori. Rata de eșec de 4% susține ideea că aceste variații nu sunt întâmplătoare, ci semnalează o instabilitate a sistemului generată de suprasolicitarea resurselor.

Metrică	După 2-3 min	După 15 min	Interpretare
Mediana (50%)	16 ms	8 ms	Majoritatea cererilor sunt rapide, dar această valoare maschează problemele de instabilitate.
95%ile	180 ms	46 ms	Performanță inconsistentă pentru utilizatori.
99%ile	440 ms	120 ms	Variații mari care corelate cu eșecurile indică instabilitatea sistemului.

Tabelul 6.1: Timp de răspuns agregat pentru toate cererile

Endpoint POST /login

Acest endpoint reprezintă principala cauză a instabilității sistemului. Cu o mediană de 60 de secunde, procesul de autentificare devine practic nefuncțional sub sarcină. Cererile de autentificare generează timpi de așteptare inacceptabil de mari, blocând resursele serverului și determinând, în cele din urmă, eșecurile evidențiate în grafice.

Metrică	După 2-3 min	După 15 min	Interpretare
Mediana (50%)	60 000 ms	60 000 ms	Utilizatorii așteaptă 1 minut pentru login.
95%ile	61 000 ms	61 000 ms	Endpoint-ul este blocat și nefuncțional pentru aproape toți utilizatorii.
99%ile	61 000 ms	61 000 ms	Timp constant, dar foarte lent.

Tabelul 6.2: Timp de răspuns pentru endpoint-ul POST /login.

Endpoint POST /edit-profile

Acest endpoint, deși mai lent decât un GET simplu, se comportă rezonabil. Mediana de 8-37 ms este bună, iar percentila 99 sub 650 ms indică faptul că nu contribuie semnificativ la instabilitatea generală a sistemului care este dominată de problemele de la /login.

Metrică	După 2-3 min	După 15 min	Interpretare
Mediana (50%)	37 ms	8 ms	Majoritatea cererilor sunt procesate rapid. Scăderea este înșelătoare, cauzată de eșecul cererilor mai lente.
95%ile	440 ms	54 ms	Îmbunătățirea drastică arată că sistemul nu mai poate procesa cererile lente care eșuează și nu mai sunt contorizate.
99%ile	650 ms	380 ms	Scăderea nu reflectă o optimizare reală, ci eșecul celor mai costisitoare cereri care nu mai sunt contorizate.

Tabelul 6.3: Timp de răspuns pentru endpoint-ul POST /edit-profile.

Endpoint GET /my-reservations

Endpoint-ul GET /my-reservations are o performanță bună cu o mediană de 11-31 ms și percentila 99 sub 370 ms. Acest lucru confirmă că operațiunile de citire sunt rapide, dar performanța lor bună este umbrită de eșecurile la nivel de sistem cauzate de alte operațiuni.

Metrică	După 2-3 min	După 15 min	Interpretare
Mediana (50%)	31 ms	11 ms	Extrem de rapid
95%ile	190 ms	48 ms	Performanță excelentă pentru majoritatea utilizatorilor
99%ile	370 ms	160 ms	Chiar și cele mai lente cereri sunt procesate într-un timp rezonabil.

Tabelul 6.4: Timp de răspuns pentru GET /my-reservations.

Concluzii pentru cazul arhitecturii monolitice

Analiza datelor evidențiază performanța instabilă și nesatisfăcătoare a sistemului în configurația monolitică sub o sarcină de 70 de utilizatori. Deși majoritatea operațiilor de citire se execută rapid, sistemul este destabilizat de endpoint-ul POST /login care înregistrează timpi de răspuns extrem de mari, de aproximativ 60 de secunde. Acest blocaj duce la epuizarea resurselor serverului și generează căderi periodice ale întregului sistem, observabile prin scăderi bruște ale RPS și vârfuri de erori ce generează o rată agregată a eșecurilor de 4%.

În concluzie, testul confirmă faptul că arhitectura monolitică nu este scalabilă pentru acest tip de sarcină. Incapacitatea de a izola și gestiona operațiile costisitoare duce la o degradare a performanței la nivelul întregii aplicații, făcând-o nesigură și oferind o experiență negativă utilizatorilor.

6.4.2 Cazul 2: Arhitectura distribuită

Pentru acest scenariu cadrul de testare a fost extins pentru a simula o arhitectură distribuită folosind două instanțe Flask și un sistem de replicare Master-Slave pentru baza de date MySQL. O instanță Flask rulează pe sistemul gazdă, iar cealaltă pe VM 2. Toate cererile POST și GET sunt gestionate de cele două instanțe Flask, în timp ce fișierele statice sunt servite de IIS pe VM 2 care primește aceste fișiere de la Nginx. Nginx acționează ca load balancer și proxy invers redirecționând cererile între instanțele Flask. Această configurare permite echilibrarea sarcinii, separă clar logica aplicației de servirea resurselor statice și demonstrează scalabilitatea orizontală a stratului de aplicație.

Serverul MySQL principal (Master) și serverul secundar (Slave) au fost active, iar Nginx a direcționat cererile către instanțele corespunzătoare. Această configurație a permis simularea unui mediu realist, în care cererile utilizatorilor sunt distribuite între instanțe, iar baza de date replicată a asigurat consistența și disponibilitatea datelor. Setul de date de test a fost același ca în cazul monolitic, conținând aproximativ 100 de utilizatori, 200 de proprietăți și 1150 de unități de cazare.

Rezultatele după 2-3 minute

Testul de performanță pentru arhitectura distribuită a implicat 70 de utilizatori virtuali și a generat un total de 40 085 de cereri, toate finalizate fără niciun eșec. Debitul agregat a fost de 203,2 cereri pe secundă demonstrând stabilitatea excepțională a aplicației și capacitatea acesteia de a gestiona un volum ridicat de cereri fără erori, chiar și sub sarcină. O reprezentare vizuală a acestui debit constant fără căderi este disponibilă în Anexa 8.

Timpii de răspuns au variat în funcție de complexitatea operațiilor, într-un mod previzibil. La nivel agregat, mediana a fost de 12 ms, iar percentila 95 a atins 40 ms, indicând o experiență rapidă și constantă pentru majoritatea utilizatorilor.

Analiza performanței pe nivel de endpoint evidențiază comportamente distincte. Endpoint-urile rapide, în special cele aferente operațiilor de citire (GET), precum `/my-reservations` și `/my-properties`, au înregistrat performanțe remarcabile. Cererea GET `/my-reservations` a avut o mediana de 18 ms și o medie de 20,92 ms, în timp ce GET `/my-properties` a înregistrat o mediana de 18 ms și o medie de 20,76 ms, ceea ce indică latențe foarte scăzute pentru majoritatea utilizatorilor. Performanțe similare au fost observate și pentru resursele statice (`/static/[...]`) unde mediana timpurilor de răspuns a fost de doar 5 ms.

Endpoint-urile de scriere, mai costisitoare din punct de vedere al resurselor, au prezentat următoarele rezultate. Endpoint-ul POST `/login`, deși este cea mai solicitantă rută, a înregistrat o mediană de 270 ms, o percentilă 95 de 630 ms și o medie de 331,08 ms. Comparativ cu arhitectura monolitică, unde aceeași operație depășea 60 000 ms, aceste valori reprezintă o îmbunătățire semnificativă și nu constituie un blocaj critic.

Similar, endpoint-ul POST `/edit-profile` a avut o mediană de 22 ms, o medie de 38,83 ms și o percentilă 95 de 250 ms. Aceste rezultate reflectă o capacitate robustă a sistemului

de a gestiona operațiunile de actualizare fără impact semnificativ asupra experienței utilizatorilor.

Resursele sistemului au fost utilizate intens: CPU-ul a atins 71% utilizare cu o frecvență de 2,77 GHz, iar memoria RAM a fost folosită în proporție de 78% (12,4 GB din 15,8 GB), ceea ce indică funcționarea aproape de capacitatea maximă.

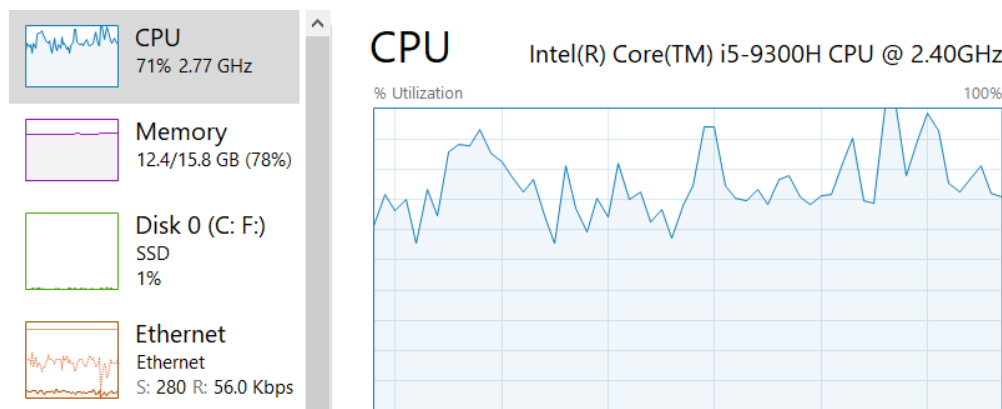


Figura 6.6: Resurse utilizate după 2-3 minute

Rezultatele după 15 minute

După 15 minute de test, sistemul distribuit a demonstrat o stabilitate remarcabilă, menținând un debit constant de 208,1 RPS fără a înregistra niciun eșec, așa cum se poate observa în Anexa 9. La nivel agregat performanța a fost excelentă cu o mediană de doar 11 ms și o percentilă 95 de 35 ms, ceea ce indică o experiență de utilizare rapidă pentru majoritatea utilizatorilor.

Analiza detaliată pe nivel de endpoint confirmă această consistență. Operațiunile de citire (GET) au fost procesate foarte rapid. Ruta `/my-reservations` a înregistrat o mediană de 18 ms, iar `/my-properties` o mediană de 18 ms. Chiar și resursele statice (`/static/[...]`) au fost livrate cu o mediană de 5 ms reflectând latențe extrem de scăzute.

În ceea ce privește operațiunile de scriere (POST), deși acestea implică un consum mai mare de resurse, performanța a rămas foarte bună. Endpoint-ul `POST /login` a prezentat o mediană de 270 ms și o percentilă 95 de 630 ms. Chiar dacă reprezintă cea mai lentă operație, aceasta nu a creat blocaje și nu a afectat stabilitatea generală a sistemului.

Interpretarea rezultatelor pe baza graficelor

Analiza performanței pentru arhitectura distribuită a urmat aceeași metodologie ca în cazul monolitic. Testul a fost monitorizat utilizând cele trei grafice principale generate de Locust: **Număr de cereri pe secundă (RPS)**, **Timpi de răspuns (ms)** și **Numărul de utilizatori**. Pentru a evalua atât comportamentul inițial, cât și stabilitatea pe termen lung s-au analizat cele două momente cheie: după aproximativ 2-3 minute de la începutul testului și după 15 minute.

Graficul pentru (RPS), prezentat în figura 6.7, evidențiază performanța robustă a arhitecturii distribuite. Rata de cereri a crescut rapid la începutul testului, atingând un vârf de

peste 200 RPS, apoi s-a menținut constant între 200 și 215 RPS pe parcursul celor 15 minute. Această consistență indică capacitatea sistemului de a procesa cereri într-un mod stabil chiar și sub sarcină ridicată. În plus, linia roșie, care reprezintă numărul de eșecuri pe secundă, rămâne la zero pe întreaga durată a testului, confirmând fiabilitatea excepțională a aplicației.

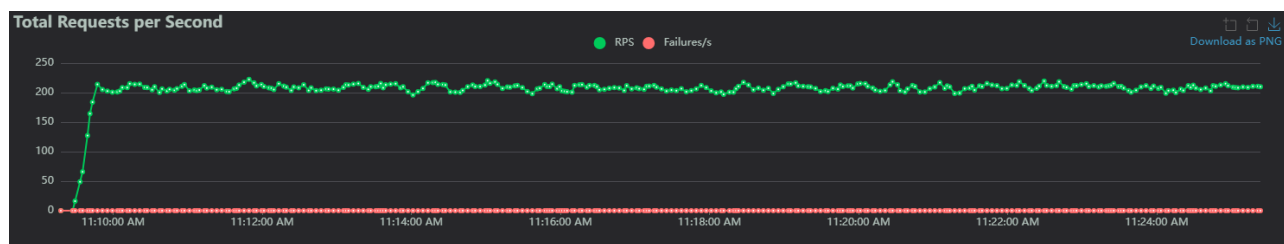


Figura 6.7: Număr de cereri pe secundă (RPS)

Graficul Timpilor de Răspuns (ms), ilustrat în figura 6.8, evidențiază latența redusă și stabilitatea sistemului. Mediana (percentila 50), reprezentată prin linia portocalie, se menține foarte scăzută, în jur de 11 și 12 ms, indicând faptul că, cel puțin jumătate dintre cereri sunt procesate aproape instantaneu. Similar, percentila 95, reprezentată prin linia violet, arată timpi la fel de stabili, în jurul valorii de 35-40 ms. Acest lucru sugerează că sistemul gestionează eficient chiar și cererile mai lente, iar după un scurt impuls inițial de cereri, performanța rămâne constantă și rapidă pe întreaga durată a testului.

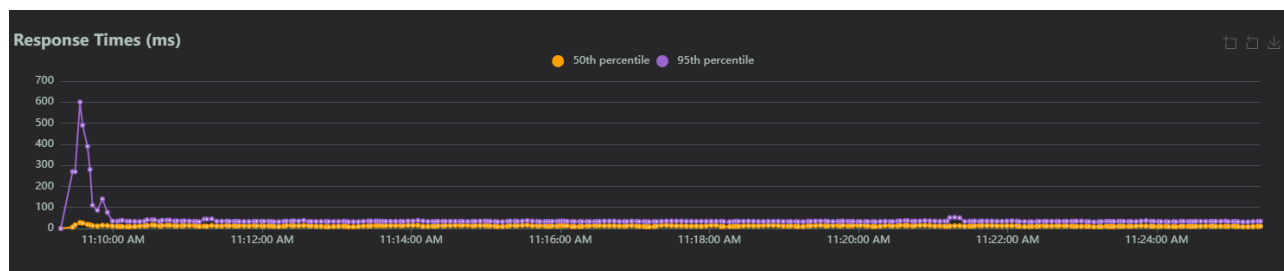


Figura 6.8: Timpuri de răspuns (ms)

Graficul Număr de Utilizatori, prezentat în figura 6.9, arată creșterea rapidă a utilizatorilor virtuali până la numărul țintă de 70, care s-a menținut constant pe durata testului. Aceasta demonstrează că sistemul distribuit poate gestiona simultan un număr semnificativ de utilizatori fără nicio degradare a performanței.

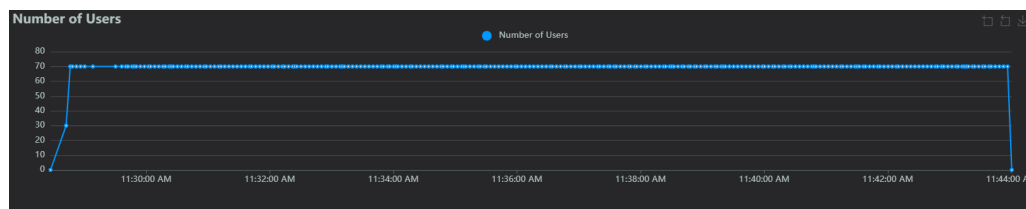


Figura 6.9: Numărul de utilizatori

Analiza timpilor de răspuns

Pentru analiza performanței aplicației am comparat timpii de răspuns pentru toate cererile agregate și pentru endpoint-urile critice folosind două seturi de teste cu durată diferită. Tabelele de mai jos prezintă valorile medii (mediana 50%), 95%-ile și 99%-ile, evidențiind atât performanța generală a aplicației, cât și punctele de blocaj specifice.

Rezultatele agregate pentru toate cererile

Analizând performanța agregată pentru toate cererile se observă că timpul median de răspuns a fost de 12 ms în primele minute și a scăzut la 11 ms după 15 minute, ceea ce indică o stabilitate și chiar o ușoară optimizare pe parcursul testului. Comparând percentila 95, timpul de răspuns a scăzut de la 40 ms la 35 ms, semnalând că majoritatea cererilor sunt procesate rapid chiar. Percentila 99 a scăzut de la 160 ms la 50 ms, demonstrând că aplicația poate gestiona eficient și solicitările lente. Sistemul a menținut performanțe consistente și rapide pe întreaga durată a testului (tabel 6.5)

Metrică	După 2-3 min	După 15 min	Interpretare
Median (50%)	12 ms	11 ms	Timpul mediu de răspuns a rămas extrem de rapid și constant, ceea ce arată stabilitatea aplicației sub sarcină moderată.
95%ile	40 ms	35 ms	Majoritatea cererilor sunt procesate rapid cu o ușoară creștere a timpului în testul mai lung, ceea ce indică o scalabilitate bună.
99%ile	160 ms	50 ms	Chiar și cele mai lente cereri au fost finalizate sub 200 ms cu o scădere minoră în timpul testului mai lung.

Tabelul 6.5: Performanța medie a cererilor pentru ambele seturi de teste

Endpoint POST /login

Tabelul 6.6 prezintă timpii de răspuns pentru endpoint-ul POST /login, considerat punctul de blocaj al aplicației. Se observă că timpul median de răspuns rămâne constant la 270 ms atât după 2-3 minute, cât și după 15 minute, ceea ce indică faptul că autentificarea utilizatorilor generează o întârziere persistentă, independentă de durata testului.

Analizând percentila 95 care are valoarea de 630 ms, se constată că majoritatea utilizatorilor întâmpină o întârziere semnificativă la autentificare. Aceasta sugerează faptul că experiența utilizatorilor poate fi afectată negativ, chiar dacă aplicația rămâne stabilă și nu apar erori.

Percentila 99, constantă la 740 ms, arată că nici cele mai lente cereri nu depășesc această valoare. Aceasta indică faptul că problema nu este generată de suprasolicitarea serverului, ci de procesul de login în sine, care include verificări complexe.

Metrică	După 2-3 min	După 15 min	Interpretare
Median (50%)	270 ms	270 ms	Timpul median de răspuns este ridicat comparativ cu alte rute, dar acceptabil pentru o operațiune de login.
95%ile	630 ms	630 ms	Majoritatea utilizatorilor așteaptă sub 0,7 secunde indicând un punct de încetinire, dar nu un blocaj.
99%ile	740 ms	740 ms	Valorile maxime rămân ridicate și constante, sugerând că problema nu este legată de sarcină, ci de procesul de login.

Tabelul 6.6: Timp de răspuns pentru POST /login

Endpoint POST /edit-profile

Se observă în Tabelul 6.7 că timpul median de răspuns rămâne stabil, între 21 ms și 22 ms, indicând o performanță constantă. Valorile 95%ile, de 46 ms și 250 ms arată că aproape toate cererile sunt procesate rapid cu o ușoară creștere pe parcursul testului mai lung. În cazul valorilor 99%ile, timpii cresc la 250–310 ms ceea ce evidențiază că doar un număr redus de cereri durează mai mult, însă chiar și acestea rămân sub 600 ms. Această distribuție indică o performanță generală eficientă cu o creștere minoră a timpilor pentru cererile mai lente în testul mai lung.

Metrică	După 2-3 min	După 15 min
Median (50%)	22 ms	21 ms
95%ile	250 ms	46 ms
99%ile	310 ms	250 ms

Tabelul 6.7: Timp de răspuns pentru cazul 2

Endpoint GET /my-reservations

Tabelul 6.8 prezintă timpii de răspuns pentru endpoint-ul GET /my-reservations, considerat unul performant. Se observă că timpul median de răspuns este extrem de scurt, 18 ms în ambele cazuri, ceea ce arată faptul că, cererile uzuale sunt procesate foarte rapid.

Percentila 95, cu valori de 42 ms și 44 ms, indică faptul că majoritatea cererilor sunt gestionate într-un interval foarte scurt.

Percentila 99, cu valori sub 61 ms, arată că și cele mai lente cereri rămân extrem de rapide. GET /my-reservations demonstrează o performanță excelentă și stabilă pe toată durata testului.

Metrică	După 2-3 min	După 15 min	Interpretare
Median (50%)	18 ms	18 ms	Performanța mediană a rămas excelentă.
95%ile	44 ms	42 ms	Chiar și 95% dintre cereri sunt procesate rapid arătând o performanță consistentă.
99%ile	61 ms	55 ms	Chiar și cele mai lente cereri au fost procesate în mai puțin de 200 ms cu o mică creștere după 15 minute.

Tabelul 6.8: Timp de răspuns pentru GET /my-reservations

Concluzii pentru cazul arhitecturii distribuite

Analiza timpilor de răspuns pentru arhitectura distribuită evidențiază o performanță stabilă, eficientă și predictibilă. Atât după 2-3 minute, cât și la finalul celor 15 minute de test, sistemul a gestionat constant sarcina generată de 70 de utilizatori simultan, fără degradări semnificative.

Performanța pe tot parcursul testului a fost remarcabilă. Conform Tabelului 6.5, mediana timpilor de răspuns a rămas foarte scăzută (11–12 ms), iar percentilele 95 și 99 au indicat valori sub 160 ms, demonstrând că majoritatea cererilor au fost procesate rapid, indiferent de tipul lor.

Operațiile rapide de citire, cum ar fi GET /my-reservations (Tabelul 6.8) și POST /edit-profile (Tabelul 6.7), au înregistrat mediane de 18 ms, respectiv 21–22 ms, indicând că fluxurile frecvente ale utilizatorilor sunt gestionate aproape instantaneu.

Endpoint-ul POST /login (Tabelul 6.6) a fost, așa cum era de așteptat, cea mai costisitoare rută, cu o mediană de 270 ms și o percentilă 99 de 740 ms. Deși reprezintă un punct de atenție, spre deosebire de arhitectura monolitică, nu a creat blocaje critice și nu a afectat stabilitatea generală a sistemului.

În concluzie, arhitectura distribuită a demonstrat o performanță robustă. Graficul timpilor de răspuns confirmă faptul că după un scurt vârf inițial de încărcare, latența s-a menținut la un nivel scăzut și constant. Deși este recomandată o optimizare viitoare a procesului de autentificare pentru a asigura o experiență uniformă, sistemul a dovedit capacitatea de a susține sarcina de testare într-un mod eficient și fiabil.

6.5 Compararea performanței între arhitectura monolitică și cea distribuită

Pentru a investiga modul în care arhitectura afectează performanța aplicației s-a realizat o comparație între două configurații distincte prezentate mai sus. Analiza s-a concentrat asupra debitului sistemului, a timpilor de răspuns pe endpoint-uri și a utilizării resurselor pe durata testelor de stres, pentru a evalua impactul arhitecturii asupra scalabilității și stabilității aplicației, precum și eficiența separării logicii aplicației de livrarea fișierelor statice prin IIS.

	Cazul 1: Monolit	Cazul 2: Două instanțe Flask cu Nginx
Număr instanțe Flask	1 instanță pe host	2 instanțe (una pe host, una pe VM 2)
Load balancing	Nu există	Nginx direcționează cererile între instanțe
Baza de date	MySQL principal pe host	Master-Slave: principal pe host, secundar pe VM 1
Servirea fișierelor statice	Direct de la Flask	Nginx redirecționează fișiere statice către IIS pe VM 2
Direcționarea cererilor	Direct către Flask	Cererile sunt distribuite între cele două instanțe

Tabelul 6.9: Configurarea mediului în cele două cazuri de testare

Rata de procesare a cererilor pe secundă (RPS)

La prima vedere ambele sisteme par să gestioneze un debit ridicat. Cu toate acestea, analiza detaliată a stabilității scoate în evidență diferențe între arhitectura monolitică și cea distribuită.

Inițial, arhitectura monolitică părea să ofere un debit mediu satisfăcător, depășind pragul de 200 RPS. O analiză detaliată a graficelor de performanță evidențiază o vulnerabilitate importantă: sistemul devine foarte instabil sub sarcină. Comportamentul său este imprevizibil, perioadele de funcționare normală fiind întrerupte brusc de scăderi majore ale performanței. În aceste momente critice, debitul scade aproape de zero, iar aplicația devine practic neutilizabilă. Aceste colapsuri coincid cu vârfuri de erori, rezultând o rată totală de eșecuri de aproximativ 4%, ceea ce înseamnă că, în medie, una din 25 de cereri eșuează. Această frecvență a erorilor compromite serios fiabilitatea aplicației și o face inadecvată pentru utilizare în producție, unde stabilitatea este esențială.

În contrast cu arhitectura monolitică, sistemul distribuit bazat pe un mecanism eficient de load balancing, a demonstrat o performanță constantă și o fiabilitate remarcabilă. Pe întreaga durată a testelor de stres, acesta a menținut un debit stabil de aproximativ 208 RPS, fără variații semnificative sau întreruperi. Niciuna dintre cele peste 194 000 de cereri nu a eșuat, rezultând o rată de eșec de 0%. Această realizare nu este doar relevantă din punct de vedere tehnic, ci evidențiază și fiabilitatea arhitecturii distribuite. Menținerea stabilității pe întreaga durată a testului și absența erorilor confirmă faptul că această abordare este adecvată și necesară

pentru aplicațiile care au nevoie de disponibilitate ridicată și capacitatea de scalare.

În concluzie, diferența de stabilitate dintre cele două arhitecturi este semnificativă: arhitectura monolitică a prezentat o performanță inconsistentă și imprevizibilă, în timp ce arhitectura distribuită a asigurat un debit constant și o fiabilitate de 100%.

Analiza timpilor de răspuns și evidențierea punctelor critice de eșec

Diferențele de performanță devin și mai evidente la analiza timpilor de răspuns, în special pentru operațiunile critice.

Tabelul 6.10 prezintă timpii de răspuns pentru endpoint-ul `POST /login`, evidențiind diferențele dramatice dintre monolit și arhitectura distribuită. În monolit, endpoint-ul reprezintă un punct de blocaj major, cu o mediană de 60 de secunde și percentilele 95 și 99 de 61 de secunde. Aceste valori indică faptul că procesul de autentificare nu este doar lent, ci practic nefuncțional sub sarcină.

Metrică	Caz 1 (Monolit)	Caz 2 (Distribuit)	Îmbunătățire
Mediana (50%)	60.000 ms	270 ms	222× mai rapid
95%ile	61.000 ms	630 ms	97× mai rapid
99%ile	61.000 ms	740 ms	82× mai rapid

Tabelul 6.10: Timp de răspuns pentru endpoint-ul `POST /login`.

În primul caz, cel al arhitecturii monolitice, login-ul a provocat un efect dramatic asupra întregului sistem. Cererea a avut un timp mediu de răspuns de aproximativ 60 de secunde, valoare care, în practică, echivalează cu un eșec funcțional. Utilizatorul se confrunta cu o întârziere inacceptabilă, iar restul sistemului devenea complet blocat. Acest comportament a fost cauzat de operațiile intensive de hashing ale parolilor, care au consumat toate resursele disponibile instanței Flask. În lipsa unui mecanism de distribuire sau izolare, cererea de login a preluat controlul, blocând procesarea altor cereri și declanșând un lanț de eșecuri.

În al doilea caz, reprezentat de arhitectura distribuită, aceeași operație de login a fost tratată mult mai eficient. Deși a rămas cea mai costisitoare dintre toate cererile analizate, timpul median de răspuns a fost de doar 270 ms, o valoare perfect acceptabilă pentru un utilizator obișnuit. Performanța semnificativ îmbunătățită se datorează prezenței unui load balancer, care a distribuit uniform cererile între cele două instanțe ale aplicației. Astfel, niciun server nu a fost suprasolicitat, iar întregul sistem a funcționat în parametrii normali, chiar și sub sarcină ridicată.

Prin comparație directă, se poate observa o îmbunătățire de peste 222 de ori în favoarea arhitecturii distribuite în ceea ce privește timpul de răspuns median al login-ului. Însă, mai important este faptul că sistemul distribuit a menținut funcționarea stabilă a aplicației, fără a afecta negativ celelalte componente sau experiența utilizatorilor.

Această analiză evidențiază clar rolul critic al designului arhitectural în comportamentul aplicației în fața cererilor solicitante. În timp ce o arhitectură monolitică poate deveni rapid un

punct de eșec, arhitectura distribuită oferă un grad înalt de reziliență și scalabilitate, esențiale pentru aplicații moderne.

Analiza agregată a tuturor cererilor

Metrică	Caz 1 15 min	Caz 2 15 min
Mediana (50%)	8 ms	11 ms
95%ile	46 ms	35 ms
99%ile	120 ms	50 ms

Tabelul 6.11: Timp de răspuns agregat pentru toate cererile

Analiza datelor prezentate în Tabelul 6.11 oferă o imagine detaliată asupra comportamentului celor două arhitecturi. La o primă vedere, ambele sisteme par să ofere performanțe bune, având timpi mediani foarte scăzuți.

Totuși, o analiză mai atentă a valorilor pentru percentilele superioare, corelată cu rata de eșec, scoate la iveală diferențe semnificative. În cazul 1, percentila 99 ajunge la 120 ms, însă această valoare este înșelătoare. Rata de eșec de 4% înseamnă că cele mai lente și problematice cereri nu sunt incluse în această statistică, deoarece au eșuat complet. Experiența reală pentru unii utilizatori a fost, așadar, mult mai inconsistentă decât sugerează cifrele.

În cazul 2, valorile sunt predictibile. Percentila 99 de sub 50 ms, susținută de o rată de eșec de 0%, înseamnă că toți utilizatorii, inclusiv cei care au prins sistemul în cel mai aglomerat moment, au avut o experiență foarte rapidă.

6.6 Concluzii

Analiza comparativă a performanței aplicației Flask în configurație monolitică și în configurație distribuită cu load balancing evidențiază diferențe semnificative între cele două arhitecturi.

Arhitectura monolitică s-a dovedit a fi instabilă și vulnerabilă în fața creșterii volumului de trafic și a cererilor costisitoare din punct de vedere al resurselor. Orice suprasolicitare a unui singur endpoint a dus la blocarea întregii aplicații, ceea ce a generat eșecuri la nivel de sistem. În practică, acest lucru înseamnă că o singură operație problematică poate compromite complet funcționalitatea sistemului.

În schimb, arhitectura distribuită a demonstrat un nivel ridicat de reziliență. Prin utilizarea unui load balancer și a mai multor instanțe de server, sistemul a reușit să izoleze și să proceseze cererile intensive, fără a afecta celelalte componente. Disponibilitatea aplicației a fost menținută constant, fără întreruperi sau degradări semnificative.

Din punct de vedere al performanței, testele au evidențiat limitările critice ale arhitecturii monolitice. În special, operația de autentificare a cauzat blocaje severe, cu timpi de răspuns de ordinul zecilor de secunde. Aceste întârzieri au fost cauzate de lipsa unui mecanism

de paralelizare, toate cererile fiind procesate secvențial de o singură instanță. Acest model de execuție a dus la o supraîncărcare rapidă, care s-a tradus prin erori și timpi mari de așteptare.

În cazul arhitecturii distribuite, aceleași operații au fost gestionate eficient, timpul mediu de răspuns pentru login fiind redus la sute de milisecunde. Cererile au fost distribuite uniform între instanțe, ceea ce a permis procesarea în paralel și evitarea blocajelor. Îmbunătățirile obținute au fost nu doar semnificative, ci și constante pe întreaga durată a testelor, confirmând superioritatea modelului distribuit în scenarii cu încărcare reală.

Diferențele dintre cele două arhitecturi se reflectă direct în experiența utilizatorului final. În cazul arhitecturii monolitice, utilizatorii se confruntă cu o experiență imprevizibilă, în care unele cereri sunt răspunse prompt, în timp ce altele se blochează sau eșuează complet. Încrederea în aplicație este afectată de această incertitudine, ceea ce o face inadecvată pentru utilizare în scenarii comerciale. În schimb, arhitectura distribuită oferă o experiență rapidă și constantă pentru toți utilizatorii, indiferent de momentul sau volumul de trafic. Toate cererile au fost procesate cu succes, fără întreruperi, iar timpii de răspuns au rămas în limite optime. Această predictibilitate este esențială în proiectarea unor aplicații web moderne, unde satisfacția utilizatorului este un criteriu de performanță esențial.

Concluzia generală

Datele experimentale obținute în urma testelor susțin ipoteza acestei lucrări. În contextul aplicațiilor web care necesită disponibilitate ridicată, scalabilitate și timp de răspuns predictibil sub sarcină, adoptarea unei arhitecturi distribuite cu mecanisme de load balancing nu este doar o optimizare opțională, ci o alegere strategică. Beneficiile sunt evidente, atât în ceea ce privește stabilitatea sistemului, cât și în capacitatea acestuia de a scala eficient odată cu creșterea numărului de utilizatori cererile.

Mai mult decât atât, arhitectura distribuită oferă o fundație solidă pentru extindere ulterioară, integrare cu alte micro servicii. Într-un ecosistem tehnologic dinamic, în care cerințele se schimbă rapid și aplicațiile trebuie să fie mereu disponibile, soluția distribuită se impune ca standard de referință pentru proiectarea sistemelor fiabile și performante.

7 Concluzii

Lucrarea a prezentat analiza și evaluarea performanței unei aplicații web Flask, comparând o arhitectură monolitică cu o arhitectură distribuită, susținută prin load balancing între două instanțe ale aplicației. Scopul principal a fost determinarea modului în care distribuirea sarcinilor influențează timpii de răspuns, debitul de cereri și utilizarea resurselor în condiții de sarcină variabilă.

Rezultatele obținute au arătat că arhitectura distribuită aduce îmbunătățiri semnificative față de sistemul monolitic. Timpul de răspuns pentru cererile critice a fost redus, debitul mediu de cereri procesate pe secundă a crescut, iar resursele disponibile au fost utilizate mai eficient, evitând supraîncărcarea unui singur server. În același timp, performanța cererilor rapide a rămas comparabilă cu cea din arhitectura monolitică, demonstrând stabilitatea și predictibilitatea sistemului.

Metodologia aplicată în cadrul lucrării s-a dovedit eficientă, permițând obținerea de date relevante privind comportamentul aplicației sub diferite scenarii de sarcină. Aceasta constituie un cadru solid pentru optimizări ulterioare, cum ar fi scalarea dinamică a instanțelor și distribuirea echilibrată a sarcinilor între servere, contribuind la creșterea robustă a performanței aplicației.

7.1 Contribuții personale

În cadrul acestui proiect, am avut un rol activ în analiza, proiectarea și evaluarea performanței aplicației web Flask. Am dezvoltat și implementat scripturi pentru generarea de sarcini variabile și pentru monitorizarea performanței aplicației, ceea ce mi-a permis să investighez comportamentul sistemului sub diferite condiții de încărcare.

Am configurat mediul distribuit, inclusiv implementarea load balancing-ului între cele două instanțe ale aplicației, și am monitorizat resursele serverelor, asigurându-mă că testele oferă rezultate relevante și reproductibile. De asemenea, am comparat performanța arhitecturii monolitice cu cea distribuită, am interpretat rezultatele obținute și am identificat avantajele și limitările fiecărei abordări.

Am redactat întreaga documentație tehnică și analiza experimentală, prezentând metodologia de testare și concluziile într-un mod clar și structurat. Aceste experiențe mi-au permis să aplic concepte teoretice într-un context practic, să îmi dezvolt abilitățile tehnice și să înțeleg mai bine modul în care optimizarea și scalarea aplicațiilor web pot fi realizate eficient.

7.2 Dificultăți în timpul lucrării

În timpul implementării, am întâmpinat dificultăți legate de rețeaua IP/ LAN privată, mașinile virtuale nu se puteau conecta între ele inițial, ceea ce a necesitat configurarea unor reguli de firewall și setarea porturilor corespunzătoare pentru comunicația între instanțe.

Configurarea arhitecturii master-slave a fost problematică, deoarece instanța slave

nu se conecta automat la master. A fost necesară ajustarea parametrilor de rețea și testarea secvențială pentru a asigura o comunicare stabilă între componente.

Dezvoltarea testelor cu Locust a fost dificilă din cauza erorilor de implementare, care făceau ca testele să nu reflecte exact scenariile de utilizare dorite, necesitând multiple ajustări și depanări pentru a obține rezultate relevante.

7.3 Cum poate fi dezvoltată lucrarea mai departe

Aplicația dezvoltată poate fi extinsă în mai multe direcții pentru a-i crește funcționalitatea și scalabilitatea. În primul rând, ar fi utilă implementarea unor funcționalități suplimentare pentru gestionarea utilizatorilor, cum ar fi autentificarea prin OAuth sau integrarea unui sistem de roluri mai complex, care să permită acces diferențiat la anumite resurse.

În continuare, aplicația poate fi optimizată pentru performanță și stabilitate. De exemplu, implementarea unui sistem de caching pentru datele frecvent accesate sau migrarea unor operațiuni costisitoare pe sarcini asincrone cu ajutorul Celery ar putea reduce timpul de răspuns și încărcarea serverului.

De asemenea, poate fi dezvoltat un front-end mai interactiv, folosind framework-uri precum React sau Vue.js, care să comunice cu Flask prin API REST sau GraphQL, astfel încât experiența utilizatorului să fie mai fluidă și mai modernă.

În ceea ce privește testarea, aplicația poate beneficia de o suită mai completă de teste automate, incluzând teste unitare, teste de integrare și teste end-to-end, pentru a asigura că funcționalitățile implementate sunt stabile și corecte.

În final, aplicația poate fi pregătită pentru implementare în cloud folosind containere Docker și Kubernetes, ceea ce permite scalarea automată în funcție de numărul de utilizatori și facilitează mentenanța pe termen lung.

Anexă



Anexa 1: Structura de directoare a aplicației Flask

```
def is_logged_in():
    return 'user_id' in session
```

Anexa 2: Funcția `is_logged_in()`

```
app.jinja_env.filters['ro_date'] = format_ro_date
```

Anexa 3: Formatarea datelor cu Jinja2

```

def get_recommendations(room_groups, requested_adult,
    requested_room_count):
    valid_combinations = []
    # Adaugăm combinații într-o listă

    # Verificăm combinațiile de minim 2 camere disponibile
    for i in range(2, max_rooms_to_check + 1):
        for combo in itertools.combinations(all_available_rooms, i):
            # Împlinirea condiției de căutare a camerelor
            if sum(room['capacity'] for room in combo) >=
requested_adult:
                valid_combinations.append(list(combo))

    recommendations = [] # Adaugăm pachetele valide

    for combo in valid_combinations:
        total_price = sum(room['price'] for room in combo)
        total_capacity = sum(room['capacity'] for room in combo)
        room_count = len(combo)

        # Scorul folosit pentru sortare
        score = (room_count != requested_room_count,
            total_capacity - requested_adult,
            total_price)

        # Creează un pachet cu detalii despre combinația de camere
        package = {"rooms": combo, "total_price": total_price, "
sort_score": score}
        recommendations.append(package)

    # Sortează recomandările după scor și returnează primele 5
    recommendations.sort(key=lambda x: x['sort_score'])
    return recommendations[:5]

```

Anexa 4: Fragmente din funcția `get_recommendations` care gestionează logica de selecție a camerelor


```
def check_server_status(host, port):
    # Verifică dacă un server este online la un host și port specificat
    sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    sock.settimeout(2)
    try:
        # Încercarea de conectare la server
        result = sock.connect_ex((host, port))
        if result == 0:
            return '<span style="color: green;">Online</span>'
        else:
            return '<span style="color: red;">Offline</span>'
    except socket.gaierror:
        return '<span style="color: orange;">IP Invalid</span>'
    finally:
        sock.close()
```

Anexa 5: Funcția `check_server_status` care verifică starea unui server.



Host

http://192.168.50.1

Status

RUNNING

Users

70

RPS

191

Failures

0%

EDIT

STOP

RESET

STATISTICS

CHARTS

FAILURES

EXCEPTIONS

CURRENT RATIO

DOWNLOAD DATA

LOGS

LOCUST CLOUD

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
POST	/add_property	341	1	34	230	650	588.09	6	60098	1199.01	3.7	0
POST	/edit_profile	1085	0	37	440	650	89.43	6	978	1311.1	10.5	0
POST	/edit_property/[id]	9	0	73	170	170	84.62	56	174	2961.78	0	0
GET	/home	1699	1	26	170	290	211.05	5	60049	8042.38	17	0
POST	/login	70	28	60000	61000	61000	42921.02	249	60717	5068.34	0	0
GET	/logout	367	0	47	260	600	82.39	9	1023	8050.26	3.8	0
GET	/my-properties	341	0	26	150	370	49.28	5	780	1175.51	3.7	0
GET	/my-reservations	1697	0	31	190	370	154.37	5	60228	1155.46	16.9	0
GET	/profile	1086	0	32	260	910	611.47	5	60212	1316.04	10.6	0
GET	/property/[id]	696	0	41	160	330	230.69	8	60324	7165.25	6.2	0
POST	/search_results	2392	3	29	140	320	121.17	6	60010	8784.21	23.1	0
GET	/static/[...]	9749	0	6	110	270	22.55	2	666	2645.1	95.5	0
	Aggregated	19532	33	16	180	440	271.59	2	60717	3809.55	191	0

Anexa 6: Interfața Locust afișând rezultatele testelor după 2-3 minute - cazul 1

LOCUST		Host http://192.168.50.1						Status RUNNING	Users 70	RPS 217.2	Failures 4%	EDIT	STOP	RESET
STATISTICS		CHARTS	FAILURES	EXCEPTIONS	CURRENT RATIO	DOWNLOAD DATA	LOGS	LOCUST CLOUD						
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s		
POST	/add_property	3011	66	12	50	180	85.68	1	60098	1033.76	3.3	0		
POST	/booking/confirm [POST]	2587	1	10	37	47	14.88	4	204	1033.66	3.8	0		
POST	/edit_profile	9104	4021	8	54	380	31.35	1	10201	700.42	11.4	5.6		
POST	/edit_property/[id]	9	0	73	170	170	84.62	56	174	2961.78	0	0		
GET	/home	15102	423	23	52	130	57.65	1	60049	9030.23	20	0		
POST	/login	70	28	60000	61000	61000	42921.02	249	60717	5068.34	0	0		
GET	/logout	2950	91	32	78	250	73.41	1	15825	9073.77	2.9	0		
GET	/my-properties	3010	65	11	46	120	18.77	1	780	1031.09	3.3	0		
GET	/my-reservations	15102	427	11	48	160	48.13	1	60228	1022.85	20.2	0		
GET	/profile	9104	244	12	50	230	113.6	1	60212	1044.14	11.4	0		
GET	/property/[id]	5972	0	15	54	140	56.55	4	60324	7009.86	7.5	0		
POST	/search_results	21213	569	25	56	110	41.65	1	60010	9711.13	27.6	0		
GET	/static/[...]	81299	0	4	27	58	8.77	1	666	2603.62	105.8	0		
Aggregated		168533	5935	8	46	120	50	1	60717	3935.61	217.2	5.6		

Anexa 7: Interfața Locust afișând rezultatele testelor după 15 minute - cazul 1

LOCUST		Host http://192.168.50.1						Status RUNNING	Users 70	RPS 203.2	Failures 0%	EDIT	STOP	RESET
STATISTICS		CHARTS	FAILURES	EXCEPTIONS	CURRENT RATIO	DOWNLOAD DATA	LOGS	LOCUST CLOUD						
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s		
POST	/add_property	747	0	19	48	99	22.34	4	222	1172.94	4	0		
POST	/edit_profile	2115	0	22	250	310	38.83	4	702	1228.88	10.6	0		
POST	/edit_property/[id]	19	0	45	67	67	45.39	17	67	3113.84	0	0		
GET	/home	3567	0	10	33	70	16.13	4	437	8024.92	17.6	0		
POST	/login	70	0	270	630	740	331.08	235	742	8300.29	0	0		
GET	/logout	707	0	23	57	140	28.16	7	350	8042.66	3.4	0		
GET	/my-properties	747	0	18	46	58	20.76	4	162	1149.12	4	0		
GET	/my-reservations	3567	0	18	44	61	20.92	3	378	1127.46	17.6	0		
GET	/profile	2116	0	17	48	130	21.1	3	387	1228.79	10.6	0		
GET	/property/[id]	1410	0	22	47	76	24.05	6	178	7100.55	8	0		
POST	/search_results	4978	0	15	38	56	18.55	5	289	8063.89	25.8	0		
GET	/static/[...]	20042	0	5	29	120	13.85	1	934	2629.61	101.6	0		
Aggregated		40085	0	12	40	160	18.44	1	934	3711.19	203.2	0		

Anexa 8: Interfața Locust afișând rezultatele testelor după 2-3 minute - cazul 2

LOCUST							Host http://192.168.50.1	Status RUNNING	Users 70	RPS 208.1	Failures 0%	EDIT	STOP	RESET
STATISTICS		CHARTS	FAILURES	EXCEPTIONS	CURRENT RATIO	DOWNLOAD DATA	LOGS	LOCUST CLOUD						
Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s		
POST	/add_property	3489	0	18	42	55	19.97	4	222	1063.75	3.4	0		
POST	/edit_profile	10381	0	21	46	250	24.84	4	702	1073.7	10	0		
POST	/edit_property/[id]	19	0	45	67	67	45.39	17	67	3113.84	0	0		
GET	/home	17375	0	10	32	43	14.82	4	437	7986.86	18.6	0		
POST	/login	70	0	270	630	740	331.08	235	742	8300.29	0	0		
GET	/logout	3468	0	23	49	64	25.77	7	350	8042.3	3	0		
GET	/my-properties	3489	0	18	43	54	20.06	4	162	1058.65	3.4	0		
GET	/my-reservations	17375	0	18	42	55	20.21	3	378	1053.19	18.6	0		
GET	/profile	10381	0	17	40	53	19.33	3	387	1073.7	10	0		
GET	/property/[id]	6980	0	21	45	55	22.57	6	263	7113.48	9	0		
POST	/search_results	24355	0	14	35	48	17.63	5	289	7907.09	27.9	0		
GET	/static/[...]	97382	0	5	27	39	10.93	1	934	2601.78	104.2	0		
Aggregated		194764	0	11	35	50	15.26	1	934	3650.03	208.1	0		

Anexa 9: Interfața Locust afișând rezultatele testelor după 15 minute - cazul 2

Bibliografie

- [1] Yuliia Podorozhko, *Scalability in Software Development: A 101 Guide for Businesses*, Accesat: 09 mai 2025, URL: <https://madappgang.com/blog/scalability-in-software-development/#:~:text=Scalability%20in%20software%20development%20refers,users%2C%20customers%2C%20or%20requests..>
- [2] Kirill Stashevsky Nadejda Alkhaldi, *What is software scalability, and why should your company take it seriously?*, Accesat: 08 mai 2025, URL: <https://itrexgroup.com/blog/what-is-software-scalability>.
- [3] LinkedIn - Computer Engineering, *What is the best way to ensure hardware scalability in a computer system?*, Accesat: 08 mai 2025, URL: <https://www.linkedin.com/advice/1/what-best-way-ensure-hardware-scalability-2pf1e>.
- [4] AWS, *What's the Difference Between Monolithic and Microservices Architecture?*, Accesat: 20 mai 2025, URL: <https://aws.amazon.com/compare/the-difference-between-monolithic-and-microservices-architecture/>.
- [5] Microsoft Azure, *N-tier architecture style*, Accesat: 21 mai 2025, URL: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/n-tier>.
- [6] Developer Mozilla, *HTTP documentation*, Accesat: 09 mai 2025, URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP>.
- [7] Developer Mozilla, *User-Agent Header Documentation*, Accesat: 09 mai 2025, URL: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/User-Agent>.
- [8] Developer Mozilla, *What is a Web Server?*, Accesat: 09 mai 2025, URL: https://developer.mozilla.org/en-US/docs/Learn_web_development/Howto/Web_mechanics/What_is_a_web_server.
- [9] Developer Mozilla, *What is a URL?*, Accesat: 09 mai 2025, URL: https://developer.mozilla.org/en-US/docs/Learn_web_development/Howto/Web_mechanics/What_is_a_URL.
- [10] Tutorialspoint, *HTTP - Requests*, Accesat: 10 mai 2025, URL: https://www.tutorialspoint.com/http/http_requests.htm.
- [11] Tutorialspoint, *HTTP - Responses*, Accesat: 12 mai 2025, URL: https://www.tutorialspoint.com/http/http_responses.htm.
- [12] Flask Documentation, *Flask*, Accesat: 12 mai 2025, URL: <https://flask.palletsprojects.com/en/stable/>.
- [13] Wikipedia, *Flask (web framework)*, Accesat: 12 mai 2025, URL: [https://en.wikipedia.org/wiki/Flask_\(web_framework\)](https://en.wikipedia.org/wiki/Flask_(web_framework)).
- [14] S. Sudarshan Abraham Silberschatz Henry Korth, *Database System Concepts*, McGraw Hill, 2010, ISBN: 9780073523323.
- [15] Marcus Pinto Dafydd Stuttard, *The Web Application Hacker's Handbook and Lab Manual*, Wiley, 2011, ISBN: 9781118026472.
- [16] Microsoft Learn, *Internet Information Services (IIS) Overview*, Accesat: 22 mai 2025, URL: <https://learn.microsoft.com/en-us/iis/>.
- [17] NGINX documentation, *nginx documentation*, Accesat: 21 mai 2025, URL: <https://nginx.org/en/docs/>.